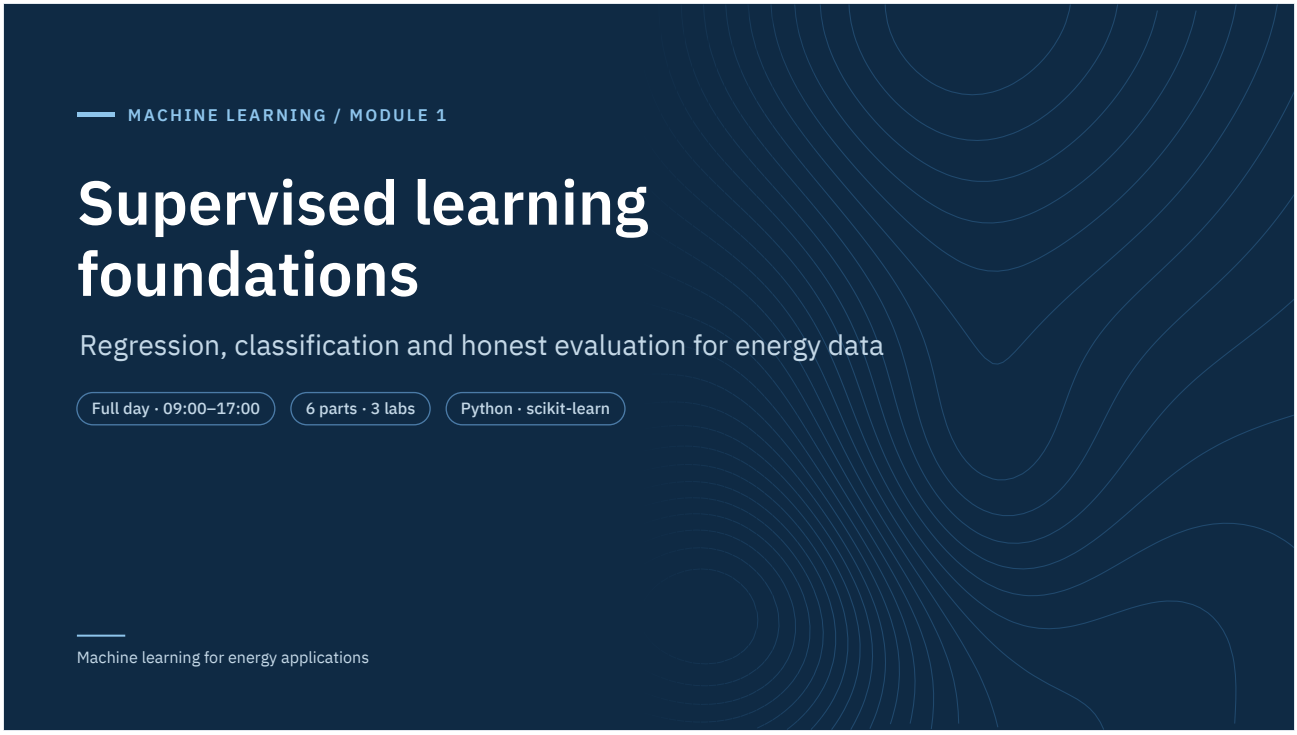
Supervised learning foundations



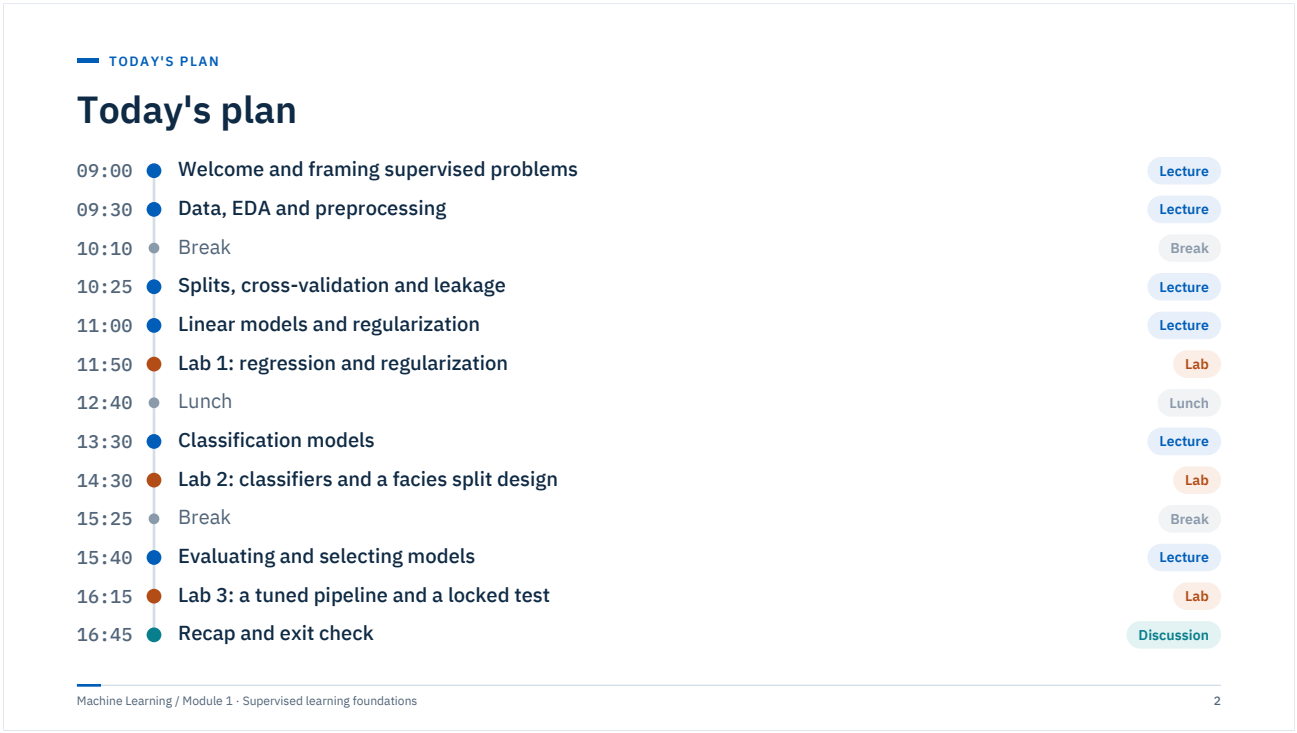
Audience: engineers, geoscientists and analysts who can read Python and basic algebra. This first day of the machine learning course sets up the vocabulary, the workflow and the evaluation habits used all week: a written prediction contract, a split that matches deployment, preprocessing fitted inside the training fold, and a baseline on identical rows.

Every plot in this deck is either a computed example on clearly labelled synthetic data or a labelled schematic. None of the numbers describe field performance. The labs use the notebooks in this module's exercises folder.

Opening question [1 min]: ask two or three people to name one prediction they would like to make in their own work, and when its inputs become available. Write the answers on a flip chart; we return to them when we write prediction contracts.

MACHINE LEARNING / MODULE 1 · SLIDE 2 OF 95

Today's plan

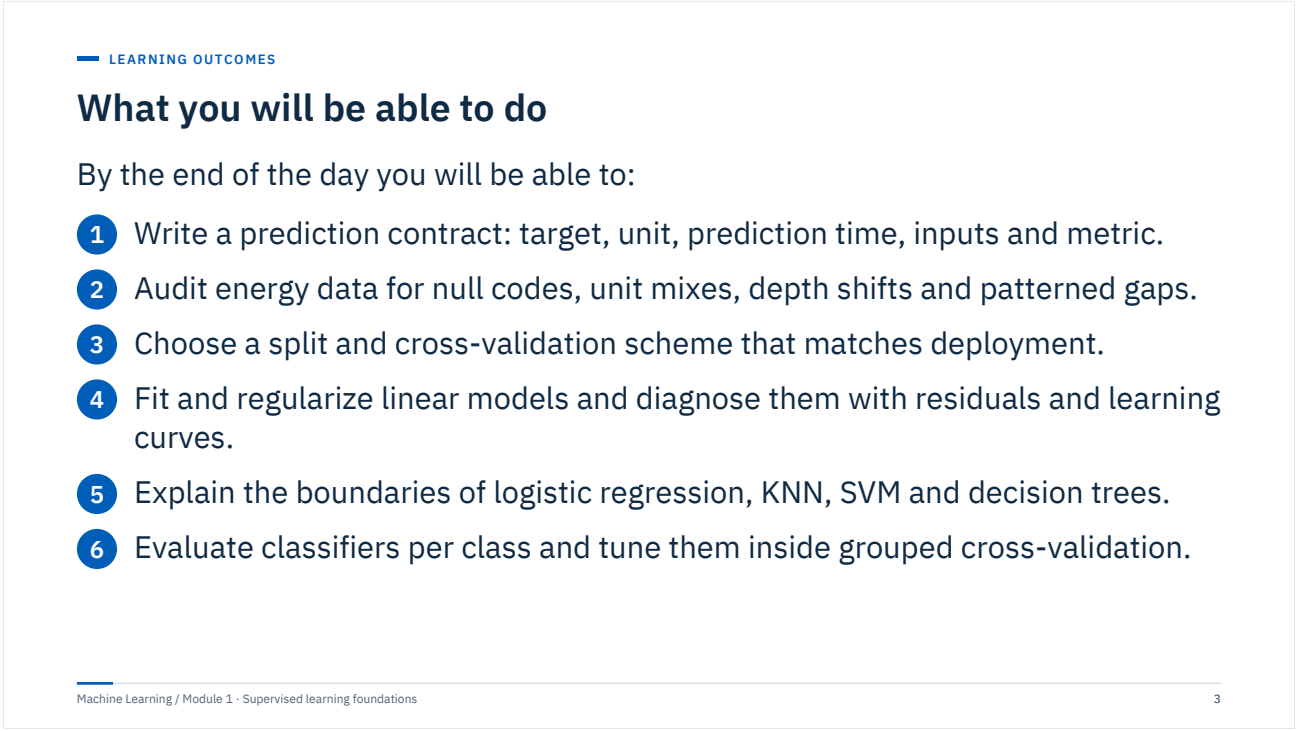


Walk through the day in one minute. Six teaching parts, each ending with a check question, and three lab blocks placed right after the material they practise. Lab 1 follows regression, Lab 2 follows the classification models, and Lab 3 closes the evaluation part with a tuned pipeline and a locked final test.

Timing guide: about 3 minutes per content slide and 4–5 minutes per exercise. If you run late, the safest places to compress are the distance slide in part 5 and the model-comparison table in part 6; do not cut the leakage exercise or the lab debriefs.

Lab 1 needs the California housing data from scikit-learn. It downloads on first use, so run the loading cell on every classroom machine before the day or provide a cached copy. The other notebooks use local files or bundled datasets.

What you will be able to do



Read the outcomes aloud and point out that each one is observable: learners will produce a contract, an audit table, a split design, diagnostic plots, a boundary explanation and a grouped tuning run during the day.

Assessment: each team submits its lab evidence (score tables with the baseline first, one diagnostic plot, one written split plan). Success means a defensible comparison, not the highest score in the room.

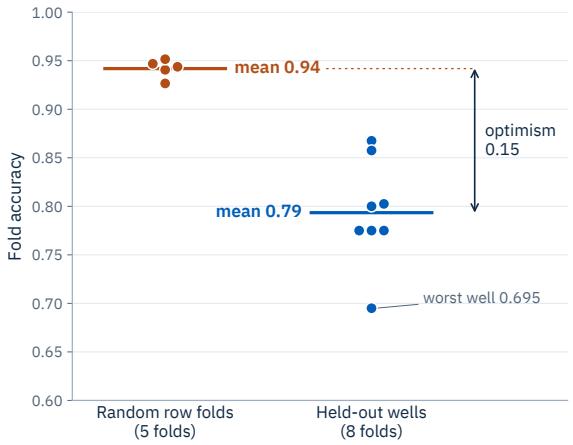
Ask: which of these outcomes feels least familiar? Most groups say "a split that matches deployment" or "patterned gaps". Promise that both get a worked example and a lab task today. [2 min]

Same model, same wells, two very different scores

MOTIVATING CASE

Same model, same wells, two very different scores

- A 1-nearest-neighbour facies model: 8 synthetic wells, 5 logs
- Random row folds: mean accuracy 0.94
- Held-out wells: mean 0.79; the worst well 0.695
- Which number belongs in the report to the asset team?



Computed on a synthetic 8-well dataset (seed 7); standardized 1-NN classifier. Not field performance.

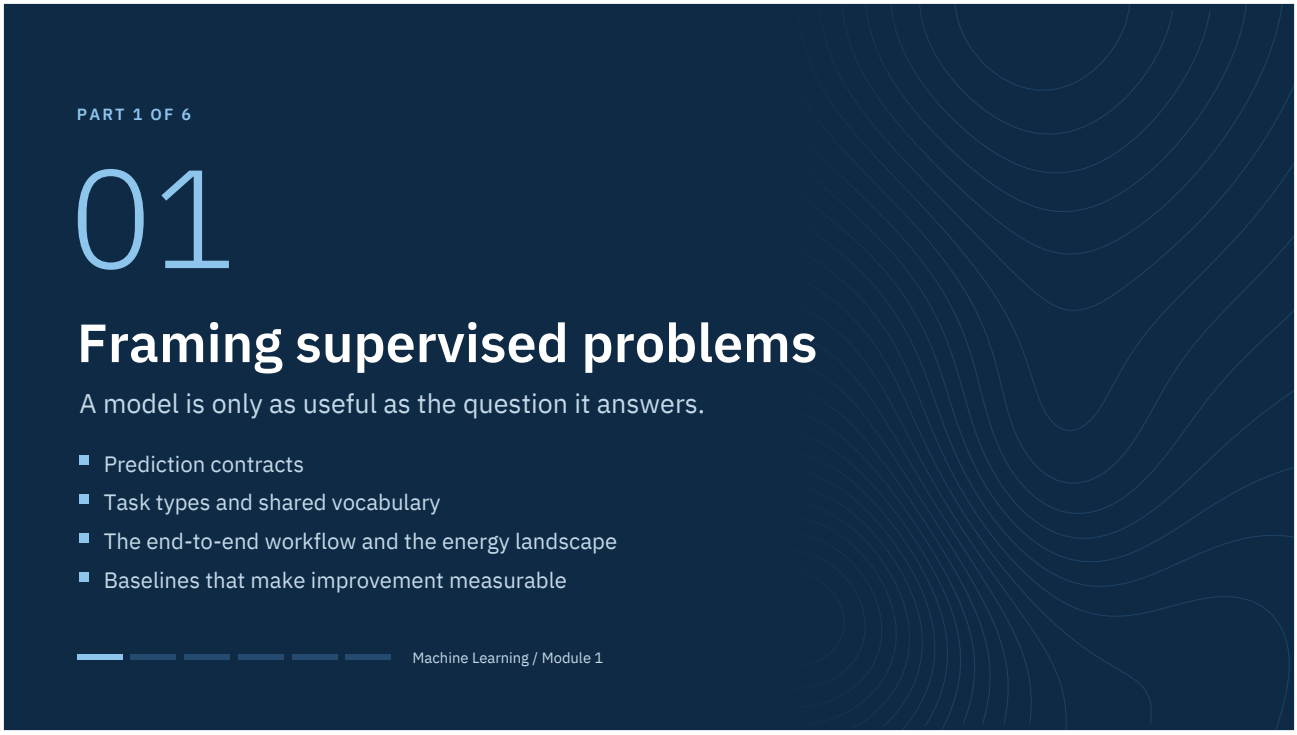
Machine Learning / Module 1 · Supervised learning foundations 4

Tell the story before explaining it. A team builds a facies classifier and reports 94% accuracy from five-fold cross-validation. The same model, evaluated by holding out one whole well at a time, averages 79%, and one well drops to 69.5%.

Ask the room which number they would report, and why the two differ [2 min]. Collect guesses. The answer we will build during the day: neighbouring depth samples are about 0.15 m apart and nearly identical. Random folds put the neighbours of every validation row into training, so a nearest-neighbour model effectively looks the answer up. Holding out whole wells asks the deployment question: how does the model do on a well it has never seen?

Keep the caveat explicit: this is synthetic data built with correlated depth samples and well-to-well calibration offsets. The size of the gap is not a claim about any real field; the mechanism is general. Transition: before any model, we need a precise question.

Framing supervised problems



Part 1 runs about 25 minutes. The goal is a shared language for the rest of the week: what a supervised problem is, how to write it down so that leakage becomes visible, and which baseline any model must beat.

Ask the room to keep their own example from the opening question in mind; they will frame it in the exercise at the end of this part.

A prediction needs a contract

PRACTICE

3 min Pairs

A prediction needs a contract

An operator wants to predict poor production.

1. What exactly is the target, and over what future interval?
2. Is one example a depth sample, a well, or a well-day?
3. Which measurements exist at the moment of prediction?

Machine Learning / Module 1 · Supervised learning foundations 6

Give pairs 3 minutes, then take two answers. A defensible example: the target is next-day oil rate per well in m^3/d, one example is one well-day, and the prediction is made at midnight using everything recorded before midnight.

Push on the third question. Ask why next-day measured pressure cannot be an input: it is not known at midnight. If a group proposes "poor production" as a class, ask for the threshold and who set it; a class defined by an arbitrary cut of a numeric target throws information away and should be justified by the decision it supports.

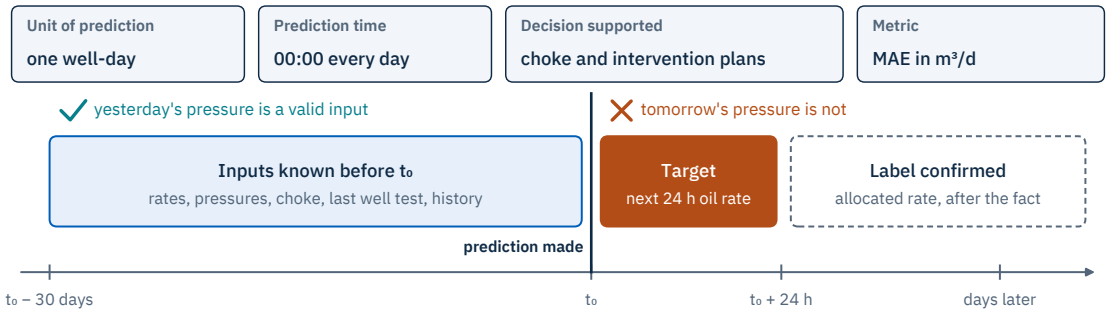
Accept alternatives with explicit timing and units, such as monthly decline per well or a weekly flag for wells below plan. Common error: describing the model ("a neural network") instead of the question. Transition: the next slide shows one complete contract as a timeline.

A contract fixes target, timing, inputs and action

FRAMING SUPERVISED PROBLEMS

A contract fixes target, timing, inputs and action

One worked answer: next-day oil rate per well, predicted at midnight.



Schematic prediction contract. The same template applies to any energy forecast or classification.

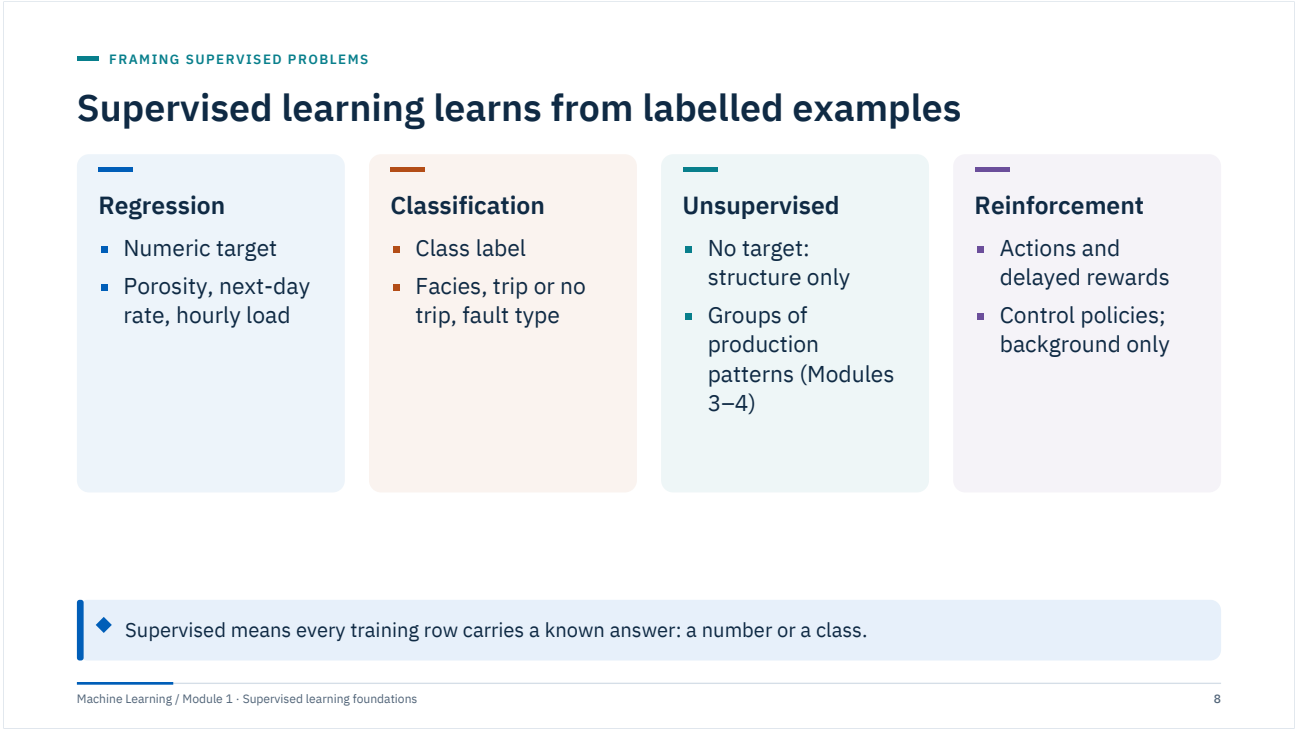
Machine Learning / Module 1 · Supervised learning foundations 7

Walk the timeline from left to right. The blue window holds everything known before the prediction time t_0: rates, pressures, choke settings, the last well test and the operating history. The orange box is the target: the oil rate over the next 24 hours. The dashed box is when the label is confirmed, often days later after allocation.

Two consequences. First, any input must exist, unchanged, before t_0; yesterday's pressure is valid, tomorrow's is not. Second, label latency matters for monitoring: you cannot score today's prediction until the allocated rate arrives.

The four boxes at the top complete the contract: the unit of one prediction, the prediction time, the decision it supports and the metric in physical units. Ask: which of these four is most often missing from project briefs you have seen? Usually the decision. [3 min]

Supervised learning learns from labelled examples



Supervised learning fits a function from inputs to a known target using examples where the answer is recorded. Regression predicts a number; classification predicts a class, usually with a probability for each class.

Unsupervised learning has no target. Clustering and dimensionality reduction, covered in Modules 3 and 4, find structure, but a cluster is not automatically a geological facies. Reinforcement learning learns actions from delayed rewards, typically in simulation; it is context only and has no lab in this course.

Context for the vocabulary: artificial intelligence is the broad field, machine learning learns patterns from data, and deep learning uses multilayer neural networks (the second course).

Ask: is predicting facies from logs regression or classification? Classification. Predicting porosity from logs? Regression. [2 min]

The vocabulary we will use all week



These terms are used identically in both courses, so fix them now.

A feature is any input that is available at prediction time; the contract decides that, not the spreadsheet. The target is what we predict. An observation is one row; a group is a set of rows that share a source, such as all depth samples from one well or all days from one pump. Groups matter because rows inside a group are not independent.

Training data fit parameters, validation data choose between settings and models, and the test data give one final estimate for the locked workflow. The well counts in the table are an example, not a rule.

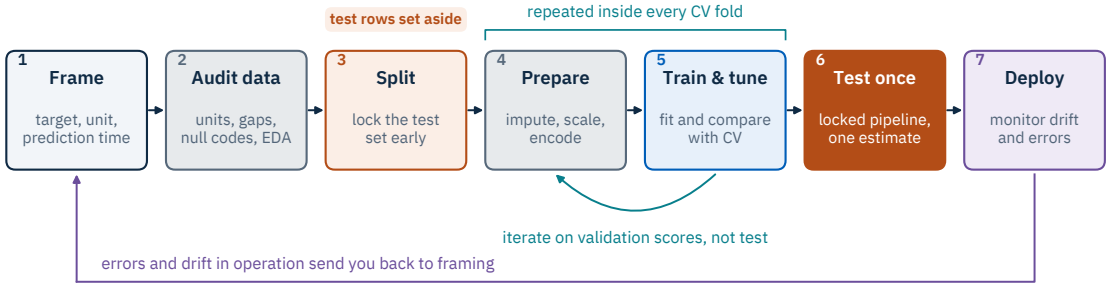
A baseline is the simplest reference any model must beat on the same rows and metric. Leakage is any route by which information that would not exist at prediction time reaches the model. [2 min]

The workflow is a loop with the test set locked early

FRAMING SUPERVISED PROBLEMS

The workflow is a loop with the test set locked early

Steps 4–5 repeat inside cross-validation; step 6 happens once per locked pipeline.



Schematic supervised-learning workflow. Colours: navy framing, grey data steps, blue model fitting, orange test data, purple operation.

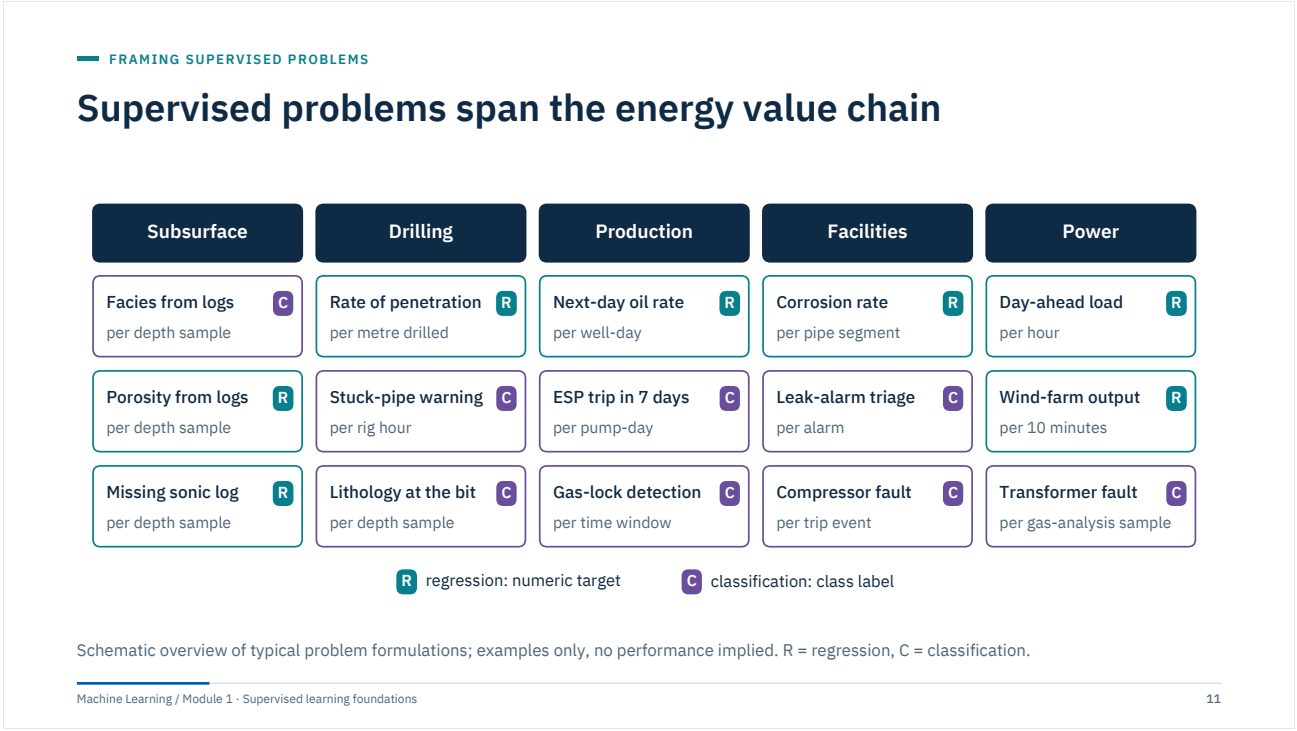
Machine Learning / Module 1 · Supervised learning foundations 10

Walk the seven steps. Framing produces the contract. Auditing checks units, gaps, null codes and distributions. The split comes early: decide which wells or periods are the test set before you look for patterns, so that exploration cannot quietly tune to them.

Preparation and model fitting repeat inside every cross-validation fold, and all comparisons use validation scores. The test set is opened once, for the locked pipeline. Deployment is not the end: errors and drift in operation send you back to framing, often because the population or the question has changed.

Ask: where in this loop do you think most project time goes? Accept answers; many practitioners report that framing, data access and auditing dominate, but make no numerical claim. [3 min]

Supervised problems span the energy value chain

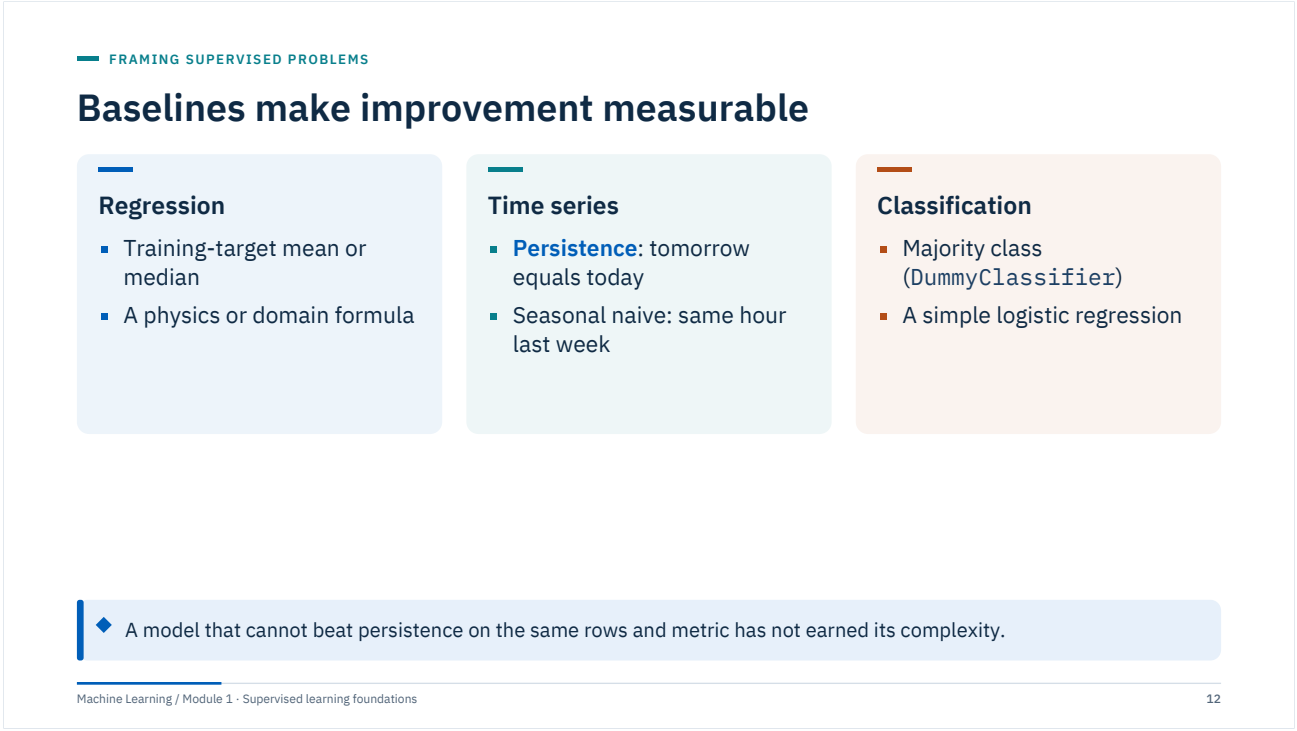


Use this map to connect the day to learners' own work. Each card is a problem formulation, not a product: the grey line gives the unit of one prediction, which already hints at the split. Depth-sample problems need grouping by well; per-hour and per-day problems need time-ordered evaluation.

Point out that the same field often supports both task types. Production gives a regression target (next-day rate) and a classification target (a pump trip within seven days). The choice depends on the decision, not on the data.

Ask two learners to place their opening-question example on the map and to name its unit. Mention that later modules revisit several of these: anomalies and rare events in Module 5, sequences in Neural Networks Day 5. [2 min]

Baselines make improvement measurable



A baseline turns a score into evidence. Without one, "RMSE 12 m³/d" means nothing: it might be excellent or worse than repeating yesterday's value.

Rules for baselines: fit their parameters on training data only (the training mean, not the overall mean); evaluate them on exactly the same rows and metric as every candidate; and record runtime and failure cases next to the score. A domain formula, such as porosity from density with a known matrix density, is often a strong and interpretable baseline.

Ask: what does an expensive nonlinear model prove if it cannot beat persistence? Nothing useful for the decision, and it adds maintenance cost. A simple baseline can be operationally valuable on its own. In scikit-learn, DummyRegressor and DummyClassifier give the trivial baselines in one line. [2 min]

Frame five energy problems

PRACTICE

5 min Pairs

Frame five energy problems

Problem	Task type	One row is	Baseline
Oil rate next month	?	?	?
ESP trip within 7 days	?	?	?
Facies from logs	?	?	?
Day-ahead hourly load	?	?	?
Corrosion rate by segment	?	?	?

1 Fill the table, then name one input for each problem that would leak.

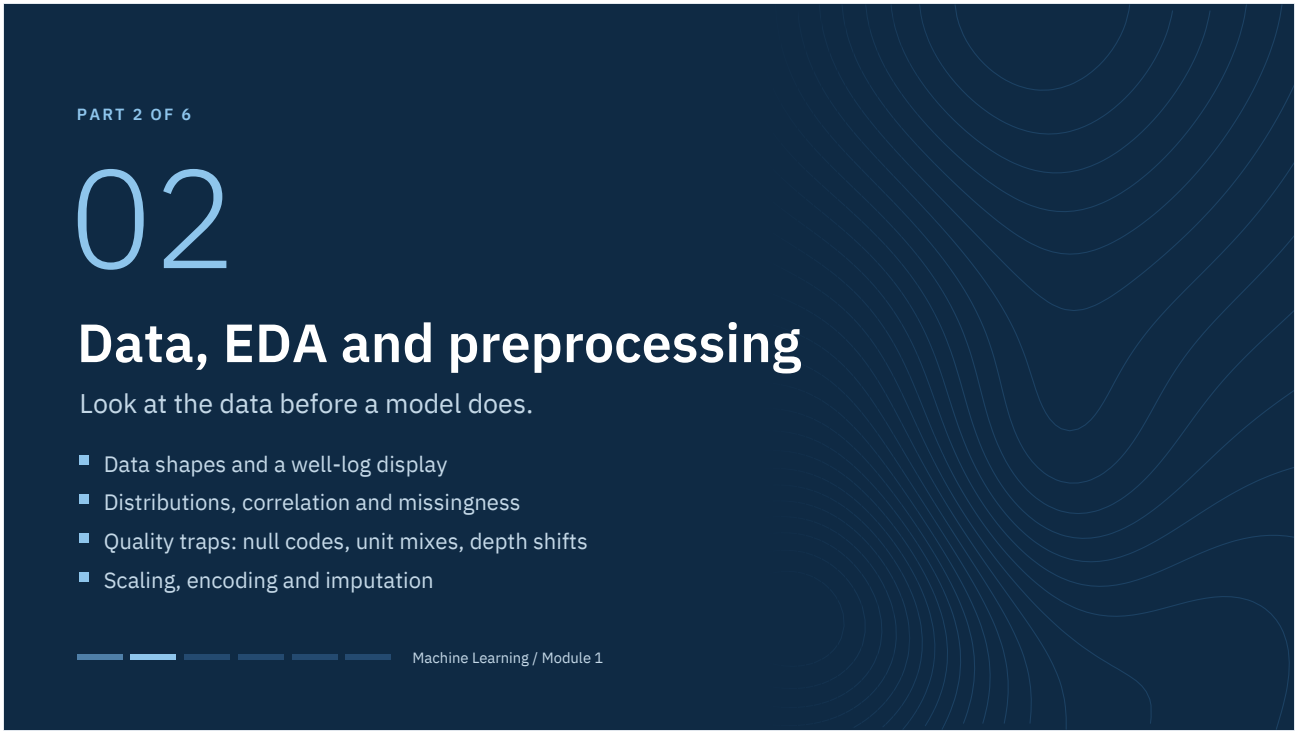
Machine Learning / Module 1 · Supervised learning foundations 13

Give pairs 4 minutes, then collect one row per pair [5 min total].

Worked answers. Oil rate next month: regression; one row is a well-month; baseline persistence or a decline-curve forecast; leaky input: an allocation revised after the month. ESP trip within 7 days: classification; one row is a pump-day; baseline majority class (never alert) or a simple rule on motor temperature; leaky input: a maintenance ticket opened after the trip. Facies from logs: classification; one row is a depth sample, grouped by well; baseline majority facies; leaky input: an interpreted curve derived from the facies labels. Day-ahead hourly load: regression; one row is an hour; baseline same hour last week; leaky input: actual temperature for the forecast day instead of the forecast temperature. Corrosion rate by segment: regression; one row is a segment-inspection; baseline segment history mean; leaky input: a repair flag recorded because of the measured rate.

Common error: naming the model instead of the unit. Transition to part 2: the unit and the timing decide how we audit and split the data.

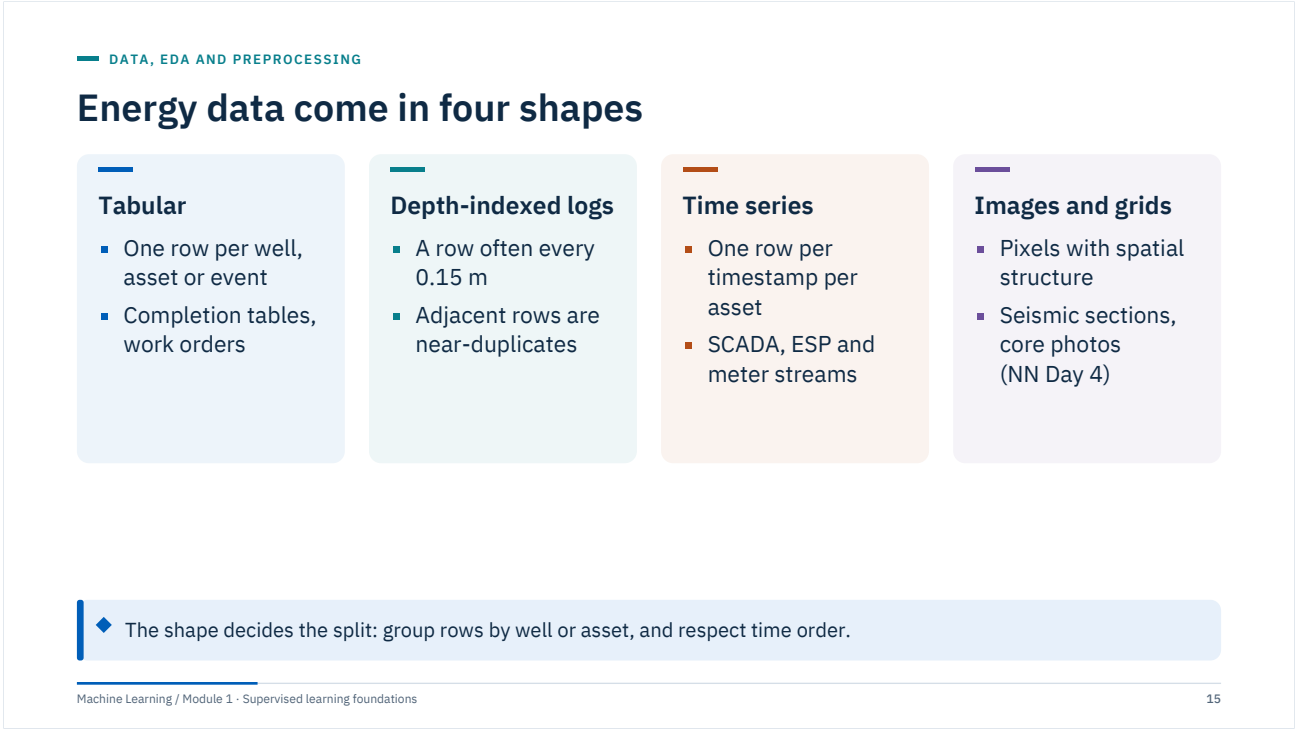
Data, EDA and preprocessing



Part 2 runs about 40 minutes. Exploratory data analysis (EDA) has two jobs: understand the structure that will drive the split and the model, and find data-quality problems before they become model behaviour.

All EDA figures use one synthetic 8-well dataset with five facies, built so that its problems are known. That lets us check every claim. The lab dataset in Lab 2 has several of the same problems in real form.

Energy data come in four shapes



Most energy datasets combine these shapes. Tabular data have one row per entity or event, like a completion table. Depth-indexed logs have one row per depth sample, often every half foot (about 0.15 m); adjacent samples see almost the same rock, so they are strongly correlated. Time series have one row per timestamp per asset; order matters, and the future must not leak into the past. Images and grids, such as seismic sections, are covered in the neural network course (Day 4).

The practical point is on the callout: the data shape tells you what "independent" means. Rows from the same well or the same pump are not independent, and random row splits overstate performance, as the motivating case showed.

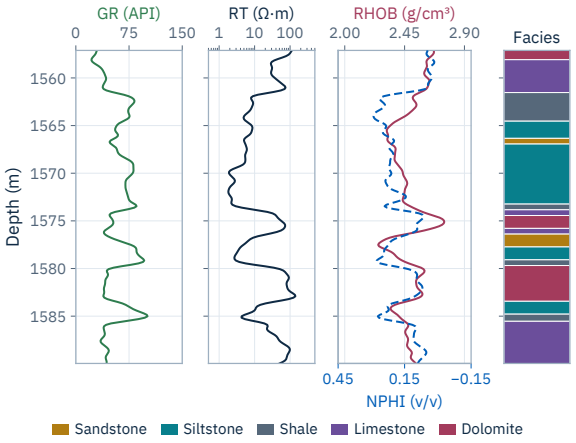
Ask: which shape does your opening-question example have? Usually a time series or depth logs. [2 min]

A well log is a depth-indexed table

DATA, EDA AND PREPROCESSING

A well log is a depth-indexed table

- Each depth sample is one row; each curve is a feature
- Facies from core or interpretation is the target
- Resistivity spans decades, so plot and model $\log_{10}$ RT
- Density and neutron both respond to porosity



Synthetic well W2 (seed 7): 33 m at 0.15 m sampling, generated from facies-dependent distributions. Not a real well.

Machine Learning / Module 1 · Supervised learning foundations 16

Read the display the way a petrophysicist would. Depth increases downward. Gamma ray (GR, in API units) is high in shaly rock and low in clean sandstone and carbonate. Resistivity (RT, ohm-metres) is on a logarithmic axis because it spans orders of magnitude. Bulk density (RHOB, g/cm³, red) and neutron porosity (NPHI, v/v, dashed blue, plotted right to left) share the third track, a standard display. The facies column is the target.

For machine learning, this is simply a table: one row per depth sample, one column per curve, plus the facies label and a well identifier. Two features of the table matter: neighbouring rows are near-copies, and log values can shift between wells because of tools, borehole conditions and calibration.

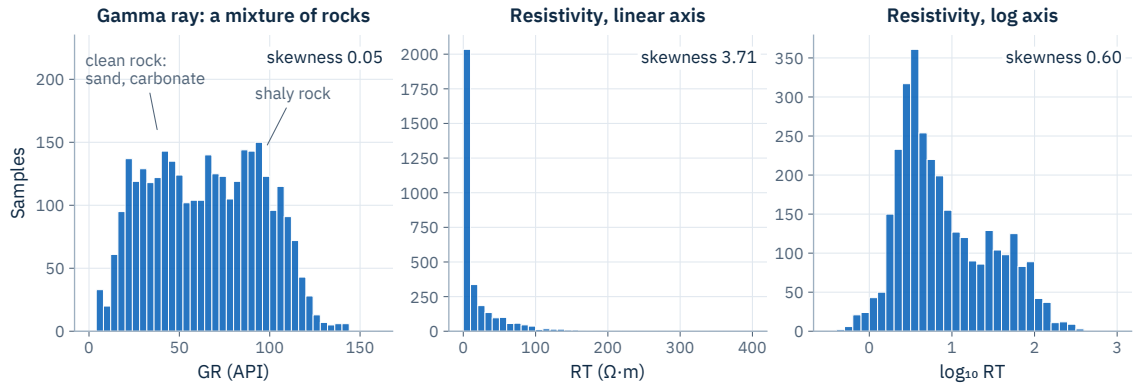
Ask: which curve would you transform before a linear model or KNN? Resistivity, to log10. [3 min]

Distributions reveal mixtures and skew

DATA, EDA AND PREPROCESSING

Distributions reveal mixtures and skew

Plot every feature before fitting: mixtures hint at classes, skew hints at a transform.



Computed on the synthetic 8-well dataset (3,200 samples, seed 7); skewness from scipy.stats.skew.

Machine Learning / Module 1 · Supervised learning foundations 17

Left: gamma ray is a mixture. The low-GR hump comes from clean rock (sandstone and carbonates) and the high-GR hump from shaly rock. A mixture tells you the target classes are visible in this feature, and that a single mean or standard deviation describes no real rock.

Middle and right: resistivity on a linear axis has a skewness of 3.71; most samples crowd near zero and a long tail dominates the variance. On a log10 axis the skewness drops to 0.60. Linear models, KNN distances and standard scaling all behave better on the transformed feature.

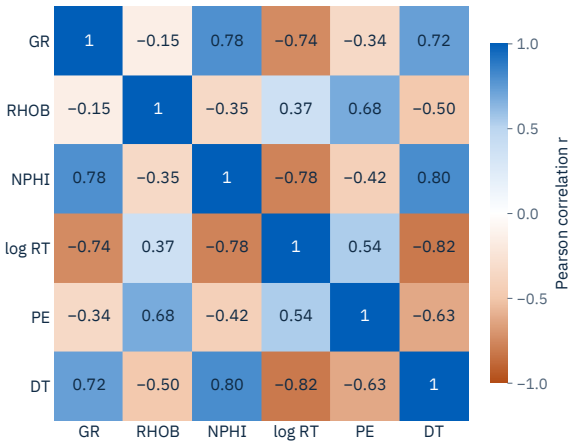
Ask: would a tree need the log transform? No: trees split on thresholds and are unaffected by monotonic transforms. That is a useful first contrast between model families. Remind learners to look at distributions per well too, not only pooled. [3 min]

Correlation shows redundancy, not causation

DATA, EDA AND PREPROCESSING

Correlation shows redundancy, not causation

- GR, NPHI and DT rise together (r = 0.72–0.80)
- log RT falls as NPHI and DT rise
- Correlated inputs make linear coefficients unstable
- r near 0 does not rule out a curved relationship



Pearson r over all 3,200 synthetic samples (seed 7). Much of it comes from mixing facies.

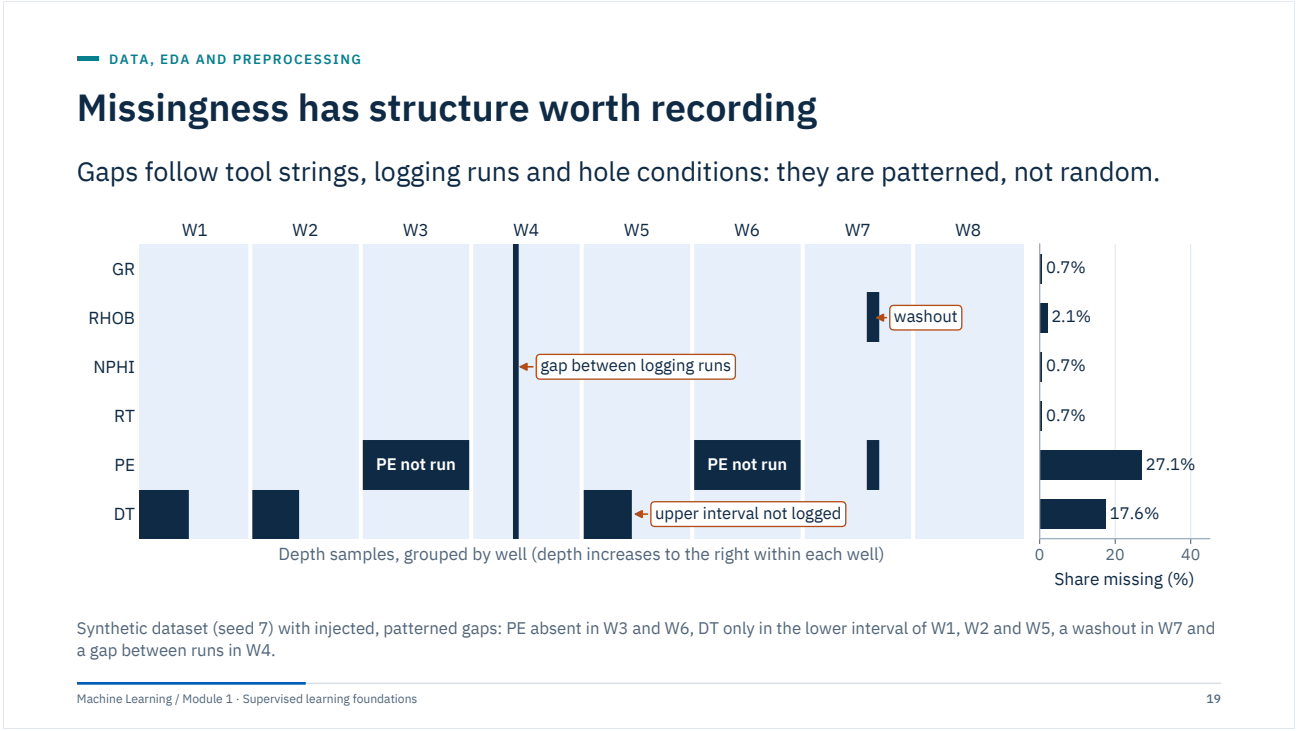
Machine Learning / Module 1 · Supervised learning foundations 18

The Pearson correlation r measures linear association between two features, from −1 to +1. In this synthetic set, GR, NPHI and sonic slowness DT increase together (r between 0.72 and 0.80), and log resistivity falls as they rise (r about −0.74 to −0.82).

Two cautions. First, much of this correlation comes from mixing facies: across rock types, shaly rock has high GR and high NPHI together. Within a single facies the correlation can be weak or even change sign. Second, correlation says nothing about cause, and r near zero does not rule out a strong nonlinear link.

Why it matters today: strongly correlated inputs make individual linear coefficients unstable and hard to interpret, which motivates ridge regression in part 4. They are not a problem for prediction as such. [3 min]

Missingness has structure worth recording



Each row of the matrix is a curve and each column a depth sample, grouped by well. Dark cells are missing. The bar chart gives the share missing per curve: PE 27.1%, DT 17.6%, RHOB 2.1%, the others 0.7%.

Every pattern has a physical story. The PE curve was not run in two wells, perhaps because an older tool string was used. Sonic was logged only over the lower interval of three wells. In W7 a washout (an enlarged borehole) cost the pad-contact measurements, density and PE. In W4 a gap between two logging runs removed all curves.

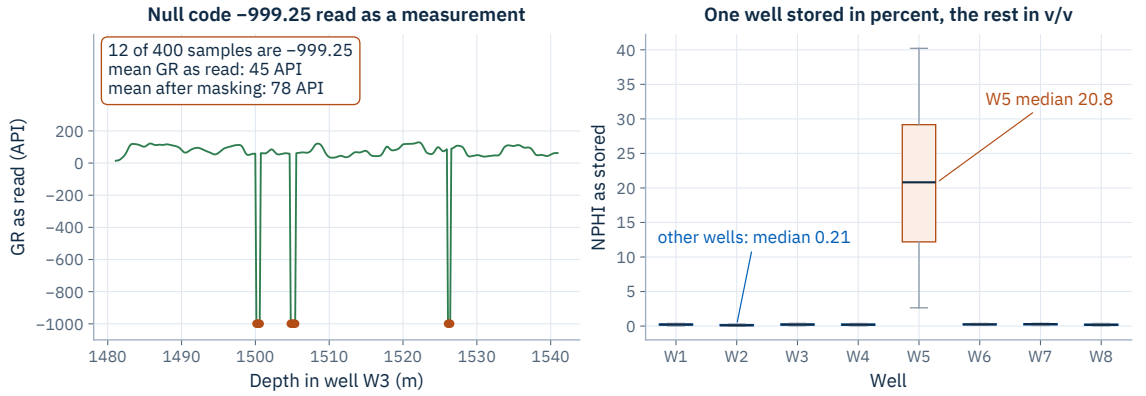
Consequences: dropping rows with any gap would delete two whole wells; imputing blindly would invent PE values for wells that never had the tool. Record the pattern, decide per curve, and consider a missing indicator. Ask: could missingness itself predict the target? Yes, if gaps follow rock type or hole condition, which is why the indicator helps and why it must be checked for leakage. [3 min]

Null codes and unit mixes masquerade as data

DATA, EDA AND PREPROCESSING

Null codes and unit mixes masquerade as data

Both pass a quick glance at a table; both distort means, scalers and distances.



Synthetic data (seed 7) with injected problems: −999.25 null codes in W3 GR and W5 NPHI stored in percent.

Machine Learning / Module 1 · Supervised learning foundations 20

Left: LAS files, the common exchange format for well logs, mark missing values with a null code, very often −999.25. If a reader does not convert it, the code becomes a measurement. Here 12 of 400 samples in W3 are −999.25; the mean gamma ray drops from 78 API to 45 API. A standard scaler fitted on this column would be ruined, and a tree might even learn a split at −500.

Right: one well stores neutron porosity in percent (median 20.8) while the others use volume fraction (median 0.21). Pooled, the column looks like a skewed distribution with a few high values; per well, the problem is obvious.

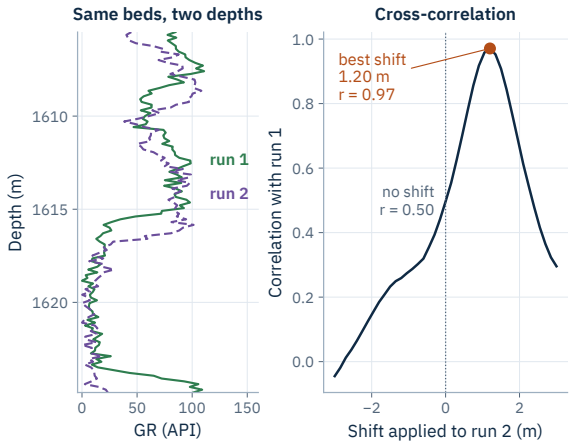
Checks that catch both: per-well minimum, median and maximum; counts of exact repeated values; and a comparison against physical ranges. Ask for another unit trap from experience: pressure in psi versus bar, depth in feet versus metres, rates per day versus per hour. [3 min]

Depth shifts misalign curves from different runs

DATA, EDA AND PREPROCESSING

Depth shifts misalign curves from different runs

- Runs can record the same bed at slightly different depths
- **Cross-correlate** a curve logged on both runs, usually GR
- Apply the best shift, then check several beds by eye
- Align before combining curves into features



Synthetic well W4 (seed 7). Run 2 repeats the GR log 1.2 m deeper with independent noise (seed 4); correlation at each trial shift.

Machine Learning / Module 1 · Supervised learning foundations 21

Logs from different runs, tools or vendors can be offset in depth, for example by cable stretch or different depth references. If density comes from one run and neutron from another, a combined feature such as a density–neutron difference compares different rock.

The standard check is to correlate a curve that was recorded on both runs, usually gamma ray. The right panel computes the correlation between run 1 and run 2 for trial shifts from −3 to +3 m. With no shift, r is 0.50; the maximum, r = 0.97, is at 1.20 m, the shift we injected.

In practice, check the match visually at several beds, because a single global shift may not fit a whole well. For machine learning, the lesson is that depth alignment is part of data preparation and belongs before feature engineering. Ask: what happens to a model trained on misaligned curves? It learns noisier, blurred relationships at every bed boundary. [3 min]

What is wrong with this log summary?

PRACTICE

5 min · Pairs

What is wrong with this log summary?

Curve	Rows present of 3,200	Min / median / max
GR (API)	3,178	−999.25 / 65.2 / 142.3
RHOB (g/cm³)	3,133	2.24 / 2.49 / 2.75
NPHI (v/v)	3,178	0.01 / 0.23 / 40.2
RT (Ω·m)	3,178	0.49 / 5.89 / 331.8
PE (b/e)	2,333	1.71 / 3.48 / 6.14
DT (µs/ft)	2,638	45.8 / 90.6 / 128.9

1 Find at least four problems and say what you would check next.

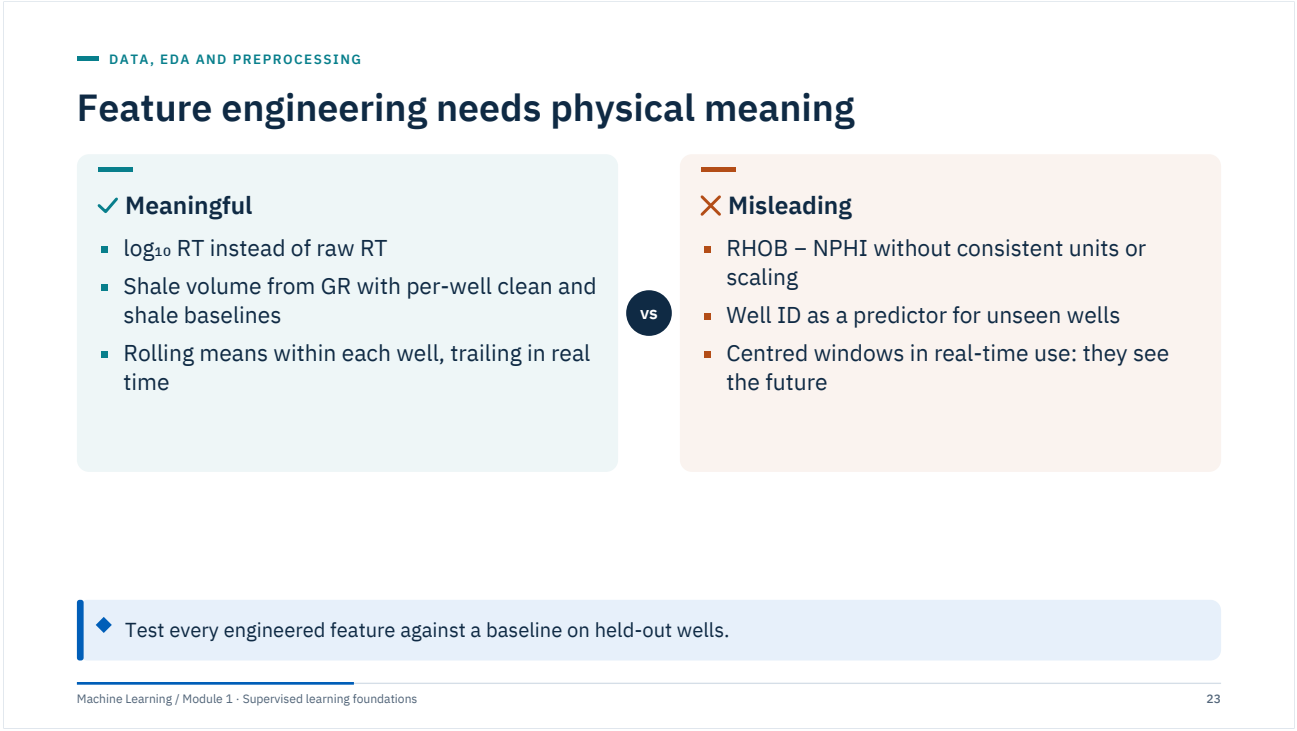
Machine Learning / Module 1 · Supervised learning foundations 22

The table is computed from the synthetic 8-well dataset with the problems injected on the previous slides. Give 4 minutes, then collect answers [5 min].

Worked answers. (1) GR minimum −999.25: a null code read as data; count and mask the exact value, then recompute statistics. (2) NPHI maximum 40.2 with median 0.23: some rows are in percent; check the per-well medians and convert. (3) PE present in only 2,333 rows (27% missing): check which wells lack it; here it is whole wells, so imputation would invent a tool that was never run. (4) DT present in 2,638 rows (18% missing): check depth coverage per well. (5) Every curve is missing at least 22 rows: a gap where all curves vanish together points to a run gap or splice. (6) RT spans 0.49 to 332 with median 5.9: skewed; plan a log transform. RHOB looks physically plausible, which is itself useful information.

Common error: fixing values without recording the fix. Every correction should be a scripted, repeatable step in the pipeline, applied identically to new wells.

Feature engineering needs physical meaning



Good features encode physics or the decision. A log transform turns multiplicative resistivity behaviour into additive steps. A gamma-ray index scaled between per-well clean and shale baselines removes some well-to-well calibration differences. Rolling statistics must be computed within each well, never across a boundary between wells, and only with the time direction the application allows.

Correct three common shortcuts. Subtracting neutron porosity from bulk density without a defined normalization has no direct physical meaning, because the units differ. A well identifier lets a model memorize wells; for unseen wells it carries no transferable information and should be a grouping key, not a feature. Centred windows are fine for offline interpretation of a finished log, but in real-time drilling they would use samples below the bit that do not exist yet.

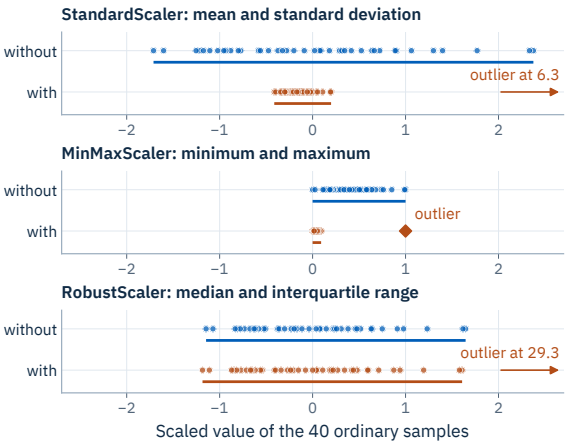
Ask: is depth a good feature? Sometimes, as a proxy for compaction, but it can also let the model memorize the stratigraphy of the training wells. Check it on held-out wells. [3 min]

Scalers react differently to one outlier

DATA, EDA AND PREPROCESSING

Scalers react differently to one outlier

- StandardScaler: mean and SD, both pulled by an outlier
- MinMaxScaler: the range is set by the extremes
- RobustScaler: median and IQR barely move
- Fit any scaler on training rows only

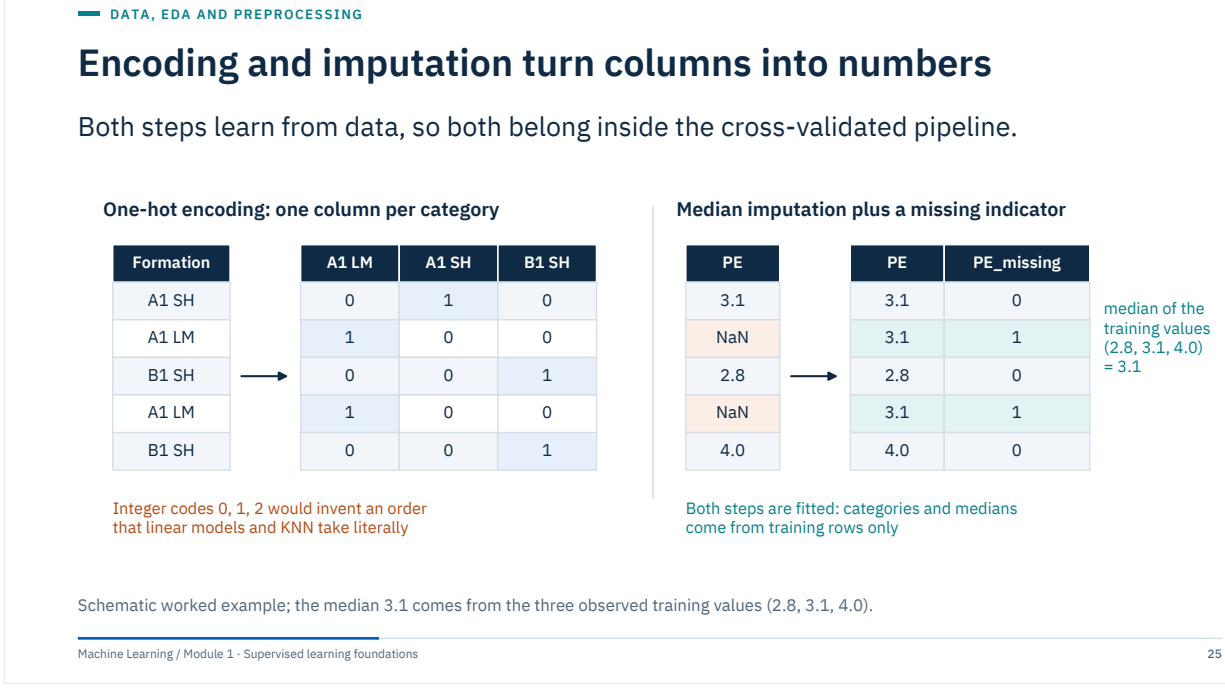


Synthetic feature: 40 samples from N(50, 10), seed 5, plus one outlier at 400. Bars show the range of the 40 ordinary samples.

Each panel fits a scaler twice, without and with a single outlier, and shows where the 40 ordinary samples land. StandardScaler subtracts the mean and divides by the standard deviation; one outlier inflates both, so the ordinary samples collapse into a narrow band and the outlier sits at 6.3. MinMaxScaler maps the minimum and maximum to 0 and 1; the outlier takes the top of the range and squeezes the ordinary samples into about the first tenth. RobustScaler uses the median and the interquartile range (IQR); both barely change, so the ordinary samples keep their spread and the outlier simply lands far away, at 29.3.

Scaling matters for distance- and gradient-based models (KNN, SVM, regularized linear models, neural networks) and not for trees. Whatever the choice, the scaler learns statistics, so it must be fitted on training rows only. Ask: would you rather fix the outlier or choose a robust scaler? Fix it if it is an error, such as a null code; use a robust method if it is real but rare. [3 min]

Encoding and imputation turn columns into numbers

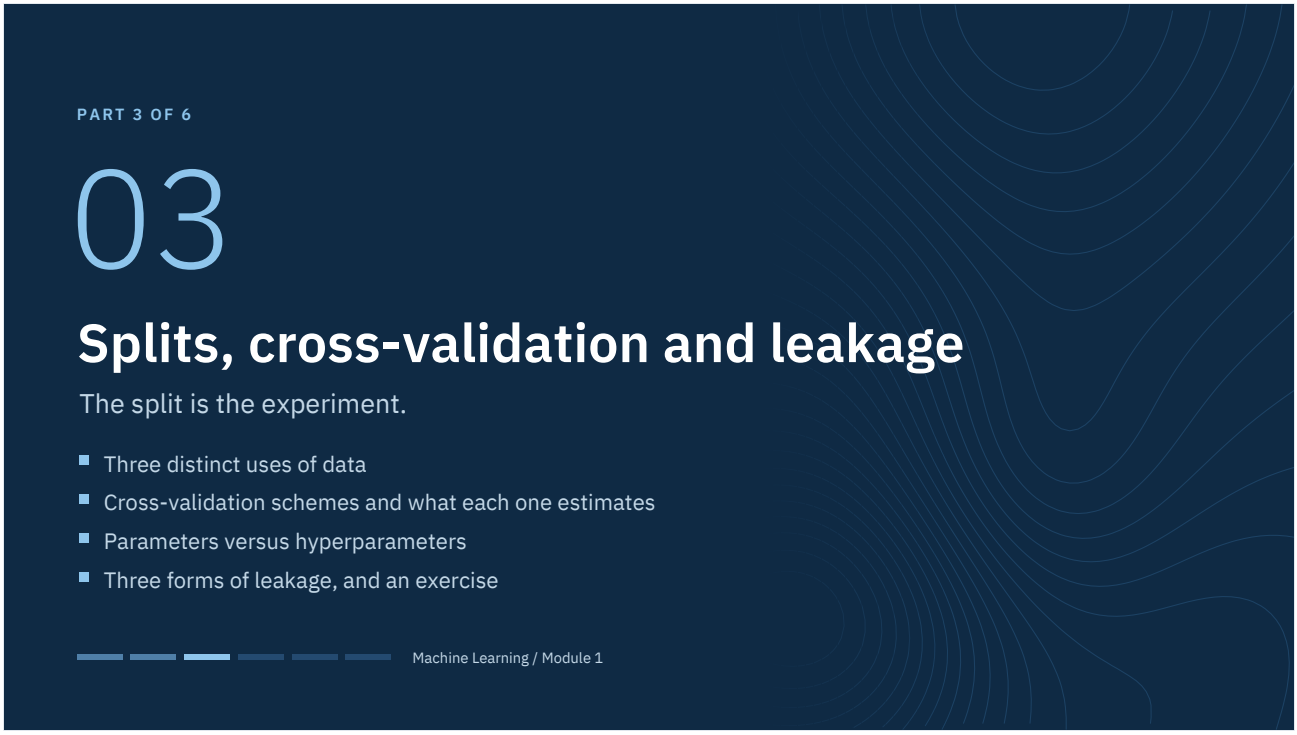


Left: one-hot encoding turns a categorical column such as formation into one 0/1 column per category. Integer codes (0, 1, 2) would tell a linear model or KNN that category 2 is "twice" category 1; trees are less affected but still split on an arbitrary order. In scikit-learn, OneHotEncoder(handle_unknown="ignore") avoids failures when a new well contains a formation not seen in training.

Right: median imputation replaces missing values with the training median, here 3.1, and SimpleImputer(add_indicator=True) adds a PE_missing column so the model can learn whether missingness itself carries information.

Both steps are fitted: the list of categories and the median are learned. Fitting them on all rows before splitting leaks the validation distribution, which leads straight into part 3. Note the facies lab notebook gives Formation integer codes (OrdinalEncoder, fitted inside its pipeline); ask learners to judge whether that is acceptable for the random forest it uses (mostly yes) and for KNN (no). Mixed column types need a ColumnTransformer, covered in Module 2. [3 min]

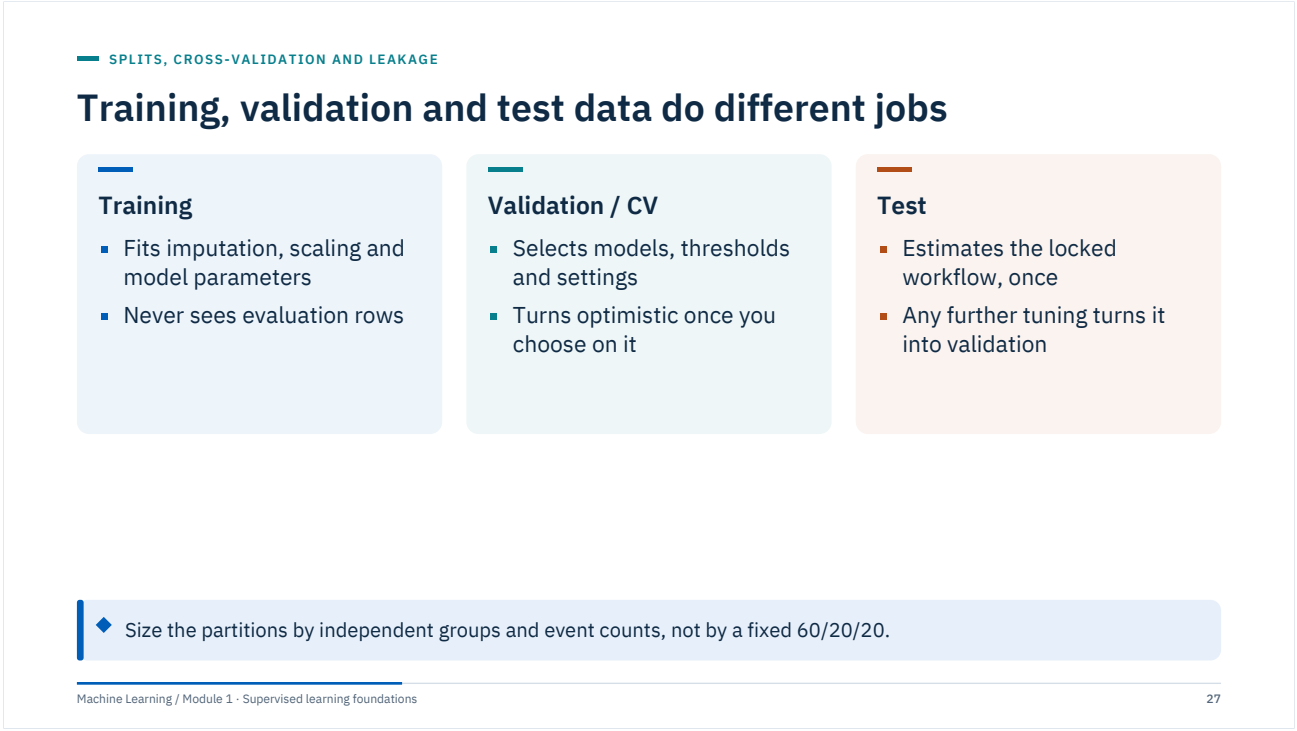
Splits, cross-validation and leakage



Part 3 runs about 35 minutes and is the core of the day's evaluation habits. Every later module assumes these ideas: a split that answers the deployment question, cross-validation that respects groups and time, and preprocessing fitted inside each training fold.

Remind learners of the motivating case: the model was the same; only the split changed the answer from 0.94 to 0.79.

Training, validation and test data do different jobs



Training rows fit everything that is learned, including preprocessing statistics. Validation rows, or cross-validation folds, compare candidates and choose settings; because we choose on them, their scores become optimistic. The test set is used once, on the locked workflow, to estimate performance for the deployment question.

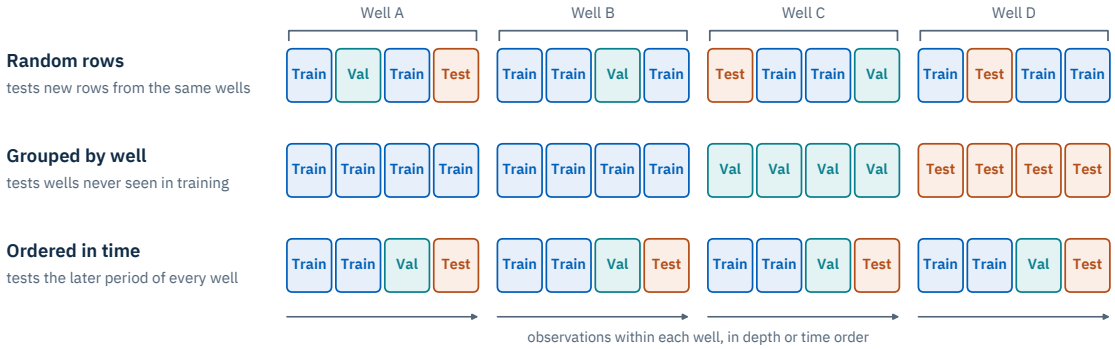
Avoid prescribing a universal 60/20/20 split. With ten wells, the number of independent wells in each partition matters far more than the number of rows. With rare events, the number of events in validation and test decides how noisy the estimates are.

Fold-score standard deviation describes variation across folds; it is not a confidence interval for deployment performance. Ask: what should you do if you looked at the test score and then changed the model? Admit it: the test set has become validation data, and you need new held-out data for an honest estimate. [3 min]

The split follows the deployment question

SPLITS, CROSS-VALIDATION AND LEAKAGE

The split follows the deployment question



Schematic split assignments. Each row answers a different generalization question, not a ranking of methods.

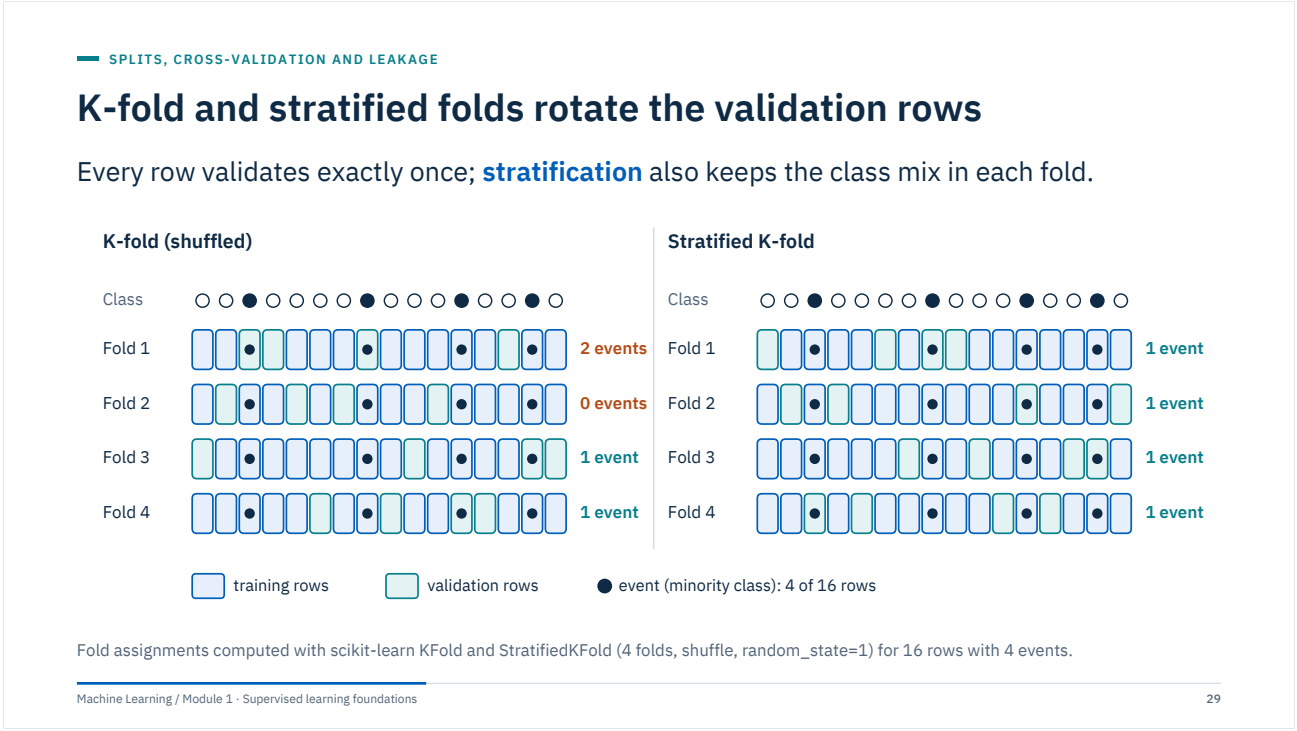
Machine Learning / Module 1 · Supervised learning foundations 28

Read each row as a question. Random rows ask how well the model predicts new rows from the same wells: appropriate only if rows are close to independent, which depth samples are not. Grouped by well asks how well it predicts a well it has never seen: the usual question for new wells. Ordered in time asks how well it predicts the period after training: the question for any forecast.

Forecasting for new wells may need both: grouping and time order at once. A random row split is not wrong in itself; it answers a question nobody usually asks in operations.

Ask: which row matches the facies model in the motivating case? Grouped by well. Which matches a day-ahead load forecast? Ordered in time, with the latest period as the test set. [2 min]

K-fold and stratified folds rotate the validation rows



K-fold cross-validation splits the rows into K folds; each fold is the validation set once while the other K – 1 train the model, and the K scores are averaged. It uses every row for both fitting and validation, which matters when data are scarce.

With a minority class, plain shuffled folds can be unbalanced by chance. Here fold 1 holds 2 of the 4 events and fold 2 holds none, so fold 2 cannot measure event recall at all. StratifiedKFold keeps the class proportions: one event per fold.

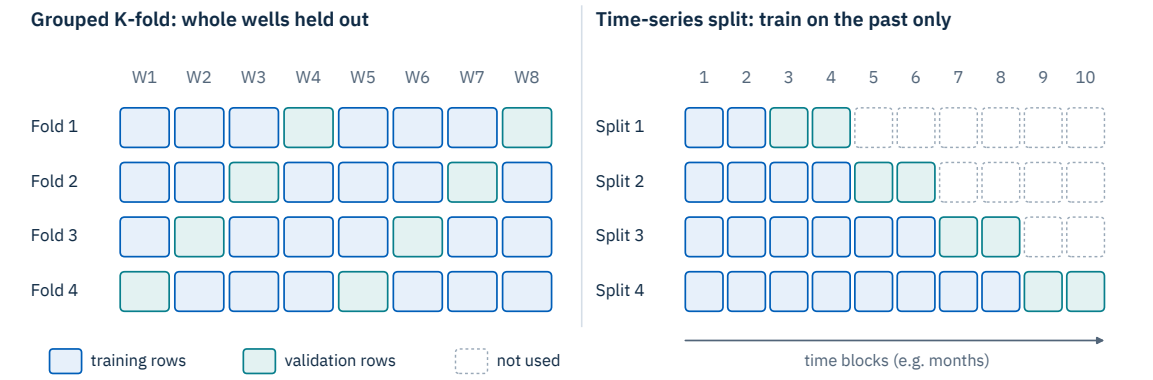
Both schemes are still row-level: they assume rows are exchangeable. That assumption fails for depth samples and time series, which the next slide fixes. Note: scikit-learn's cross_val_score uses stratified folds by default for classifiers when you pass an integer cv. [3 min]

Grouped and time-ordered folds mirror deployment

SPLITS, CROSS-VALIDATION AND LEAKAGE

Grouped and time-ordered folds mirror deployment

Hold out whole wells for new-well questions; train on the past only for forecasts.



Computed with scikit-learn GroupKFold(4) for 8 equal-size wells and TimeSeriesSplit(4) for 10 time blocks.

Left: GroupKFold assigns whole groups to folds, here two wells per fold, so no well is ever split between training and validation. LeaveOneGroupOut is the extreme case with one well per fold, as in the motivating example. StratifiedGroupKFold tries to keep class proportions while respecting groups; check the folds, because with few wells it cannot always succeed.

Right: TimeSeriesSplit trains on an expanding window of past blocks and validates on the next block. Later blocks are never used to predict earlier ones. Its gap parameter leaves blocks unused between training and validation, which you need when the target is several steps ahead or features use trailing windows.

Grouped folds usually give lower and more variable scores than random folds. That is the honest price of the question we actually ask. [3 min]

Each scheme estimates a different future

SPLITS, CROSS-VALIDATION AND LEAKAGE

Each scheme estimates a different future

Scheme	Estimates performance on	Watch out for
K-fold	New rows like the training rows	Correlated rows inflate scores
Stratified K-fold	The same, with a stable class mix	Still row-level
Grouped K-fold	Wells or assets never seen	Few groups give noisy estimates
Time-series split	The period after training	Needs a gap for the forecast horizon

New wells and future periods at once: group by well and keep time order.

◆ Start from the prediction contract: the deployment question picks the scheme.

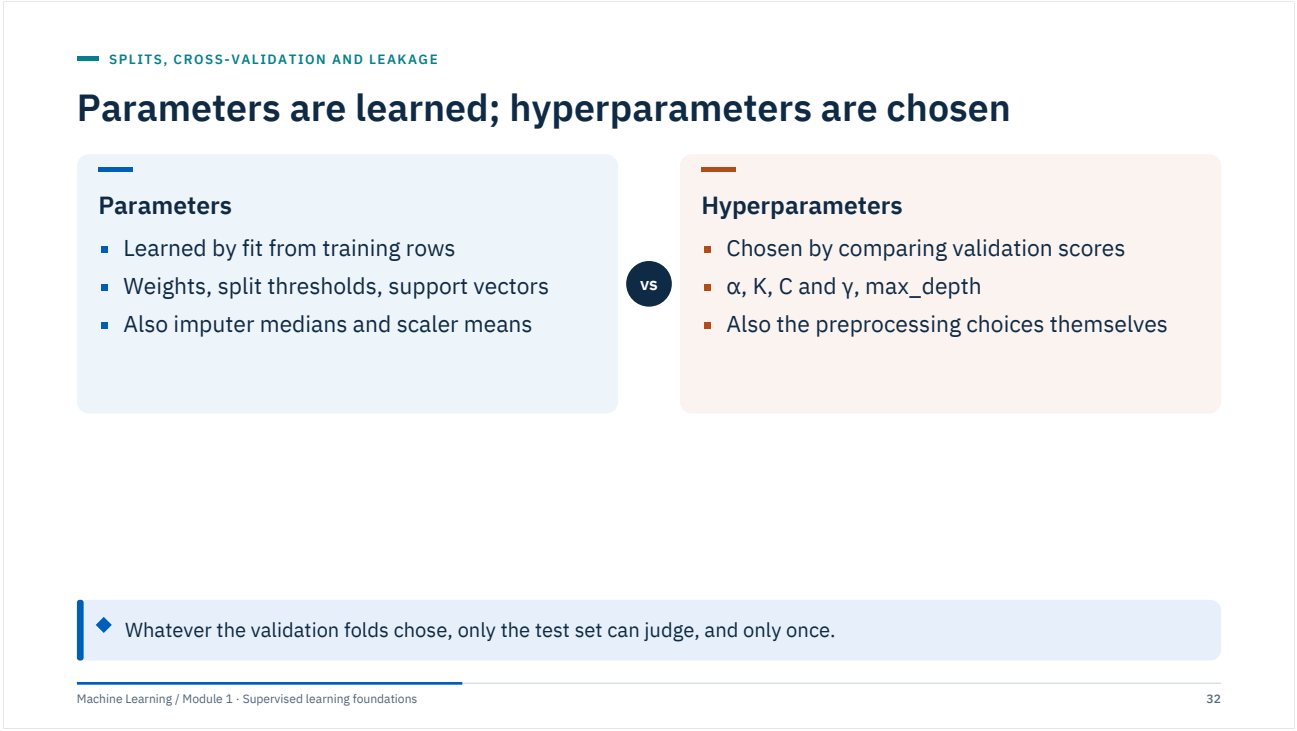
Machine Learning / Module 1 · Supervised learning foundations 31

Use this table as the decision aid for the labs. Start from the deployment question in the contract, then pick the scheme whose "estimates performance on" column matches it.

Two practical notes. With few groups, say six wells, grouped cross-validation has only a handful of folds and each fold score is noisy; report every fold score, not just the mean. For time series, choose the gap from the forecast horizon and the length of any trailing feature window.

Ask: a model predicts the next month's rate for new wells in a developing field. Which scheme? Grouped by well and time-ordered: train on older wells and periods, validate on newer wells in later periods. Module 5 returns to model selection with nested cross-validation. [2 min]

Parameters are learned; hyperparameters are chosen

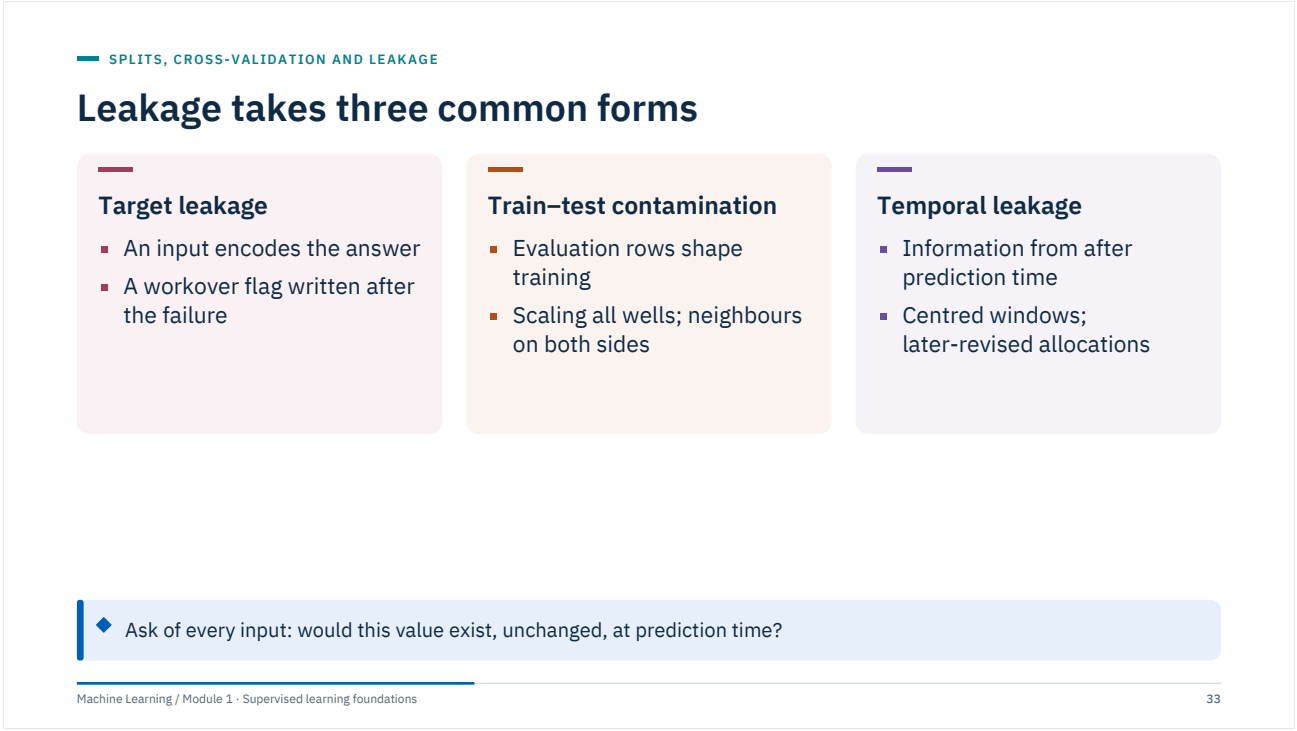


Parameters are the numbers a fit call computes from training rows: regression weights and intercept, tree split thresholds, the support vectors of an SVM, and also the medians of an imputer and the means of a scaler.

Hyperparameters are settings we choose before fitting, by comparing validation scores: the ridge or lasso penalty α, the number of neighbours K, the SVM's C and γ, a tree's max_depth. Choices such as which scaler, which features or which model family are hyperparameters too.

Because validation scores were used to choose, the best validation score is optimistic: it was selected as the maximum of several noisy numbers. That is why the locked workflow needs one final test. Ask: is the number of cross-validation folds a hyperparameter of the model? No: it is part of the evaluation design and should follow the data structure. [2 min]

Leakage takes three common forms



Target leakage: a feature that is a consequence or a re-expression of the target. A workover flag recorded after an ESP failure predicts the failure perfectly in the historical table and not at all in operation. So does a porosity curve interpreted from the same core you try to predict.

Train–test contamination: information from evaluation rows reaches training. Fitting a scaler or feature selection on all wells before splitting does it; so does placing adjacent depth samples of one bed on both sides of a random split, as in the motivating case, and duplicated records.

Temporal leakage: information from after the prediction time. Centred rolling windows, features computed over the whole history, and allocated or revised values that were not available on the day all leak the future.

Kaufman and colleagues (2012) give a formal treatment and many practical examples; see the reference slide. Ask for one leak from learners' experience. [3 min]

Leakage can enter before the model

PRACTICE

3 min Pairs

Leakage can enter before the model

You scale all wells, select features using all labels, then split by well.

1. Which steps have already used the future validation wells?
2. Where should each step run during cross-validation?

Machine Learning / Module 1 · Supervised learning foundations 34

Give 2 minutes, then debrief with the next slide [3 min total].

Answer: both steps have used the validation wells. Scaling on all wells used their means and standard deviations, a mild form of contamination. Feature selection using all labels is serious: it chose features partly because they correlate with the target in the validation wells, so validation scores are biased upward even when the features are pure noise. The grouped split came too late to help.

Where they belong: inside the cross-validation loop, fitted on the training wells of each fold and only applied to the validation wells. In scikit-learn, placing both steps in a Pipeline and passing the pipeline to cross_val_score or GridSearchCV achieves that automatically.

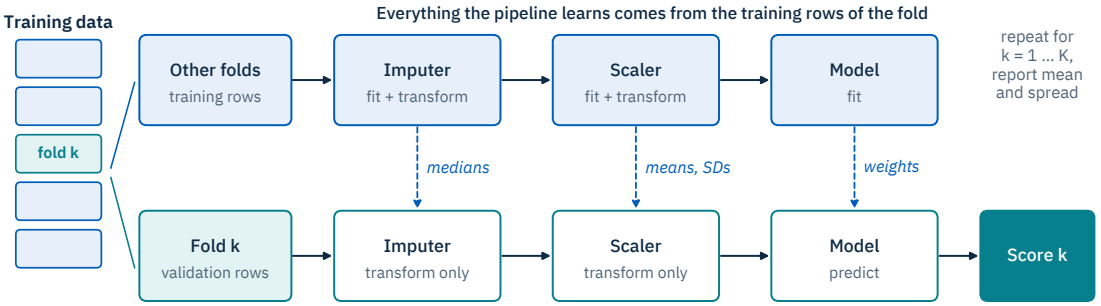
Common error: thinking unsupervised preprocessing is always safe. It is less dangerous than supervised selection, but it still lets validation data shape the model. Module 2 measures how much the wrong order inflates scores in a controlled experiment.

Fit every step inside the training fold

DISCUSSION

Fit every step inside the training fold

Answer: refit the imputer, scaler and model on the training rows of each fold; validation rows are only transformed.



Schematic of one cross-validation fold. Module 2 measures how much the wrong order inflates scores.

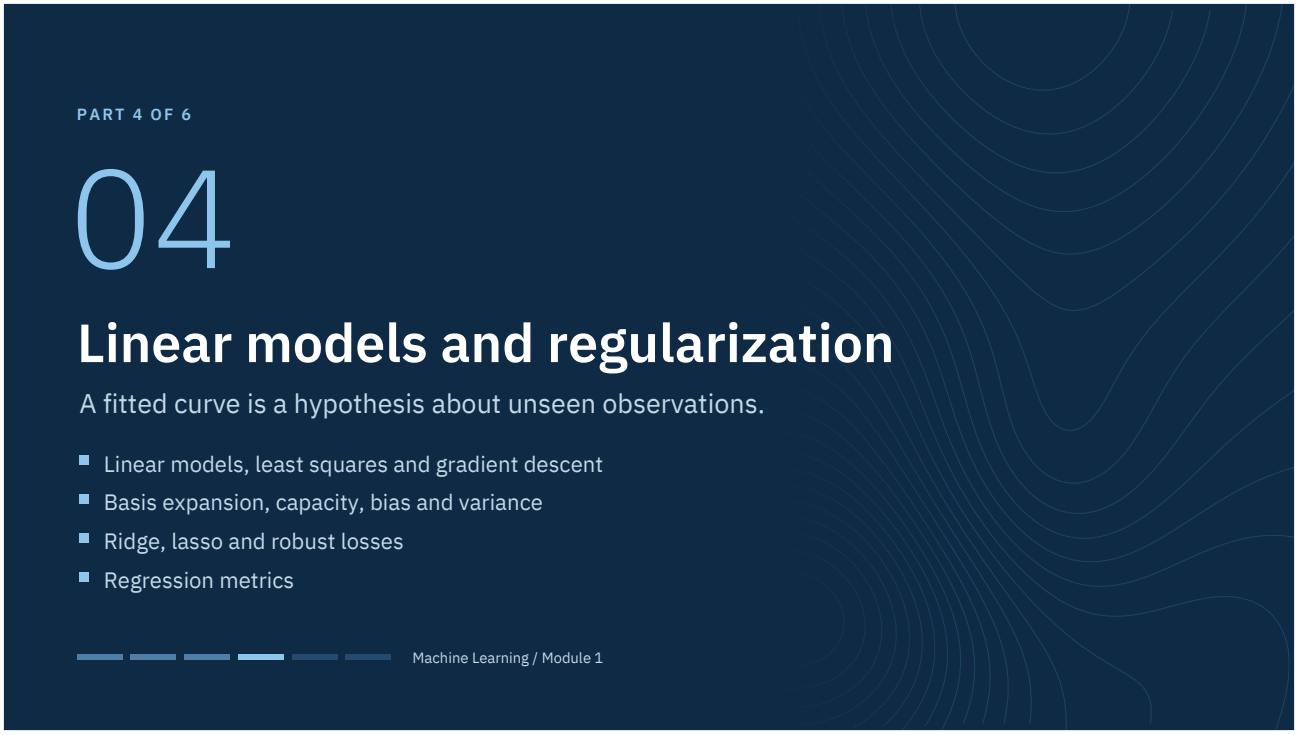
Machine Learning / Module 1 · Supervised learning foundations 35

Walk through one fold. The training rows of fold k flow through the top lane: the imputer learns medians, the scaler learns means and standard deviations, and the model learns its weights. The validation rows flow through the bottom lane and are only transformed with those learned values, then scored. Repeat for every fold and report the mean and the spread of the K scores.

The dashed arrows are the only information that may cross from training to validation: learned parameters, never statistics computed from validation rows.

In code, this is what a scikit-learn Pipeline does inside cross_val_score, cross_validate and GridSearchCV; we write that code in part 6. Ask: where does a feature selection step go? Between the scaler and the model, in the same top lane. [2 min]

Linear models and regularization



Part 4 runs about 50 minutes and ends with Lab 1. We start with the simplest supervised model because every idea of the day shows up in it: parameters, a loss, an optimizer, capacity, regularization and evaluation.

All regression figures use synthetic data with known generating functions, so we can compare what the model learns with the truth.

Ask the room before starting: which of your own targets would you model with a straight line first, and why? Keep one answer on the flip chart; we test it against residuals, capacity and regularization during this part.

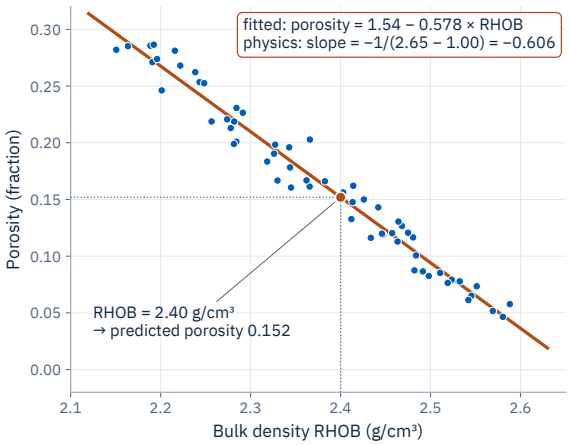
Linear models are linear in their weights

LINEAR MODELS AND REGULARIZATION

Linear models are linear in their weights

$$\hat{y} = b + \sum_{j=1}^{d} w_j x_j$$

- Each **weight** is the change in prediction per unit of its input, others fixed
- Density–porosity is linear by physics: slope −1/(ρ_ma − ρ_f)
- Weights depend on units and on correlated inputs



Synthetic data (seed 3) from the density–porosity relation, matrix 2.65 and fluid 1.00 g/cm³.

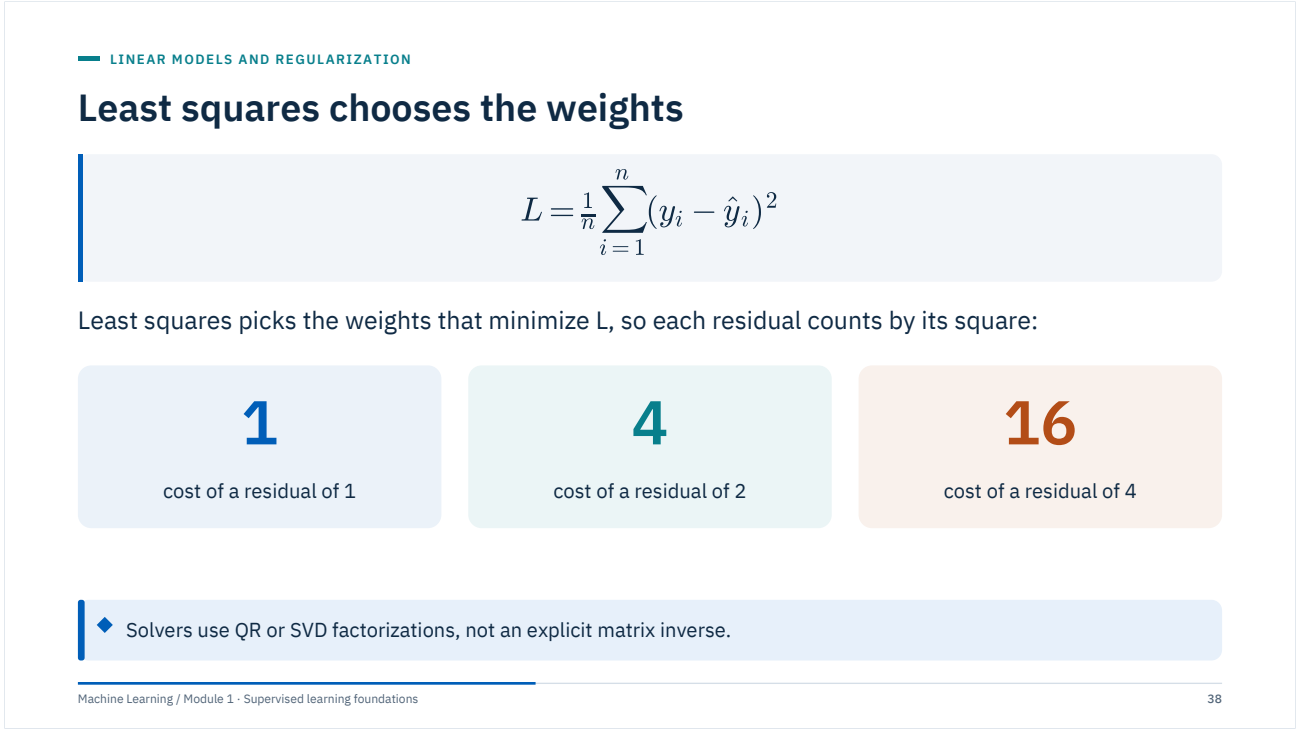
Machine Learning / Module 1 · Supervised learning foundations 37

The model predicts ŷ as an intercept b plus a weighted sum of d inputs x_j with weights w_j. Worked example: with ŷ = 2 + 3x, an input x = 4 gives 2 + 12 = 14.

The figure shows a case where linearity is physical. Porosity from bulk density follows φ = (ρ_ma − ρ_b)/(ρ_ma − ρ_f), with matrix density ρ_ma and fluid density ρ_f. For a sandstone matrix of 2.65 and fluid of 1.00 g/cm³ the slope is −1/1.65 = −0.606 per g/cm³. The least-squares fit on noisy synthetic data gives 1.54 − 0.578 × RHOB; at RHOB 2.40 it predicts a porosity of 0.152.

Interpretation cautions: a weight has units (porosity per g/cm³ here), so it changes with the unit of its input; with correlated inputs, one weight cannot be read as the effect of that input alone, and never as a causal effect without a causal design. Ask: what does the intercept 1.54 mean? The prediction at RHOB = 0, a meaningless extrapolation. [3 min]

Least squares chooses the weights



The loss L is the mean squared residual over the n training rows, where a residual is y minus ŷ. Squaring makes all residuals positive and weights large misses heavily: a residual of 4 costs 16 times as much as a residual of 1.

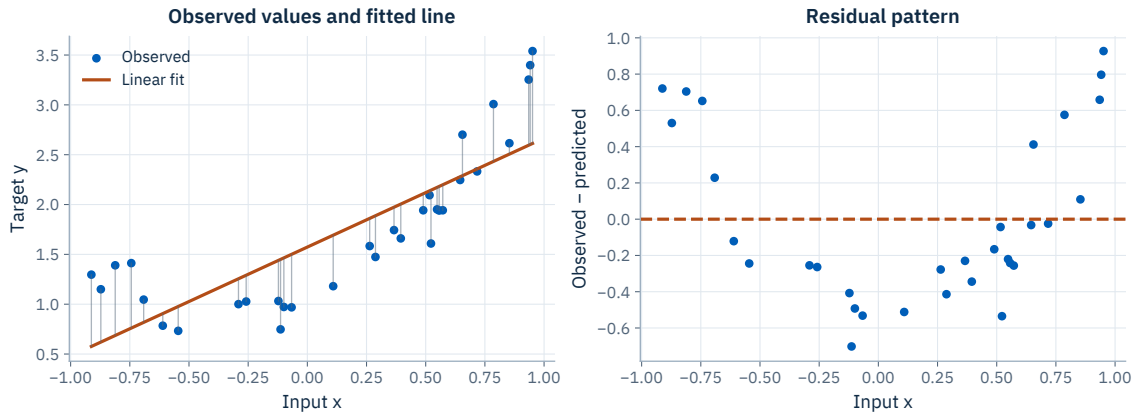
For ordinary linear regression this loss is convex, so there is a single minimum. Optional derivation for the mathematically inclined: setting the gradient to zero gives the normal equations $X^TX\,w = X^Ty$. Writing $w = (X^TX)^{-1}X^Ty$ requires full column rank and is numerically fragile when inputs are nearly collinear; libraries use QR or SVD factorizations or a pseudoinverse instead.

Ask: what happens when two inputs are perfect copies of each other? X^TX is singular and the weights are not unique; any split of the weight between the copies fits equally well. Ridge regression, later in this part, fixes that. [2 min]

A residual shows what the model missed

LINEAR MODELS AND REGULARIZATION

A residual shows what the model missed



Computed synthetic example. Residual = observed value minus prediction. A curved residual pattern indicates missing structure.

Machine Learning / Module 1 · Supervised learning foundations 39

Before showing the right panel, ask learners to predict the sign of the residuals at both ends of the input range and in the middle. The straight line underestimates at both ends and overestimates in the middle, so residuals are positive, then negative, then positive: a curved pattern.

A residual plot is the most useful single diagnostic for regression. Look for curvature (missing structure, try a transform or basis expansion), a spread that grows with the prediction (heteroscedasticity, consider a log target), isolated large residuals (outliers or data errors), and systematic offsets by group.

For energy data, plot residuals by well, by operating regime or by time. A small overall error can hide one well that is badly wrong, and that well may be the one the decision depends on. [2 min]

One gradient step, worked by hand

LINEAR MODELS AND REGULARIZATION

One gradient step, worked by hand

$$w_{new} = w - \alpha \frac{\partial L}{\partial w}$$

One sample x = 2, y = 5; intercept fixed at 0; start at w = 1; learning rate α = 0.1.

1. **Predict** — ŷ = wx = 2, so the squared error is (5 − 2)² = 9
2. **Gradient** — ∂L/∂w = 2x(wx − y) = 2 · 2 · (2 − 5) = −12
3. **Step** — w = 1 − 0.1 × (−12) = 2.2
4. **Check** — ŷ = 4.4, and the squared error falls to (5 − 4.4)² = 0.36

◆ Too large a learning rate overshoots the minimum and can increase the loss.

Machine Learning / Module 1 · Supervised learning foundations 40

Least squares has a closed-form solution, but most models do not, so we introduce the general tool now. Gradient descent moves each weight a small step against the gradient of the loss: the gradient points uphill, so we step downhill. α is the learning rate.

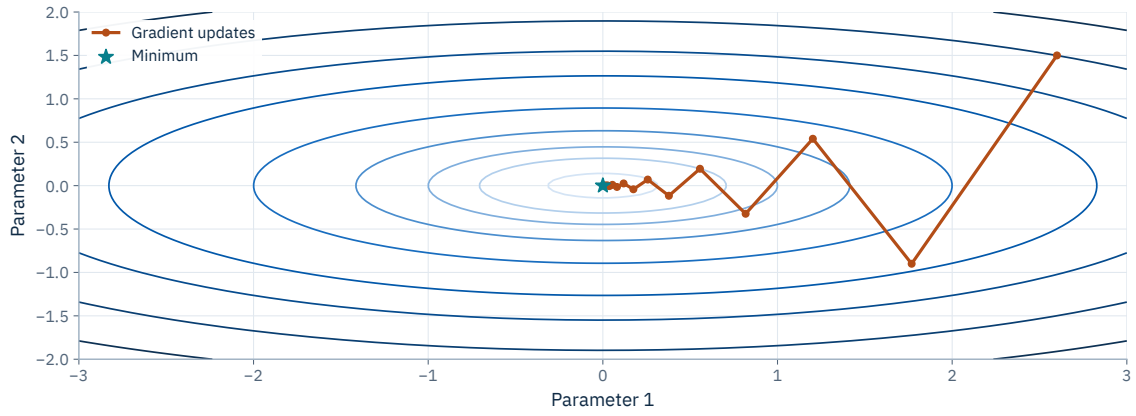
Work the numbers aloud. Prediction wx = 1 × 2 = 2. Squared error (5 − 2)² = 9. Gradient 2x(wx − y) = 2 × 2 × (2 − 5) = −12. New weight 1 − 0.1 × (−12) = 2.2. New prediction 4.4. Ask learners to compute the new squared error: (5 − 4.4)² = 0.36, down from 9.

The example fixes the intercept for simplicity. With many rows, the gradient is the average over rows (batch) or over a random subset (mini-batch). A learning rate that is too large can overshoot and increase the loss; one that is too small wastes iterations. The neural network course builds on exactly this step. [3 min]

Gradient descent zigzags when curvatures differ

LINEAR MODELS AND REGULARIZATION

Gradient descent zigzags when curvatures differ



Computed on the exact quadratic loss $L = w_1^2 + 5w_2^2$, learning rate 0.16. The path follows successive updates from one start.

Machine Learning / Module 1 · Supervised learning foundations 41

The contours show a loss that is five times steeper in the second parameter than in the first. Each step moves perpendicular to the contour, which points mostly across the narrow valley, so the path overshoots and zigzags in w_2 while making slow progress in w_1.

The practical lesson: when inputs have very different scales, the loss surface becomes elongated in exactly this way. Standardizing inputs makes the curvatures more similar and gradient descent converges faster. That is one more reason scaling appears in almost every pipeline that uses gradient-based fitting.

Ask: what would a larger learning rate do here? Beyond a critical value (0.2 for this loss, where the w_2 direction has curvature 10) the zigzag grows instead of shrinking and the method diverges. [2 min]

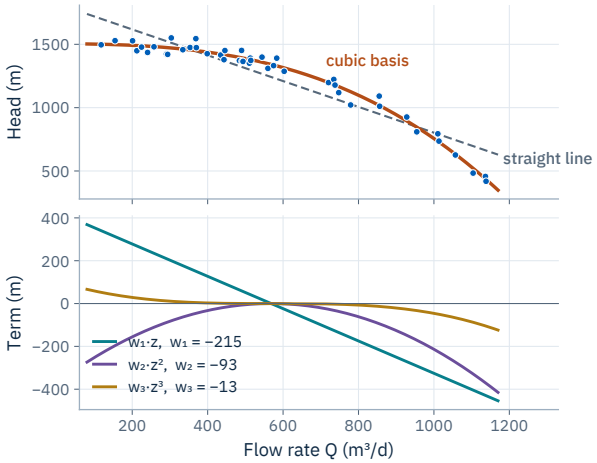
Basis expansion lets a linear model bend

LINEAR MODELS AND REGULARIZATION

Basis expansion lets a linear model bend

$$\hat{y} = b + w_1 z + w_2 z^2 + w_3 z^3$$

- Still linear in the weights, so least squares still applies
- z is standardized flow: raw powers of Q would be badly conditioned
- The degree is a hyperparameter: choose it by validation



Synthetic head–flow data (seed 8), 45 points; cubic fit on the standardized flow z.

Machine Learning / Module 1 · Supervised learning foundations 42

A straight line cannot follow the curved head–flow relationship in the top panel. Adding columns z^2, z^3, where z is the standardized flow, lets the same least-squares machinery fit a curve. The model stays linear in its weights; only the inputs are transformed.

The bottom panel shows why. The fitted curve is the intercept (1,341 m) plus three weighted terms: $w_1 z$ with $w_1 = -215$, $w_2 z^2$ with $w_2 = -93$, and $w_3 z^3$ with $w_3 = -13$. Each term is a simple shape; their sum is the orange curve.

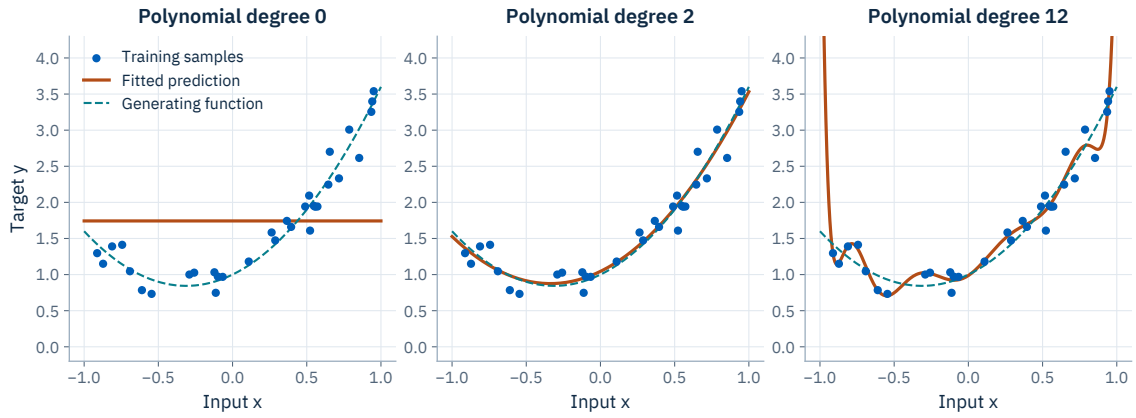
Standardize first: powers of raw flows around 1,000 would span nine orders of magnitude and make the problem ill-conditioned. Other bases include splines and physically motivated transforms such as log RT.

In scikit-learn: make_pipeline(PolynomialFeatures(3), StandardScaler(), LinearRegression()). Ask: what is the danger of degree 15? Wild oscillation between data points, the subject of the next slides. [3 min]

Capacity changes the fitted function

LINEAR MODELS AND REGULARIZATION

Capacity changes the fitted function



Three models fitted to the same synthetic training observations. The dashed curve is the generating function, not an available training label.

Machine Learning / Module 1 · Supervised learning foundations 43

Capacity is how flexible a model family is. Degree 0 predicts a constant and misses the structure: underfitting. Degree 2 matches the generating function closely. Degree 12 passes near every training point and swings wildly between and beyond them: overfitting, fitting the noise.

Ask which model fits the training data best: degree 12 has the smallest training error. Ask which you would trust for a new x: degree 2. The training error cannot choose the capacity; it always prefers more.

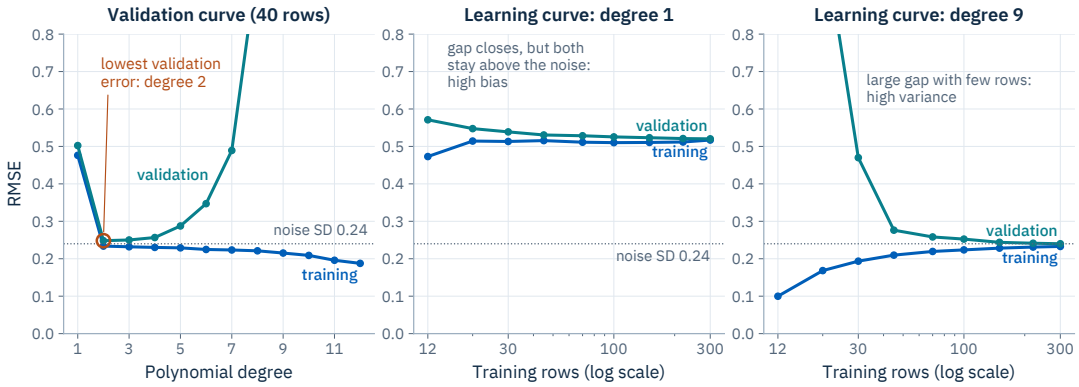
Do not choose a model by how attractive its curve looks, and remember that this figure shows one dataset: the right degree depends on the data. Held-out data drawn like the deployment population decide, which the next slide does systematically. [2 min]

Held-out curves diagnose capacity and data needs

LINEAR MODELS AND REGULARIZATION

Held-out curves diagnose capacity and data needs

Validation curves choose complexity; learning curves show whether more data would help.



Synthetic y = 1 + x + 1.6x² + noise (SD 0.24): validation curve on 40 rows (seed 94, 5-fold CV repeated 12 times); learning curves from a 400-row pool (seed 12). Mean RMSE.

Left, the validation curve: training RMSE keeps falling as the degree grows, while cross-validated RMSE falls to a minimum at degree 2, the true degree here (0.248, with degree 3 practically tied at 0.250), then rises from degree 5 and steeply beyond degree 7 as the model fits noise. At the minimum, both errors sit close to the noise SD of 0.24, the error no model can remove; a training RMSE well below it (0.19 at degree 12) is a sign of fitting noise. Choose the degree at or near the minimum, preferring the simpler model when scores are within noise.

Middle, a learning curve for degree 1: training and validation errors converge quickly and both stay near 0.52, far above the noise level of 0.24. More data will not help; the model is too simple (high bias).

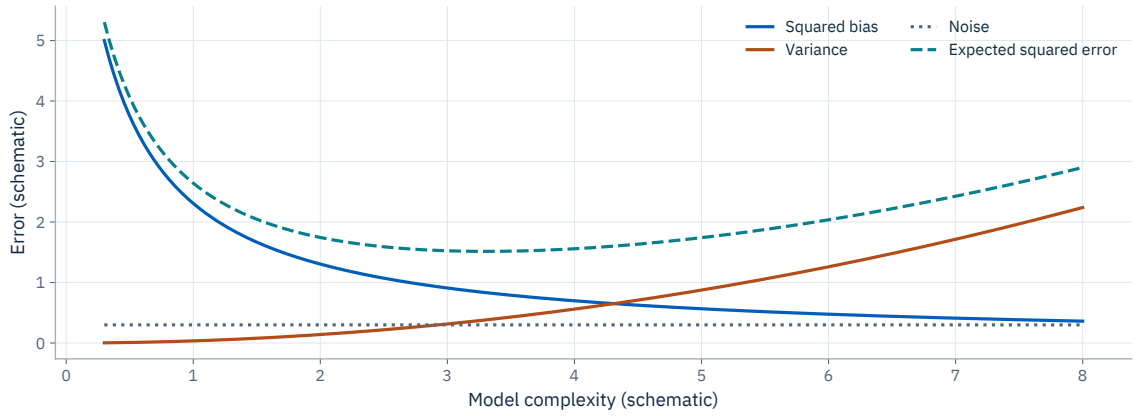
Right, degree 9: with few rows the validation error is huge and far above training error; the gap closes as rows increase and both approach the noise floor. More data would help this flexible model (high variance).

In scikit-learn these are validation_curve and learning_curve; both accept groups and a grouped cv. Ask: which diagnosis fits the motivating case? Neither directly: its problem was the split, a reminder that curves are only as honest as their folds. [3 min]

Bias and variance explain different failures

LINEAR MODELS AND REGULARIZATION

Bias and variance explain different failures



Schematic squared-error decomposition. These curves illustrate a concept and are not measured learning curves.

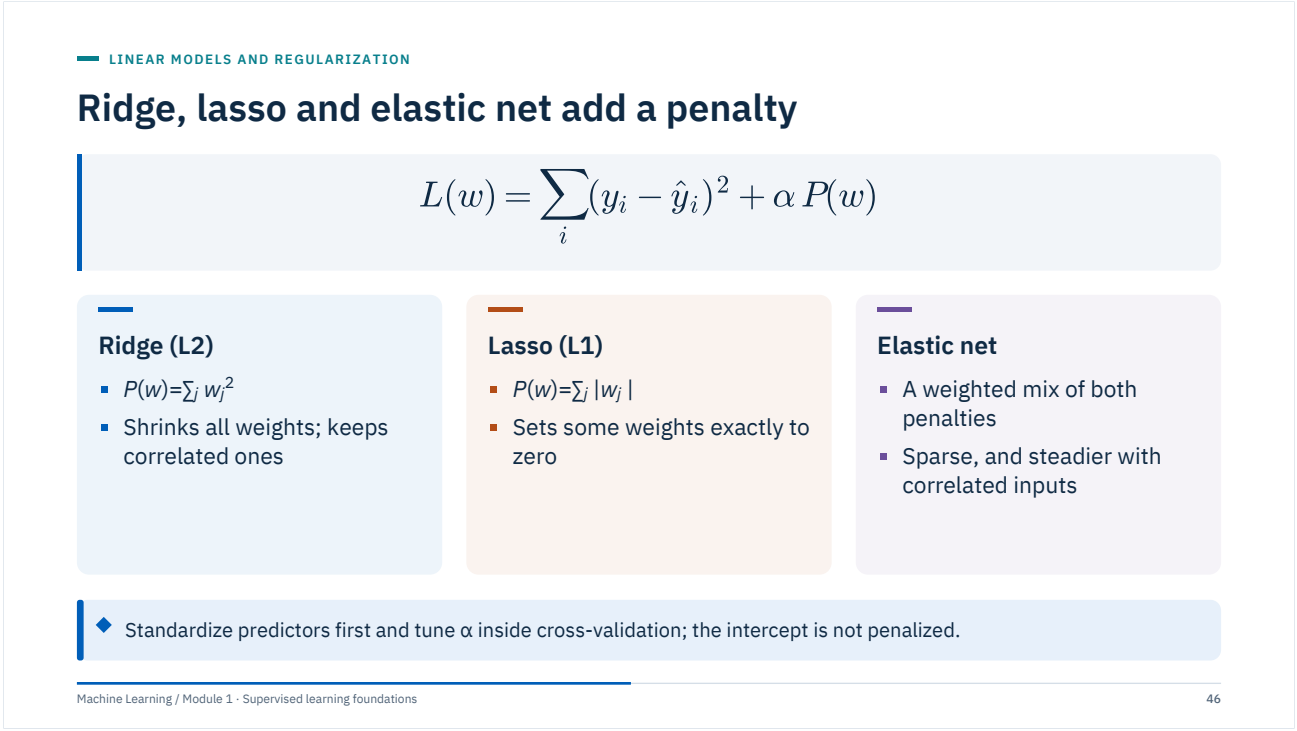
Machine Learning / Module 1 · Supervised learning foundations 45

For squared error, the expected prediction error at a point decomposes into squared bias, variance and irreducible noise. Bias is systematic error from a model too simple for the structure. Variance is sensitivity to the particular training sample: fit the same flexible model on another sample and the predictions change. Noise is what no model can remove.

As capacity grows, bias falls and variance rises; the total is smallest somewhere in between. This is the story behind the validation curve on the previous slide.

Caveat: the classical picture is useful but not a universal law. Heavily overparameterized models, including large neural networks, can behave differently. Diagnose actual behaviour with held-out learning and validation curves rather than assuming the U-shape. Ask: which lever reduces variance without changing the model family? More data, averaging (Module 2) or regularization, next. [2 min]

Ridge, lasso and elastic net add a penalty



Regularization adds a penalty on the size of the weights to the least-squares loss. α controls its strength: α = 0 is ordinary least squares; very large α pushes all weights towards zero.

Ridge penalizes squared weights and shrinks them smoothly; with correlated inputs it spreads weight across them and stabilizes the fit. Lasso penalizes absolute weights and can set some exactly to zero, a built-in feature selection. Elastic net mixes both, keeping sparsity while behaving better when inputs are correlated.

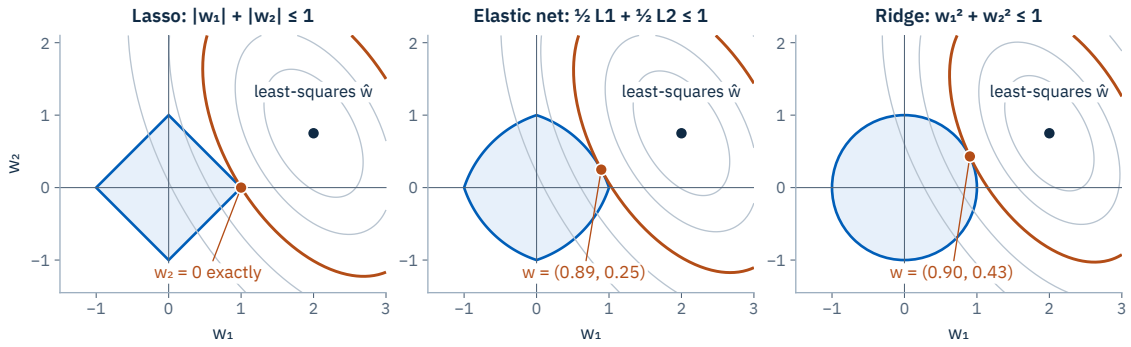
Details that matter in practice: the penalty compares weights across inputs, so inputs must be on comparable scales (standardize first). Libraries scale α differently relative to the loss, so α values are not portable between libraries or even between Ridge and Lasso in scikit-learn. The intercept is not penalized. Ridge does not perform exact selection, and lasso may pick one of several correlated features arbitrarily. [2 min]

L1 and L2 penalties have different geometry

LINEAR MODELS AND REGULARIZATION

L1 and L2 penalties have different geometry

The fit sits where the smallest loss contour touches the allowed region: corners on the axes give exact zeros.



Exact quadratic loss centred at ŵ = (2.0, 0.75); constrained minima computed numerically. Elastic net uses half L1, half L2.

Machine Learning / Module 1 · Supervised learning foundations 47

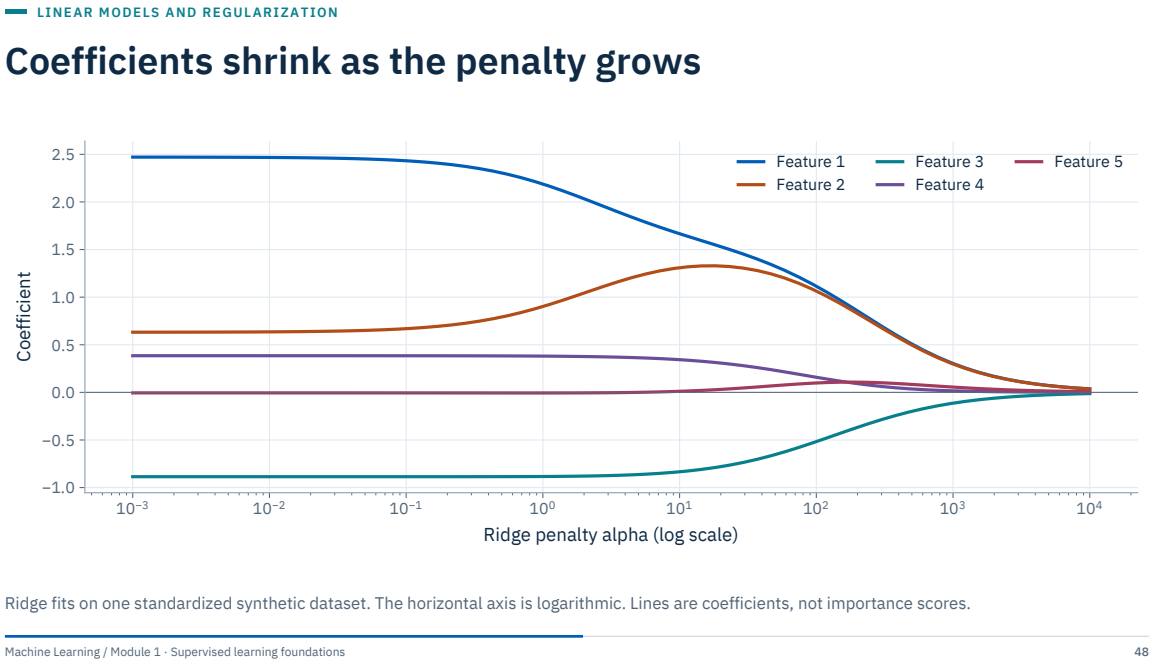
Each penalty can be written as a constraint: minimize the squared loss while keeping $|w_1| + |w_2| \le t$ (lasso) or $w_1^2 + w_2^2 \le t^2$ (ridge). For every α there is a matching budget t.

The grey ellipses are contours of the squared loss around the least-squares solution $\hat{w} = (2.0, 0.75)$. The constrained solution is where the smallest contour touches the allowed region. The diamond's corners stick out along the axes, so the contour often touches a corner: here exactly (1, 0), with $w_2 = 0$. The circle is smooth, so its touching point, (0.90, 0.43), generally has both weights nonzero.

The middle panel mixes the two penalties equally. Its region keeps corners on the axes but its edges bulge outwards; here the fit lands at (0.89, 0.25), shrinking w_2 harder than ridge without zeroing it. With other data, elastic net also returns exact zeros.

In more dimensions the diamond has many corners and edges on the axes, which is why lasso produces sparse solutions. Ask: if w_2 belongs to a useful but correlated feature, which penalty would you prefer? Ridge or elastic net, to keep it. [3 min]

Coefficients shrink as the penalty grows



Trace the ridge path from left to right. With little penalty, the coefficients are close to least squares. As α grows they shrink towards zero together, smoothly, and none becomes exactly zero at any finite α.

Look at features 1 and 2, which are highly correlated by construction: as the penalty grows, ridge moves weight from feature 1 to feature 2 so that they share the effect, then shrinks both. That sharing is the stabilizing behaviour we want with collinear logs.

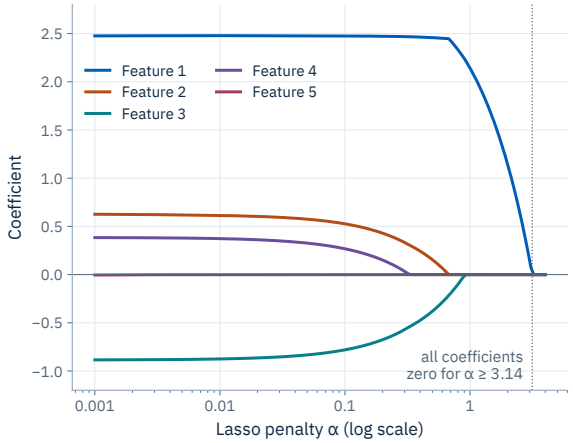
Ask why the preferred α cannot be read from this plot. The path shows what the model does, not how well it predicts; the choice comes from cross-validated error at each α, for example with RidgeCV. Also note that shrinking coefficients can reduce variance but eventually underfits. [2 min]

Lasso sets coefficients exactly to zero

LINEAR MODELS AND REGULARIZATION

Lasso sets coefficients exactly to zero

- Same standardized data as the ridge path
- As α decreases, weights leave zero one at a time
- Of the correlated features 1 and 2, lasso drops feature 2 first
- Above α = 3.14 every weight is zero



Same synthetic data as the shared ridge path (seed 42); computed with scikit-learn lasso_path.

Machine Learning / Module 1 · Supervised learning foundations 49

Read this path from right to left. For α at or above 3.14 every weight is exactly zero; in scikit-learn's scaling this threshold is the largest $|x_j^T y|$ over the standardized features, divided by the number of rows. As α decreases, feature 1 enters first, then feature 3, then feature 2 and feature 4. Feature 5, which has no true effect, stays at or very near zero.

Compare with ridge: lasso reaches exact zeros, so it performs selection. With the correlated pair, lasso keeps feature 1 strongly and holds feature 2 at zero until α is small; which of the two survives can change with a small change in the data, so lasso selections among correlated features are unstable. Elastic net reduces that instability.

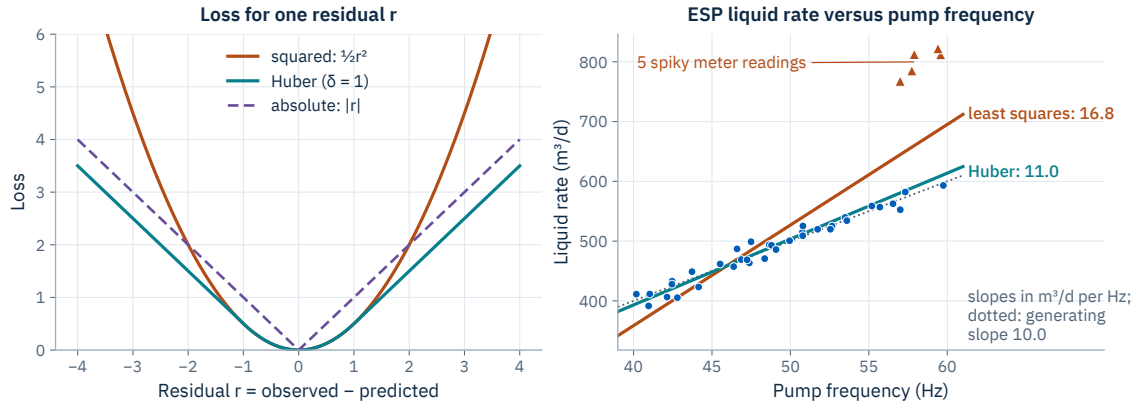
Use LassoCV or ElasticNetCV with grouped folds to choose α. Ask: would you report the zeroed features as irrelevant to the physics? No: they may be redundant given the others, not irrelevant. [3 min]

Squared loss lets a few spikes pull the fit

LINEAR MODELS AND REGULARIZATION

Squared loss lets a few spikes pull the fit

Huber loss is quadratic for small residuals and linear for large ones, so spikes pull less.



Synthetic ESP data (seed 6): rate = 10 × frequency + noise, with 5 upward spikes. Slopes: least squares 16.8, HuberRegressor 11.0 m³/d per Hz.

Machine Learning / Module 1 · Supervised learning foundations 50

Left: squared loss grows with the square of the residual, so one residual of 4 costs as much as sixteen residuals of 1. Huber loss is quadratic up to a threshold δ and linear beyond it; absolute loss is linear everywhere.

Right: a synthetic ESP liquid rate that truly rises by 10 m³/d per Hz, with five spiky meter readings at high frequency. Least squares tilts the line towards the spikes: slope 16.8. HuberRegressor, with its default threshold on scaled residuals (epsilon = 1.35), largely ignores them: slope 11.0, close to the generating 10.

Robust losses are a safety net, not a substitute for auditing. If the spikes are meter errors, remove or flag them and document the rule. If they are real events, they may be exactly what the decision cares about, and down-weighting them would hide them. Ask learners which it is for their own data. [2 min]

Regression metrics answer different questions

LINEAR MODELS AND REGULARIZATION

Regression metrics answer different questions

$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

MAE

- Average miss, in target units
- Treats all error sizes linearly

RMSE

- In target units; large misses count more
- Sensitive to outliers

R²

- Improvement over the evaluation mean
- Can be negative; depends on the spread

◆ Report errors by well or operating regime as well as overall.

Machine Learning / Module 1 · Supervised learning foundations 51

Mean absolute error (MAE) averages the absolute residuals and is in target units: easy to explain. Root mean squared error (RMSE) squares, averages and takes the root: also in target units, but large misses count more. The ratio RMSE/MAE grows when a few large errors dominate. Worked example: residuals 1, −1 and 4 give MAE 2 and RMSE √6 ≈ 2.45.

R^2 compares the model's squared error with that of predicting the mean of the evaluation targets: 1 is perfect, 0 is no better than the mean, and negative values are worse than the mean. Because the denominator depends on the evaluation set, R^2 is not comparable across datasets with different spreads, and a high R^2 does not guarantee an acceptable error in physical units.

Choose the metric from the decision: if a 20 m³/d miss costs twice a 10 m³/d miss, MAE fits; if large misses are disproportionately costly, RMSE. [2 min]

Compute MAE, RMSE and R² by hand

PRACTICE

4 min · Individual

Compute MAE, RMSE and R² by hand

Observed y = 10, 12, 9, 15; predicted ŷ = 9, 13, 9, 11.

1. Compute the residuals, the MAE and the RMSE.
2. Compute R^2, given that the mean of y is 11.5.
3. Which metric reacts most to the one large miss?

Machine Learning / Module 1 · Supervised learning foundations 52

Give 3 minutes, then work it on the board [4 min].

Residuals $y - \hat{y}$: 1, −1, 0, 4. MAE = (1 + 1 + 0 + 4)/4 = 1.5. Squared residuals 1, 1, 0, 16, sum 18, mean 4.5, so RMSE = $\sqrt{4.5} \approx 2.12$.

R^2: the total sum of squares around the mean 11.5 is $(-1.5)^2 + 0.5^2 + (-2.5)^2 + 3.5^2 = 2.25 + 0.25 + 6.25 + 12.25 = 21$. $R^2 = 1 - 18/21 \approx 0.14$. The model barely beats predicting the mean, although three of four predictions are within 1 unit.

The one miss of 4 contributes 16 of the 18 squared units, so RMSE (2.12) is well above MAE (1.5) and R^2 is low. Common errors: forgetting the square root, dividing by n − 1, or computing R^2 around the mean of the predictions. Transition: in Lab 1 you will compute the same metrics with scikit-learn and add a baseline row.

Lab 1: regression and regularization

LABORATORY

40 min

Lab 1: regression and regularization

- ☐ §1.1–1.2: split three ways; fit linear and degree-2 models; record R^2 and MAE
- ☐ Put a DummyRegressor mean baseline in the first row of your score table
- ☐ §1.3: plot training and validation R^2 against degree, raw and standardized
- ☐ §2: compare ridge and lasso over α on validation rows; count nonzero terms
- ☐ §2.3: lock one model, then score the test rows once

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table, baseline first
- Degree plots: raw powers and standardized columns
- One sentence: which α, and how it was chosen

Machine Learning / Module 1 · Supervised learning foundations 53

Timing: 40 minutes of work, then the 10-minute debrief before lunch (11:50–12:40). Pairs are fine.

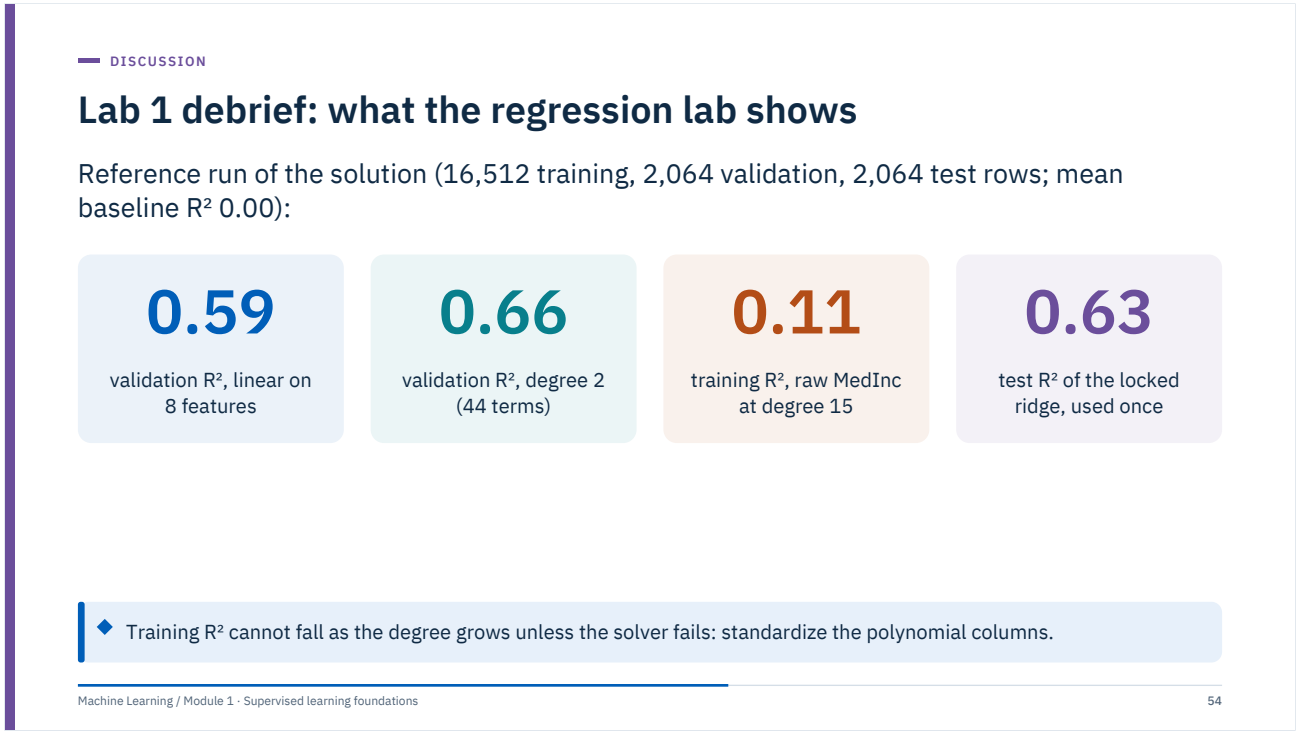
Setup: the notebook loads the California housing data with fetch_california_housing, which downloads about 0.4 MB on first use and caches it in the scikit-learn data folder. Run the solution notebook's loading cell on every machine before the day, or copy a pre-filled cache; offline, the cell stops with a message that says what to do. It is not an energy dataset, but the mechanics transfer directly.

Instructions to give: the notebook splits before any fitting: 16,512 training rows, and 4,128 held-out rows halved into validation and test rows. Training rows fit the scaler, the polynomial features and the models; validation rows compare models and choose α; the test rows stay closed until §2.3, where the locked model is scored once. In §2 use Lasso(max_iter=10000) as the notebook suggests to avoid convergence warnings.

In §1.3 the notebook fits raw MedInc powers and then the same polynomials with standardized columns. Ask learners to predict what happens to training R^2 at degree 15; the debrief explains the difference.

Common errors to watch for: scaling before splitting; choosing α on the test rows, or reopening them after §2.3; reading lasso zeros as "irrelevant features". Fast finishers: replace the single validation split with 5-fold cross-validation on the training rows for the α choice (RidgeCV or LassoCV).

Lab 1 debrief: what the regression lab shows

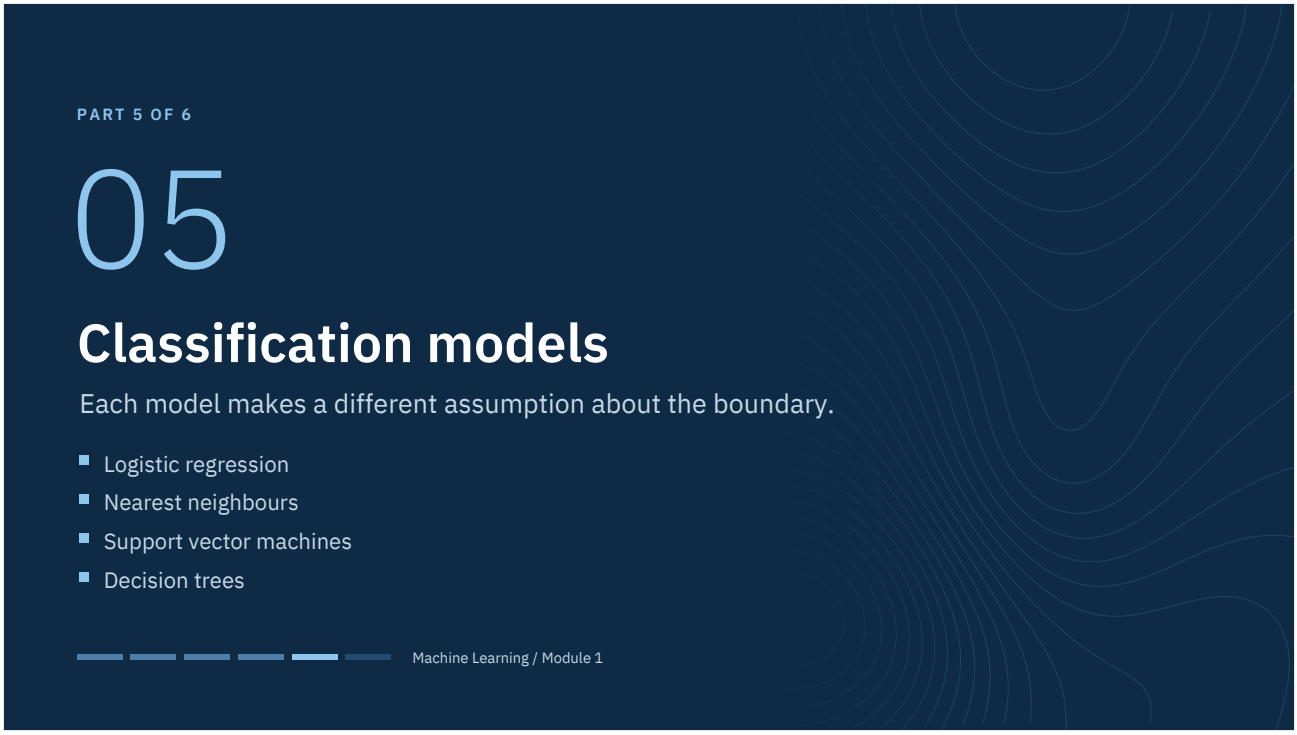


Debrief for the last 10 minutes before lunch. Numbers come from the solution notebook run in a fresh kernel with scikit-learn 1.9. Ask two teams to show their score tables; check that the baseline row comes first and that every row uses the same validation rows and metric. Validation scores: DummyRegressor R^2 −0.001, MAE 0.91 (units of 100,000 dollars); linear 0.59, MAE 0.53; degree 2 on the standardized features 0.66, MAE 0.46.

The surprise is §1.3. With raw MedInc powers, training R^2 rises from 0.477 (degree 1) to 0.491 (degree 5), then falls to 0.113 at degree 15. A bigger nested model cannot fit the training rows worse, so the solver failed on ill-conditioned powers up to 15^{15}. The failed value depends on the version (0.398 with scikit-learn 1.7; 1.9 truncates small singular values at LinearRegression's tol), and in 1.9 standardizing MedInc before the powers is not enough. With the polynomial columns standardized, degree 15 gives training 0.49 and validation 0.48. With 16,512 rows and one feature there is little room to overfit.

§2: lasso keeps 27 of 44 terms at $\alpha = 0.01$, 6 at 0.1, 1 at 1 and none from 10 upwards, where it equals the baseline. Ridge scores 0.654–0.664 on validation, best at $\alpha = 0.01$; that locked model scores test R^2 0.63 and MAE 0.47 (baseline 0.00 and 0.91). The candidates were nearly tied, so selection explains little of the drop from 0.66: the two halves of the held-out rows simply differ. Ask: is one validation split enough to choose α? No; the ridge scores differ by at most 0.01, less than that gap, so use cross-validation.

Classification models



Part 5 runs about 60 minutes. For each model family we ask the same three questions: what shape of boundary can it draw, what does it learn from the training rows, and what do we have to tune and preprocess?

Several figures reuse the same synthetic "moons" data so that boundaries can be compared; the facies examples use synthetic sand, shale and carbonate samples. None of this is a benchmark: the purpose is to see assumptions, not to crown a winner.

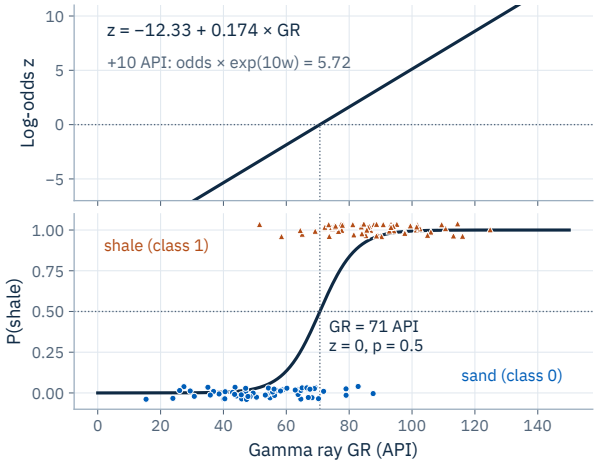
Logistic regression maps a score to a probability

CLASSIFICATION MODELS

Logistic regression maps a score to a probability

$$p = \sigma(z) = \frac{1}{1+e^{-z}}, \qquad z = b + w^\top x$$

- z is the **log-odds**, ln(p / (1 − p)), linear in the inputs
- Each +1 in an input multiplies the odds by e^w
- p = 0.5 exactly where z = 0



Synthetic sand and shale GR (60 each, seed 2); fitted weight 0.174 per API.

Machine Learning / Module 1 · Supervised learning foundations 56

Logistic regression is a linear model for the log-odds. It computes a score z = b + w^Tx exactly like linear regression, then squeezes it through the sigmoid σ into a probability between 0 and 1.

Top panel: the fitted log-odds z = −12.33 + 0.174 × GR is a straight line. Bottom panel: the same model as a probability of shale. At GR = 71 API, z = 0 and p = 0.5. Every extra 10 API multiplies the odds of shale by e^(10 × 0.174) ≈ 5.7.

Odds are p/(1 − p): p = 0.8 means odds of 4 to 1. Interpreting weights as odds multipliers is the classic strength of this model, with the same cautions as for linear weights: units and correlated inputs matter.

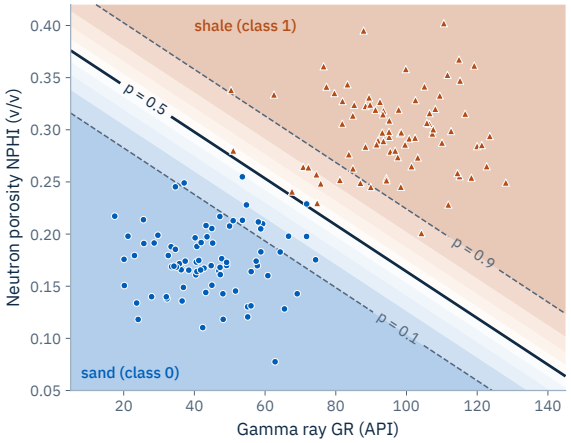
Ask: why not use linear regression on 0/1 labels? Its predictions can fall below 0 or above 1, and squared loss is a poor fit for probabilities. [3 min]

The decision boundary is where p equals 0.5

CLASSIFICATION MODELS

The decision boundary is where p equals 0.5

- With two inputs the **decision boundary** z = 0 is a straight line
- Probability contours run parallel to it
- Curved boundaries need basis expansion or another model



Synthetic sand and shale samples (80 each, seed 4); standardized logistic regression; contours of predicted probability.

Machine Learning / Module 1 · Supervised learning foundations 57

With two inputs, $z = b + w_1 \cdot GR + w_2 \cdot NPHI$, and the set where z = 0 is a straight line: the decision boundary at probability 0.5. The dashed lines are the 0.1 and 0.9 contours; they are parallel because every contour is a line z = constant.

Points close to the boundary get probabilities near 0.5; the model is uncertain there. Far from it, probabilities approach 0 or 1.

Two notes. First, "linear boundary" refers to the input space you give the model: adding squared terms or interactions (basis expansion from part 4) produces curved boundaries while the model stays linear in its weights. Second, a probability of 0.5 is not automatically the right decision threshold; the costs of false alarms and misses decide that, as part 6 shows. [2 min]

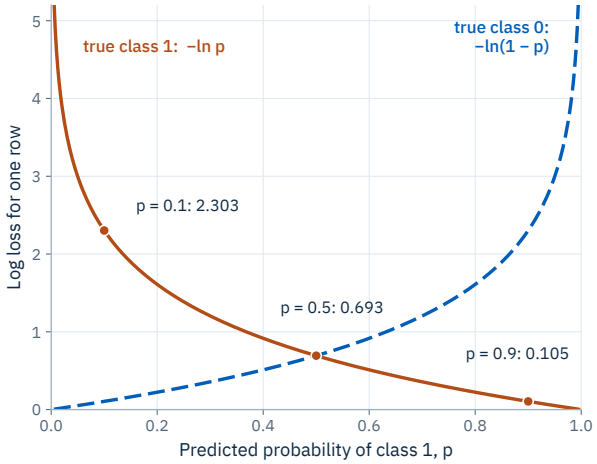
Log loss punishes confident mistakes

CLASSIFICATION MODELS

Log loss punishes confident mistakes

$$L = -[y \ln p + (1 - y) \ln(1 - p)]$$

- True shale predicted at p = 0.9 costs 0.105
- The same shale at p = 0.1 costs 2.303, about 22 times more
- Fitting minimizes the mean log loss with an iterative solver



Exact functions; marked values −ln 0.9, −ln 0.5 and −ln 0.1.

Machine Learning / Module 1 · Supervised learning foundations 58

Logistic regression is fitted by minimizing the average log loss, also called binary cross-entropy, over the training rows. For a row whose true label is 1, the loss is −ln p; for a true 0 it is −ln(1 − p).

Work the marked values: a true shale predicted at p = 0.9 costs −ln 0.9 = 0.105; at p = 0.5 it costs 0.693; at p = 0.1 it costs 2.303, about 22 times the confident correct case. As p approaches 0 for a true 1, the loss grows without bound. Confident mistakes dominate the total.

There is no closed-form solution, so solvers such as lbfgs iterate, using gradients as in part 4. scikit-learn adds an L2 penalty by default. The same loss trains classification neural networks in the second course, and Module 5 assesses probability calibration with reliability diagrams and the Brier score. [2 min]

A worked prediction from two standardized logs

CLASSIFICATION MODELS

A worked prediction from two standardized logs

Weights b = −0.4, w_{GR} = 1.5, w_{RHOB} = 0.8; one sample at GR = 1.2 and RHOB = −0.5 (z-scores).

1. **Score** — z = −0.4 + 1.5 × 1.2 + 0.8 × (−0.5) = −0.4 + 1.8 − 0.4 = 1.0
2. **Probability** — σ(1.0) = 1 / (1 + e^{-1}) ≈ 0.73
3. **Decision** — 0.73 ≥ 0.5, so predict shale
4. **Loss if the sample is shale** — −ln 0.73 ≈ 0.31
5. **Loss if the sample is sand** — −ln 0.27 ≈ 1.31

◆ GR adds +1.8 and RHOB −0.4 to the score: such comparisons only make sense on standardized inputs.

Machine Learning / Module 1 · Supervised learning foundations 59

Work each step on the board. The inputs are z-scores: the sample's GR is 1.2 standard deviations above the training mean and its density 0.5 below.

Score: z = b + w_GR × 1.2 + w_RHOB × (−0.5) = −0.4 + 1.8 − 0.4 = 1.0. Probability: σ(1.0) = 1/(1 + e^−1) = 1/(1 + 0.368) = 0.731. Decision: with a threshold of 0.5 we predict shale. Loss: if the sample really is shale, the loss is −ln 0.731 = 0.313; if it is sand, the loss is −ln(1 − 0.731) = −ln 0.269 = 1.313.

Ask: which input pushed the prediction towards shale? GR, contributing +1.8. Density contributed −0.4 because this sample is lighter than average, which points towards sand. A coefficient can only be read this way because both inputs share the same scale, one more benefit of standardization.

Common error: applying the sigmoid to each term separately instead of to the total score. [3 min]

Scale, fit, then read probabilities with predict_proba

CLASSIFICATION MODELS

Scale, fit, then read probabilities with predict_proba

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(C=1.0, max_iter=1000),
)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_val)[:, 1]
pred = (proba >= 0.5).astype(int)
```

- C is the inverse L2 strength: smaller C means stronger regularization
- More than two classes: a softmax over classes, by default
- Choose thresholds on validation rows, not 0.5 by habit

Machine Learning / Module 1 · Supervised learning foundations 60

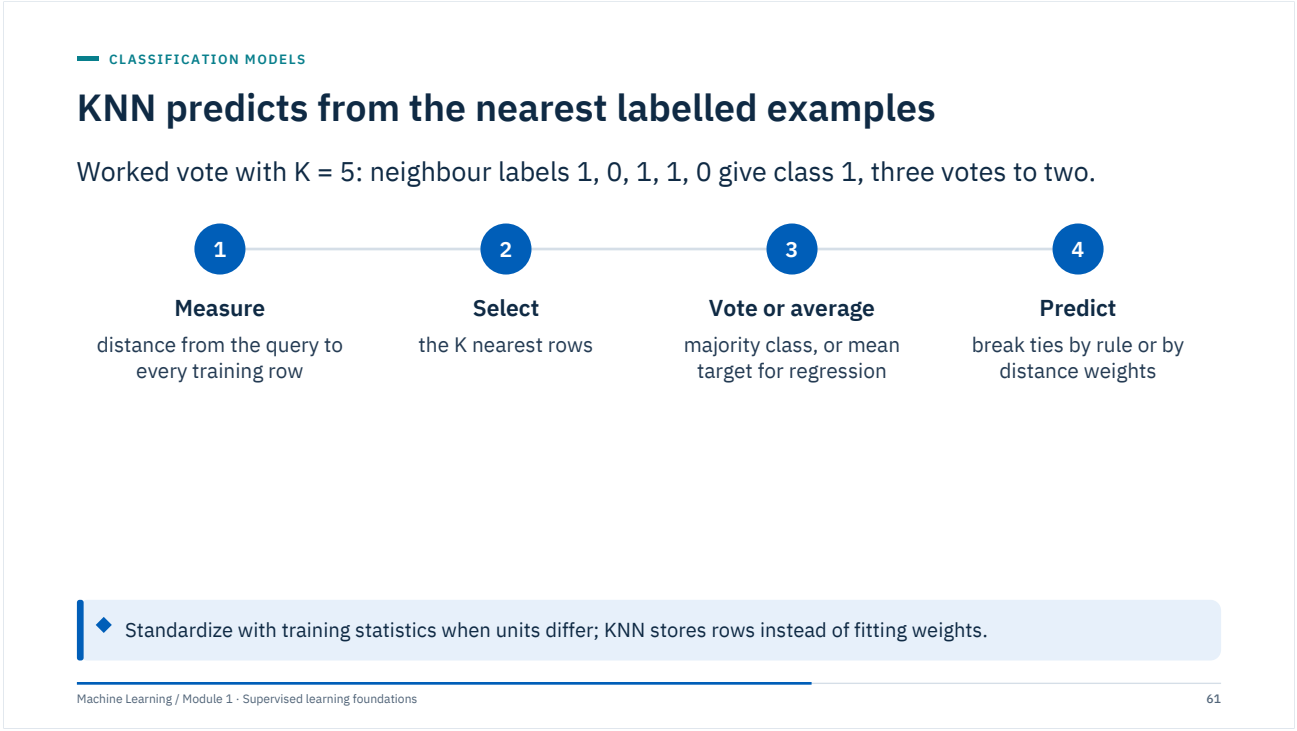
This is the complete pattern. The pipeline standardizes inputs with training statistics and fits logistic regression. predict_proba returns one column per class; column 1 is the probability of the positive class. Thresholding gives class predictions; clf.predict does the same with 0.5 for two classes.

C is the inverse of the regularization strength: C = 1 is moderate, C = 0.01 shrinks the weights strongly, and very large C approaches an unpenalized fit. Tune it like α, inside cross-validation. max_iter=1000 avoids convergence warnings on poorly scaled data.

For facies with five classes the same code works: with the default lbfgs solver the model fits a multinomial (softmax) model that outputs five probabilities per row, summing to 1.

X_train, y_train and X_val stand for your training and validation arrays. Ask: why is the threshold a separate decision from the model? Because it encodes the costs of false alarms and misses, which the model does not know. [2 min]

KNN predicts from the nearest labelled examples



K-nearest neighbours (KNN) has no training step beyond storing the rows. For a query it measures the distance to every stored row, takes the K closest and predicts by majority vote (classification) or by averaging their targets (regression).

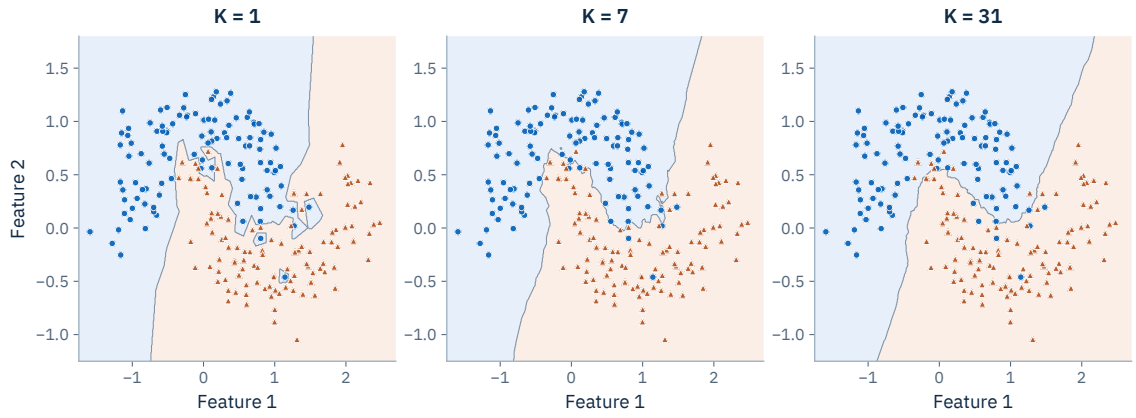
Work a vote: neighbour labels 1, 0, 1, 1, 0 give three votes for class 1 and two for class 0, so K = 5 predicts class 1. With an even K, ties need a rule; weights="distance" makes closer neighbours count more and breaks most ties.

Costs and caveats: prediction requires distances to all stored rows, which can be slow for large datasets at the wellsite. Missing values must be imputed before any distance is computed. Distances depend on feature scales, so standardization is part of the model. The Lab 2 extension notebook builds KNN from scratch with NumPy and checks it against scikit-learn. [2 min]

K controls the boundary's detail

CLASSIFICATION MODELS

K controls the boundary's detail



Fitted classifiers on the same synthetic data. Backgrounds show predictions; point shape and colour show observed class.

K is KNN's capacity dial, in the opposite direction to polynomial degree. K = 1 draws a boundary around every training point, including isolated islands for single noisy points: low bias, high variance. K = 7 smooths most islands away. K = 31 is smoother still and starts to cut across the curved moons: higher bias, lower variance.

Ask which differences would be unstable if we resampled the data: the small islands at K = 1. They come from individual points and would move with every new sample.

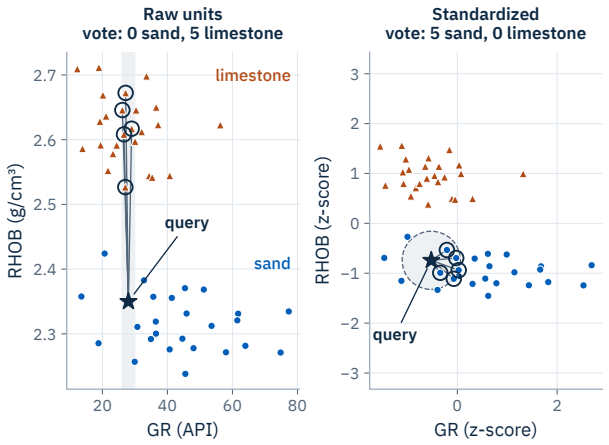
The right K depends on validation performance and on local sample density; choose it with grouped cross-validation for well data. Recall the motivating case: K = 1 with random folds looked excellent precisely because each validation sample's nearest neighbour was its own neighbour in depth. [2 min]

Scaling changes who your neighbours are

CLASSIFICATION MODELS

Scaling changes who your neighbours are

- GR spans tens of API units; RHOB spans tenths of a g/cm³
- Raw distance is almost pure GR: all 5 neighbours are limestone
- After standardization RHOB counts: all 5 neighbours are sand



Synthetic sand and limestone (25 each, seed 9); query at GR 28 API, RHOB 2.35 g/cm³ (star); K = 5.

Machine Learning / Module 1 · Supervised learning foundations 63

The query (star) is a low-GR, low-density sample: by its density it looks like sandstone. In raw units, a difference of 10 API in GR contributes 100 to the squared distance, while a difference of 0.25 g/cm³ in density contributes only 0.06. Distance is therefore almost pure GR, and the five nearest rows are the ones with GR close to 28, all limestone, despite their much higher density: the vote is 0 sand to 5 limestone.

After standardization, each feature is measured in its own standard deviations, density differences count, and all five neighbours are sandstone.

Standardization is itself a modelling choice: it gives every feature equal weight, which may not be right either. Domain weighting or feature selection can be better. Whatever the scaling, fit it on training rows inside each fold. [2 min]

Distance is a modelling choice

CLASSIFICATION MODELS

Distance is a modelling choice

Distance between two rows a and b over features j: KNN, RBF kernels and clustering all depend on it.

Euclidean

- $\sqrt{(\sum_j (a_j - b_j)^2)}$
- Large differences dominate; scale matters

Manhattan

- $\sum_j |a_j - b_j|$
- One large difference counts less

Cosine distance

- $1 - \cos\theta$, θ the angle between the rows
- Compares direction, not magnitude

◆ In many dimensions, nearest and farthest points become harder to tell apart (Module 4).

Machine Learning / Module 1 · Supervised learning foundations 64

Euclidean distance is the default and is dominated by features with large ranges or large differences. Manhattan distance sums absolute differences, so a single large difference contributes linearly instead of quadratically. Cosine distance compares the direction of two vectors and ignores their length.

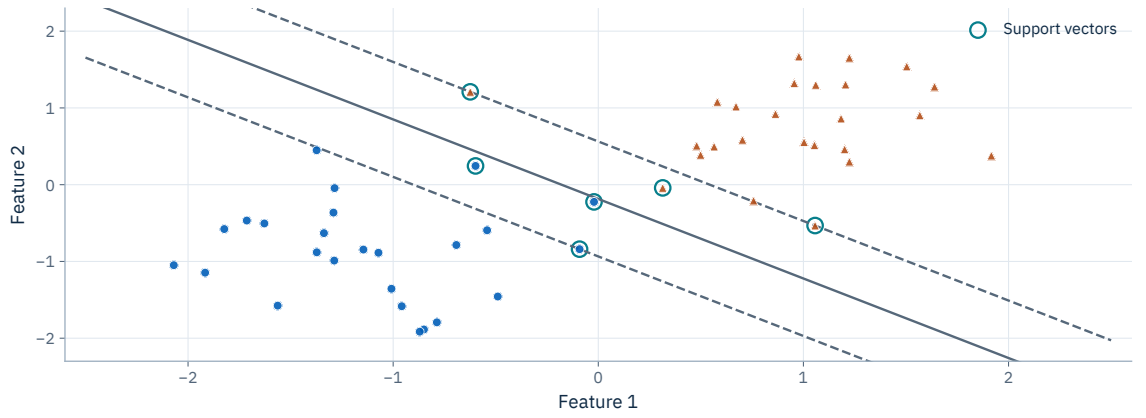
Ask whether two production profiles with the same shape but different magnitudes should count as similar. With cosine distance they are identical; with Euclidean distance they are far apart. The right answer depends on the decision: shape of decline, or volume?

The note at the bottom previews the curse of dimensionality: with many irrelevant features, distances concentrate and "nearest" loses meaning. Module 4 covers this with feature selection and dimensionality reduction. [2 min]

SVM separates classes with a margin

CLASSIFICATION MODELS

SVM separates classes with a margin



Linear SVM fitted to synthetic data. Dashed lines mark the margins; circled observations are support vectors.

A support vector machine (SVM) looks for the separating line, or hyperplane, with the widest margin: the widest empty band between the classes. The solid line is the decision boundary; the dashed lines are the margins.

Only the circled points, the support vectors, determine the boundary: move any other point without crossing the margin and the fit does not change. The geometric margin width is 2 divided by the norm of the weight vector, so a wide margin means small weights, a form of regularization.

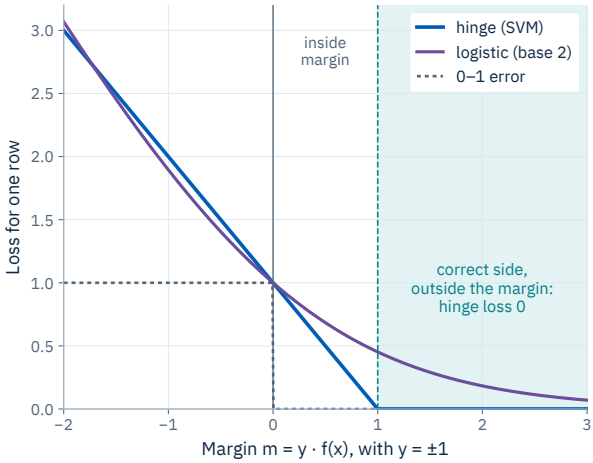
In this soft-margin fit, three circled points lie inside the margin and three exactly on it. None is on the wrong side here, but a soft margin allows that too; every violation costs a penalty controlled by C. Ask: what happens if you add a far-away point on the correct side? Nothing: it is not a support vector. [2 min]

Hinge loss ignores points beyond the margin

CLASSIFICATION MODELS

Hinge loss ignores points beyond the margin

- Margin m = y·f(x), with labels y = −1 or +1
- **Hinge loss** max(0, 1 − m) is zero once m ≥ 1
- Only rows on or inside the margin shape the fit: the support vectors
- Logistic loss never reaches zero, so every row contributes



Exact loss functions of the margin m = y·f(x).

Machine Learning / Module 1 · Supervised learning foundations 66

Write the SVM score as f(x) and code the labels as −1 and +1. The margin m = y·f(x) is positive when the row is on the correct side and larger the further inside its own class it is.

The SVM minimizes the hinge loss max(0, 1 − m) plus a penalty on the weights. Rows with m ≥ 1, correct and outside the margin, cost nothing and do not influence the fit; that is exactly why only support vectors matter. Rows inside the margin or misclassified pay linearly.

Logistic regression's loss, drawn here in base 2 so it passes through 1 at m = 0, is a smooth version that never reaches zero: every row pulls a little. Both are convex surrogates for the 0–1 error, which counts mistakes but cannot be optimized directly.

Consequence: SVM scores are not probabilities. SVC(probability=True) adds an internal calibration step, whose evaluation belongs to Module 5. [3 min]

The C parameter trades margin for violations

CLASSIFICATION MODELS

The C parameter trades margin for violations

$$\min_{w,\,b} \tfrac{1}{2}\|w\|^2 + C\sum_i \max(0,\, 1 - y_i f(x_i))$$

Small C

- Violations are cheap: wider margin
- More support vectors, smoother boundary

vs

Large C

- Violations are expensive: narrower margin
- Chases hard training rows; can fit noise

◆ Search C and γ together after scaling; SVM scores are not probabilities by default.

Machine Learning / Module 1 · Supervised learning foundations 67

Here f(x) = $w^T x + b$ is the SVM score and ‖w‖ the length of the weight vector; minimizing ½‖w‖² widens the margin 2/‖w‖. C multiplies the total hinge loss relative to that margin penalty. With small C, violating the margin is cheap, so the SVM prefers a wide margin with more points inside it: more support vectors and a smoother, more regularized boundary. With large C, violations are expensive, so the margin narrows to classify more training rows correctly, which can mean fitting noise.

Avoid the slogan "larger C always overfits": the effect depends on the data, the kernel and γ. Search C on a logarithmic grid (0.1, 1, 10, 100), jointly with γ for the RBF kernel, inside cross-validation and after scaling, which SVMs require because the kernel uses distances.

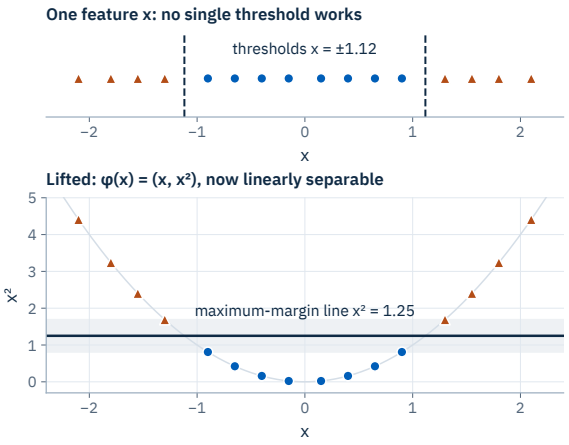
Note the relationship with logistic regression's C in scikit-learn: in both, C is an inverse regularization strength. [2 min]

The kernel trick lifts data into a separable space

CLASSIFICATION MODELS

The kernel trick lifts data into a separable space

- One feature, class 0 in the middle: no single threshold works
- Map x to (x, x²): a straight line now separates the classes
- Back in x, that line becomes two thresholds at ±1.12
- The **kernel trick** computes such similarities without building φ(x)



Symmetric toy data, 8 points per class; maximum-margin line x² = 1.25 in the lifted space.

Top: class 0 (blue) sits between two groups of class 1 (orange) on one feature. No single threshold separates them.

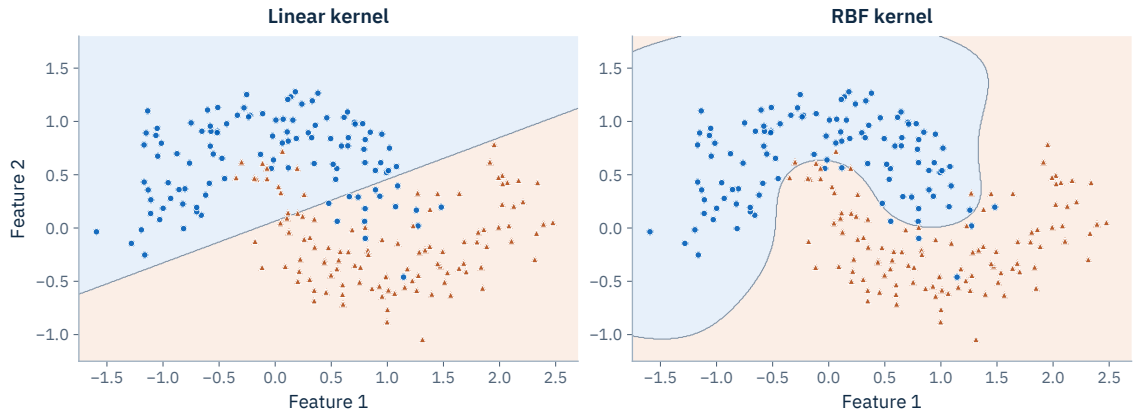
Bottom: add the feature x². Each point moves onto the parabola, and now a horizontal line separates the classes. The maximum-margin line lies midway between the highest blue point (0.81) and the lowest orange point (1.69), at x² = 1.25. Mapped back to x, one straight line in the lifted space becomes two thresholds, x = ±1.12.

The kernel trick: an SVM needs only dot products between rows. A kernel function computes the dot product in a lifted feature space directly from the original inputs, so the lift never has to be built explicitly. A polynomial kernel corresponds to a space of polynomial terms; the RBF kernel corresponds to an infinite-dimensional space. Choosing a kernel does not remove the need for scaling and tuning. [3 min]

Kernels change the space of boundaries

CLASSIFICATION MODELS

Kernels change the space of boundaries



Linear and RBF SVM fits on the same synthetic observations. The RBF model can represent a curved boundary.

Same data, two kernels. The linear kernel draws a straight boundary and misclassifies the tips of both moons. The RBF (radial basis function) kernel measures similarity as a decreasing function of distance, so the boundary can curve around the data.

A polynomial kernel is another option; it corresponds to polynomial basis expansion and can be unstable at high degree. A linear kernel is often a strong choice when there are many features relative to rows.

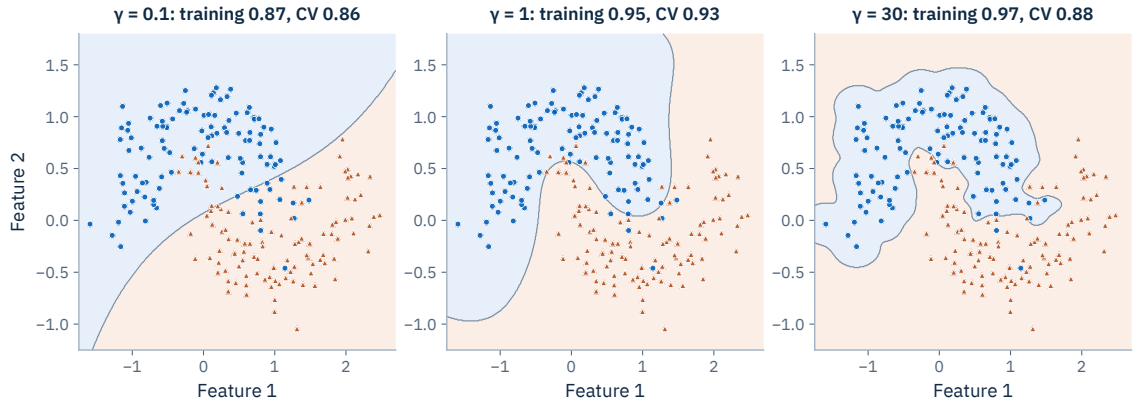
Ask: why does the RBF kernel need standardized inputs? Because it is built on Euclidean distance, the same issue we saw with KNN. [1 min]

Gamma sets how far each example's influence reaches

CLASSIFICATION MODELS

Gamma sets how far each example's influence reaches

RBF kernel exp(−γ × squared distance): larger γ gives each training row a more local influence.



Same synthetic moons as the shared figures (seed 42); standardized RBF SVM with C = 1; accuracy on training rows and 5-fold stratified CV.

Machine Learning / Module 1 · Supervised learning foundations 70

γ sets the width of each training row's influence. With γ = 0.1 the influence is wide and the boundary almost linear: training accuracy 0.87, cross-validated 0.86 (underfitting). With γ = 1 the boundary follows the moons: training 0.95, CV 0.93. With γ = 30 each support vector influences only its immediate surroundings and the boundary wraps tightly around the training points: training accuracy rises to 0.97 while CV drops to 0.88 (overfitting).

The pattern mirrors K in KNN and degree in polynomials: one hyperparameter moves the model from too smooth to too wiggly, and only held-out data can place it.

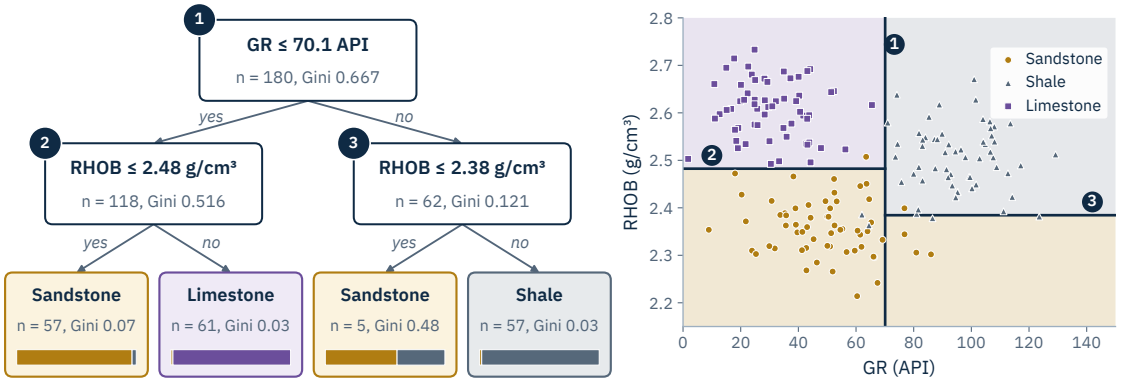
In scikit-learn the default gamma="scale" sets γ from the number of features and the variance of the training data, a sensible starting point. Tune γ together with C. [2 min]

Trees ask a sequence of threshold questions

CLASSIFICATION MODELS

Trees ask a sequence of threshold questions

Each internal node tests one feature against a threshold; each leaf predicts its training majority.



Synthetic facies samples (60 per class, seed 2); a depth-2 tree drawn from scikit-learn's fitted tree_ arrays.

Machine Learning / Module 1 · Supervised learning foundations 71

Left: the fitted tree. Split 1 asks whether GR ≤ 70.1 API; low-GR samples go left. Split 2 separates sandstone from limestone by density at 2.48 g/cm³. Split 3 separates a small group of high-GR, low-density samples from shale at 2.38 g/cm³. Each box shows the number of training samples and its Gini impurity; the coloured bars show the class mix in each leaf. Training accuracy is 0.967.

Right: the same tree as a partition of the GR–density plane into axis-aligned rectangles, one per leaf. That picture explains both strengths and weaknesses: trees capture thresholds and interactions naturally, but approximate an oblique boundary with a staircase.

Point to the third leaf: only 5 samples, with Gini 0.48. Leaves this small give fragile predictions; min_samples_leaf controls them (this tree was grown with min_samples_leaf=5, the smallest leaf it allows). Trees do not need scaling, since they only compare a feature with a threshold. [3 min]

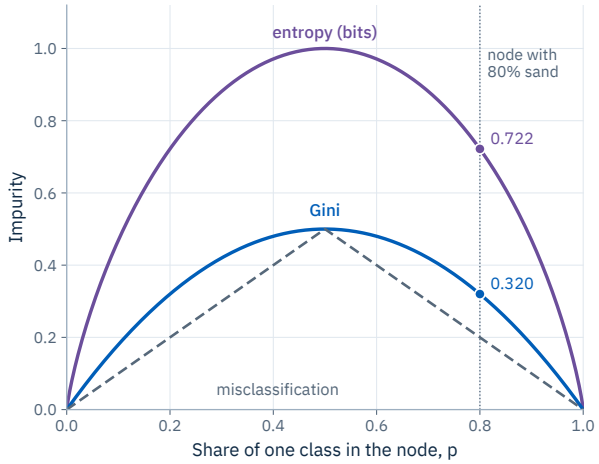
Impurity measures how mixed a node is

CLASSIFICATION MODELS

Impurity measures how mixed a node is

$$\text{Gini} = 1 - \sum_k p_k^2, \qquad H = -\sum_k p_k \log_2 p_k$$

- Both are zero for a pure node and largest at 50/50
- A split is scored by the drop in size-weighted child **impurity**
- Gini and entropy usually choose similar splits



Exact two-class impurity functions; marked values at p = 0.8.

Machine Learning / Module 1 · Supervised learning foundations 72

p_k is the share of class k in the node. Gini impurity is 1 minus the sum of squared shares; entropy H sums $-p \log_2 p$. For two classes, Gini is $2p(1 - p)$, with a maximum of 0.5 at p = 0.5; entropy has a maximum of 1 bit.

Worked values at the marked point, a node with 80% sand: Gini = $1 - 0.8^2 - 0.2^2 = 0.32$; entropy = $-0.8 \log_2 0.8 - 0.2 \log_2 0.2 = 0.722$ bits.

To choose a split, the tree computes the impurity of each child, weights it by the child's share of samples and subtracts the result from the parent's impurity. The split with the largest decrease wins. Misclassification error is also shown: it is less sensitive to changes in the class mix, which is why it is rarely used to grow trees. In scikit-learn, criterion="gini" is the default and criterion="entropy" the alternative; they usually agree. [2 min]

Choose the better first split

PRACTICE

5 min Pairs

Choose the better first split

Candidate split	Left child: sand / shale	Right child: sand / shale
A: GR ≤ 60 API	8 / 2	2 / 8
B: RHOB ≤ 2.40 g/cm³	9 / 5	1 / 5

1. The parent holds 10 sand and 10 shale: compute its Gini impurity.
2. Compute each child's Gini, weight by child size, and pick the split.

Machine Learning / Module 1 · Supervised learning foundations 73

Give 4 minutes; work it on the board [5 min].

Parent: $1 - 0.5^2 - 0.5^2 = 0.5$.

Split A: left child 8/2, Gini = $1 - 0.8^2 - 0.2^2 = 0.32$; right child 2/8, also 0.32. Weighted: $(10 \times 0.32 + 10 \times 0.32)/20 = 0.32$. Decrease: $0.5 - 0.32 = 0.18$.

Split B: left child 9/5 of 14, Gini = $1 - (9/14)^2 - (5/14)^2 = 1 - 0.413 - 0.128 = 0.459$; right child 1/5 of 6, Gini = $1 - (1/6)^2 - (5/6)^2 = 1 - 0.028 - 0.694 = 0.278$. Weighted: $(14 \times 0.459 + 6 \times 0.278)/20 = (6.43 + 1.67)/20 = 0.405$. Decrease: 0.095.

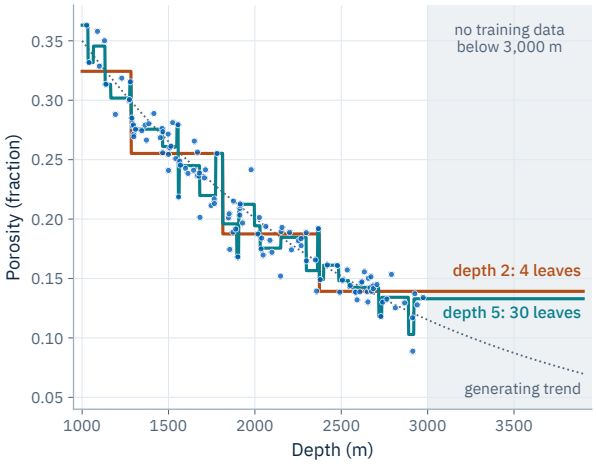
Split A wins with a decrease of 0.18 against 0.095. Split B's right child is purer, but it holds only 6 of 20 samples; the weighting rewards splits that clean up many samples. Common errors: forgetting to weight by child size, and using counts instead of shares inside the squares.

Regression trees predict a constant in each leaf

CLASSIFICATION MODELS

Regression trees predict a constant in each leaf

- Each leaf predicts the mean target of its training rows
- Splits are chosen to reduce the squared error
- The fit is a staircase and cannot extrapolate beyond the data
- Depth and min_samples_leaf set the number of steps



Synthetic compaction trend (seed 10), 120 samples; trees of depth 2 and 5.

Machine Learning / Module 1 · Supervised learning foundations 74

For a numeric target, a tree uses the same recursive splitting but scores splits by the reduction in squared error, and each leaf predicts the mean of its training targets. The result is a step function: here porosity versus depth for a synthetic compaction trend.

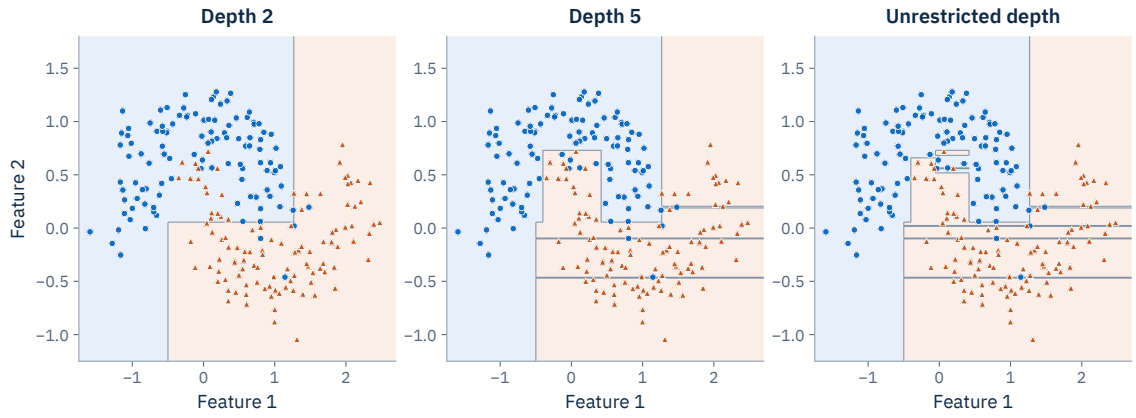
Depth 2 gives 4 leaves, a coarse staircase that misses the curvature. Depth 5 gives 30 leaves and starts to follow individual noisy samples. The right size is again a validation question.

Two properties matter for energy data. Trees cannot extrapolate: beyond 3,000 m, the grey band with no training data, both trees stay flat at their last leaf's value while the dotted generating trend keeps falling. And steps create discontinuities that can be unphysical for smooth quantities. Ensembles of trees in Module 2 smooth the steps but keep the extrapolation limit. [2 min]

Depth controls tree complexity

CLASSIFICATION MODELS

Depth controls tree complexity



Trees fitted to the same synthetic observations. More depth permits smaller regions and finer partitions of the training data.

Depth 2 draws a few large rectangles and misclassifies parts of both moons. Depth 5 follows the curved shapes with a staircase. Unrestricted depth keeps splitting until every leaf is pure, carving thin slivers around single training points: a classic overfit.

Controls in scikit-learn: max_depth limits depth; min_samples_leaf sets a minimum leaf size; max_leaf_nodes caps the number of leaves; ccp_alpha applies cost-complexity pruning after growing. All are hyperparameters for grouped cross-validation.

A shallow tree is easy to inspect, which is valuable, but interpretability does not guarantee correctness or stability: small changes in the data can change the top split and the whole tree below it. Module 2 turns that instability into an asset with bagging and random forests. [2 min]

Lab 2: classifiers and a facies split design

LABORATORY

45 min

Lab 2: classifiers and a facies split design

- ☐ §3: KNN on Iris; plot training and CV accuracy against K
- ☐ §4: trees of depth 1, 2, 3, 5 and unlimited; lock one model, test it once
- ☐ Facies notebook §2–4: audit missing and implausible values by well
- ☐ Facies notebook §6 and §9: write a well-grouped evaluation and tuning plan

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- Accuracy against K and depth, with a majority-class baseline
- A facies data-audit table by well
- A split and tuning plan for unseen wells

Machine Learning / Module 1 · Supervised learning foundations 76

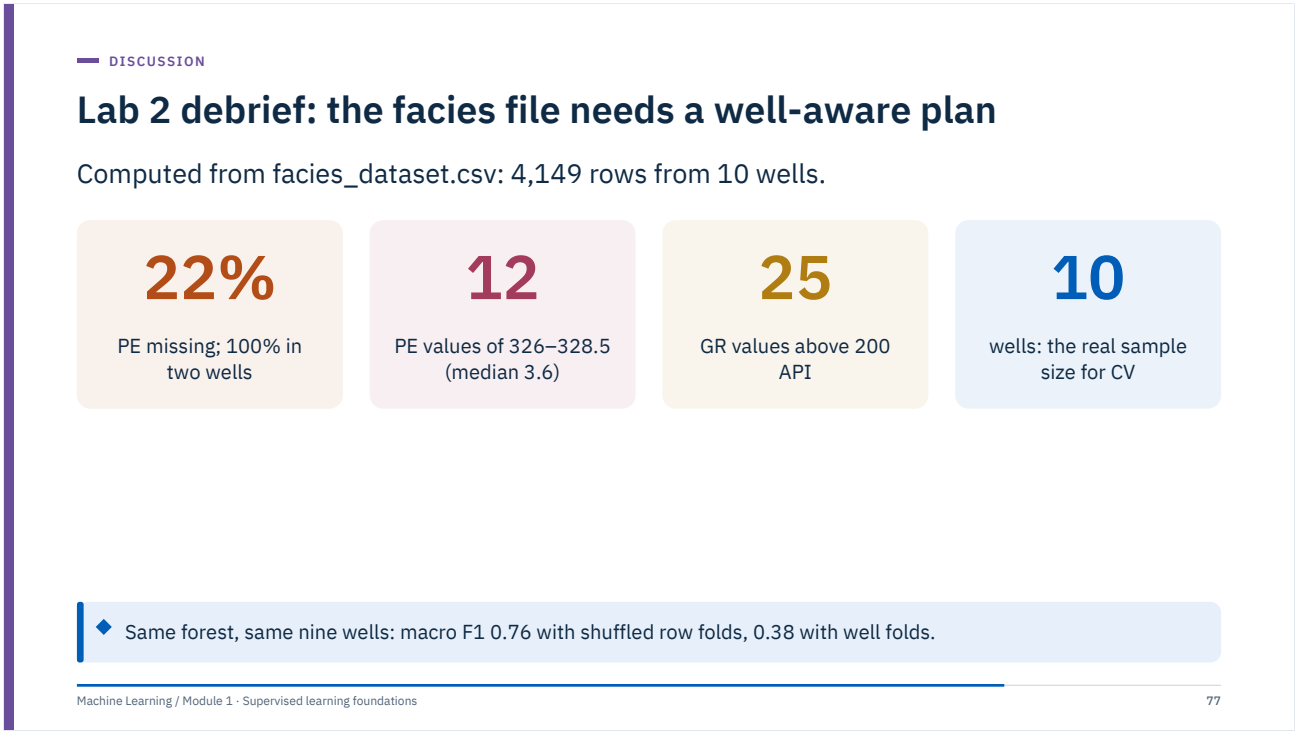
Timing: 45 minutes, then a 10-minute debrief. Suggested split: 20 minutes on sections 3 and 4 of the basics notebook, 25 minutes on the facies notebook in exercises/04_facies_classification.

Basics notebook: Iris is small and clean; its purpose here is mechanics. The notebook compares K and depth with 5-fold stratified cross-validation on the 120 training rows, next to a DummyClassifier baseline, and opens the 30 test rows once, in §4.3, for the locked choice. Ask learners how many rows each fold and the test set hold (24 and 30).

Facies notebook: do not aim to finish the random forest. Load and merge the files, then build the audit table per well in §2.1: rows, missing GR and PE, and values outside plausible ranges (compare each curve's maximum with its median). Then read section 6 (SHANKLE held out as the test well), section 7 (baseline and default forest on GroupKFold folds of whole wells, with shuffled row folds for contrast) and section 9 (grouped tuning with groups passed to fit), and write their own plan for keeping wells apart. The notebook uses seaborn, which is in requirements-labs.txt.

Extension for fast finishers: the KNN-from-scratch notebook (exercises/02_knn_from_scratch). Common errors: dropping every row with a missing PE (which deletes two wells) and treating Well Name as a feature.

Lab 2 debrief: the facies file needs a well-aware plan



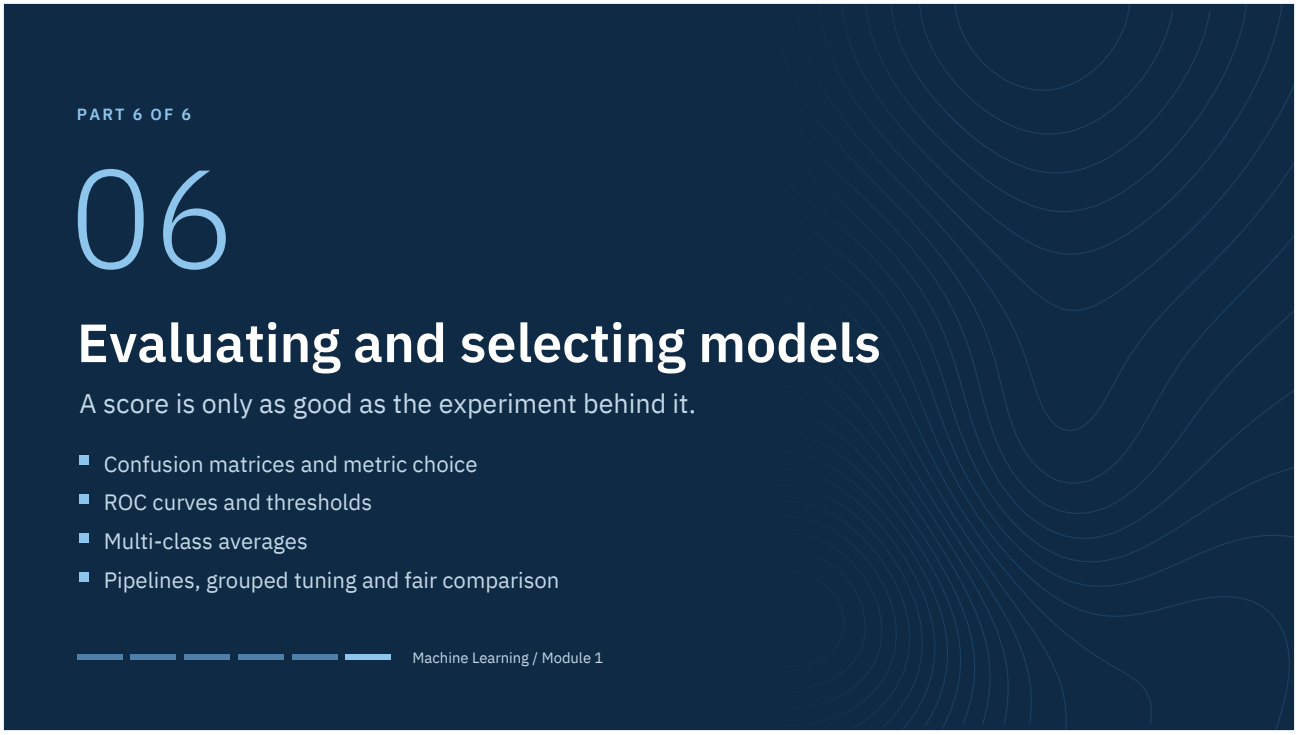
Numbers computed from the supplied file. PE is missing in 917 of 4,149 rows (22.1%): all of ALEXANDER D and KIMZEY A and 15% of Recruit F9. Median imputation alone would invent PE for two wells that never had it; the notebook adds missing-value indicators inside its pipeline, and models with and without PE are another option. Twelve PE values lie between 326 and 328.5 against a median of 3.56 (the notebook sets PE above 20 to missing), and 25 GR values exceed 200 API (maximum 361): flag them before training. GR is missing in 7.8% of rows, 5–10% in every well, a pattern that looks random and deserves a question to the data owner.

Other findings: Recruit F9 has only 80 rows, all baffestone, with depths out of order, so treat it with care as a group; the Description column gives depths in metres at 0.5 m spacing, as the notebook states, so confirm units before any depth-derived feature.

Iris (120 training rows, 5-fold stratified CV): every K scores 0.94–0.96, within two rows; trees stop improving at depth 3 (0.93); the baseline scores 0.33. The locked K = 5 scores 30/30 on the test rows, opened once; one error would cost 3.3 points, so this test set can confirm a choice but could not rank the candidates.

Facies design (sections 6–9): SHANKLE is the test well; baseline, default forest and grid search use GroupKFold(3) on the nine training wells, with groups passed to fit. Well folds give the default forest macro F1 0.38 (baseline 0.04); shuffled row folds report 0.76 for the same model. Tuning reaches 0.40, within the fold spread; SHANKLE, opened once, gives accuracy 0.49 and macro F1 0.38.

Evaluating and selecting models



Part 6 runs about 35 minutes and then Lab 3 puts it into practice. We move from single numbers to evidence: counts, per-class metrics, thresholds and fold-score distributions.

Module 5 extends this part with precision–recall curves, calibration, rare-event evaluation, nested cross-validation and statistical model comparison; today establishes the habits those tools build on.

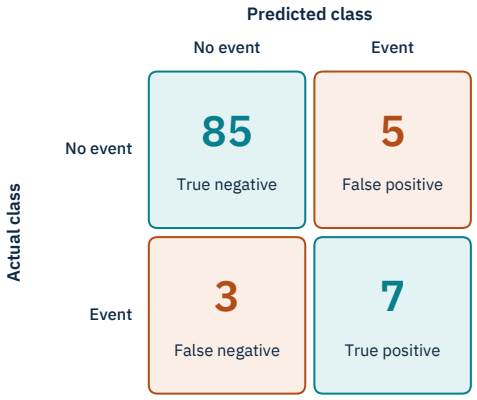
Opening question: in your last model review, what was reported besides a single score? Most answers are "nothing". By the end of this part every team should report per-class results, all fold scores and one test score for a locked pipeline.

A confusion matrix preserves the counts

EVALUATING AND SELECTING MODELS

A confusion matrix preserves the counts

- Rows are actual classes; columns are predictions
- False positives are false alarms; false negatives are missed events
- State which class is the event before quoting any metric
- Some tools transpose this layout: check before reading



Illustrative counts: 100 observations, 10 actual events. Not model performance.

Machine Learning / Module 1 · Supervised learning foundations 79

A confusion matrix counts every combination of actual and predicted class. With "event" as the positive class: true negatives (85) are correctly quiet, false positives (5) are false alarms, false negatives (3) are missed events, and true positives (7) are caught events.

Ask learners to locate the false alarms and the missed events, then ask which costs more for an ESP trip warning. Usually the miss: a trip costs production and possibly a workover, while a false alarm costs an inspection. For other decisions the balance reverses.

Every classification metric is a function of these four counts, so always keep them. Conventions differ between libraries and papers; scikit-learn's confusion_matrix puts actual classes in rows and predictions in columns, as here. [2 min]

Accuracy can hide the minority class

PRACTICE

4 min Pairs

Accuracy can hide the minority class

Using TN = 85, FP = 5, FN = 3, TP = 7:

1. Calculate accuracy, event precision and event recall.
2. Compare with a model that always predicts no event.
3. Which error would matter most for the proposed use?

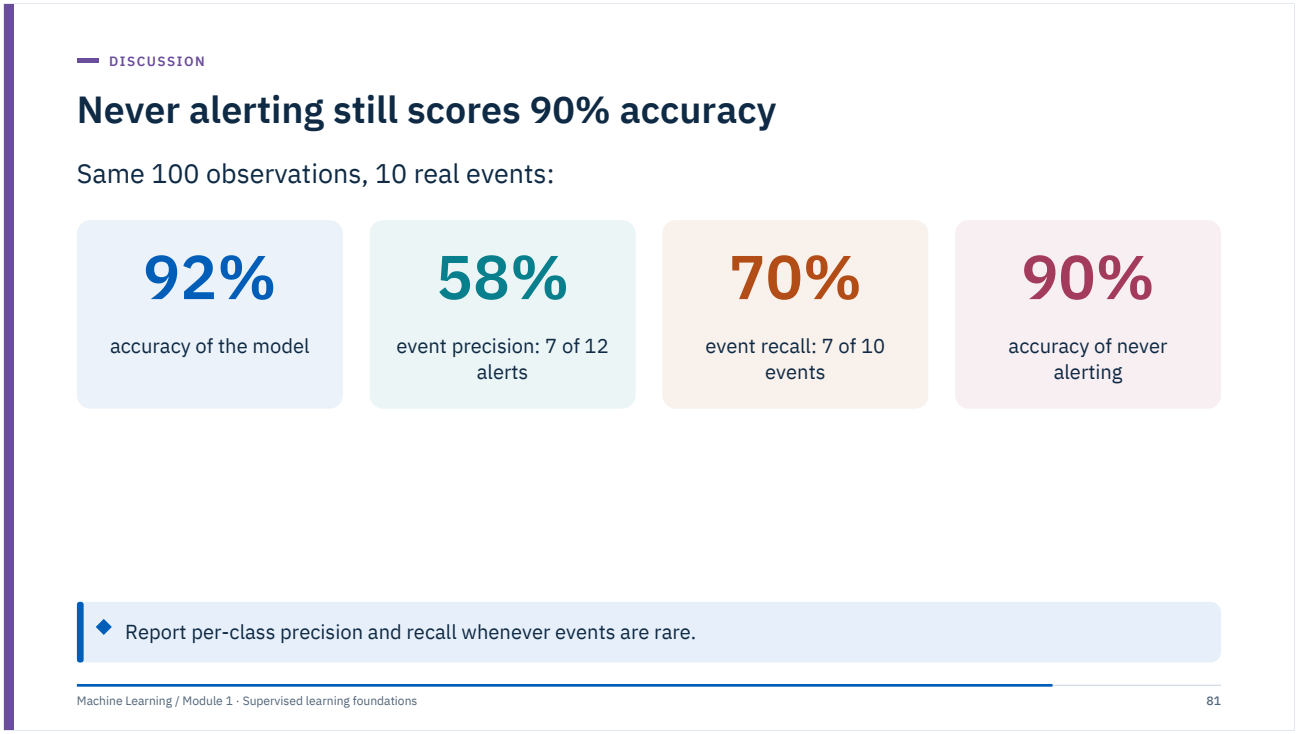
Allow 3 minutes and require learners to write each denominator. Then reveal the next slide [4 min].

Accuracy = (TN + TP)/total = (85 + 7)/100 = 92%. Precision = TP/(TP + FP) = 7/12 = 58.3%: of 12 alerts, 7 were real. Recall = TP/(TP + FN) = 7/10 = 70%: of 10 events, 7 were caught.

The always-negative model predicts no event for all 100 rows: TN = 90, FN = 10, TP = FP = 0. Its accuracy is 90%, only 2 points below our model, but its recall is 0% and it produces no useful alerts. Its precision is undefined (0/0).

The third question has no single answer; business costs are context-specific and should come from the decision owner, not be assumed by the analyst. Common error: computing precision with FN in the denominator.

Never alerting still scores 90% accuracy

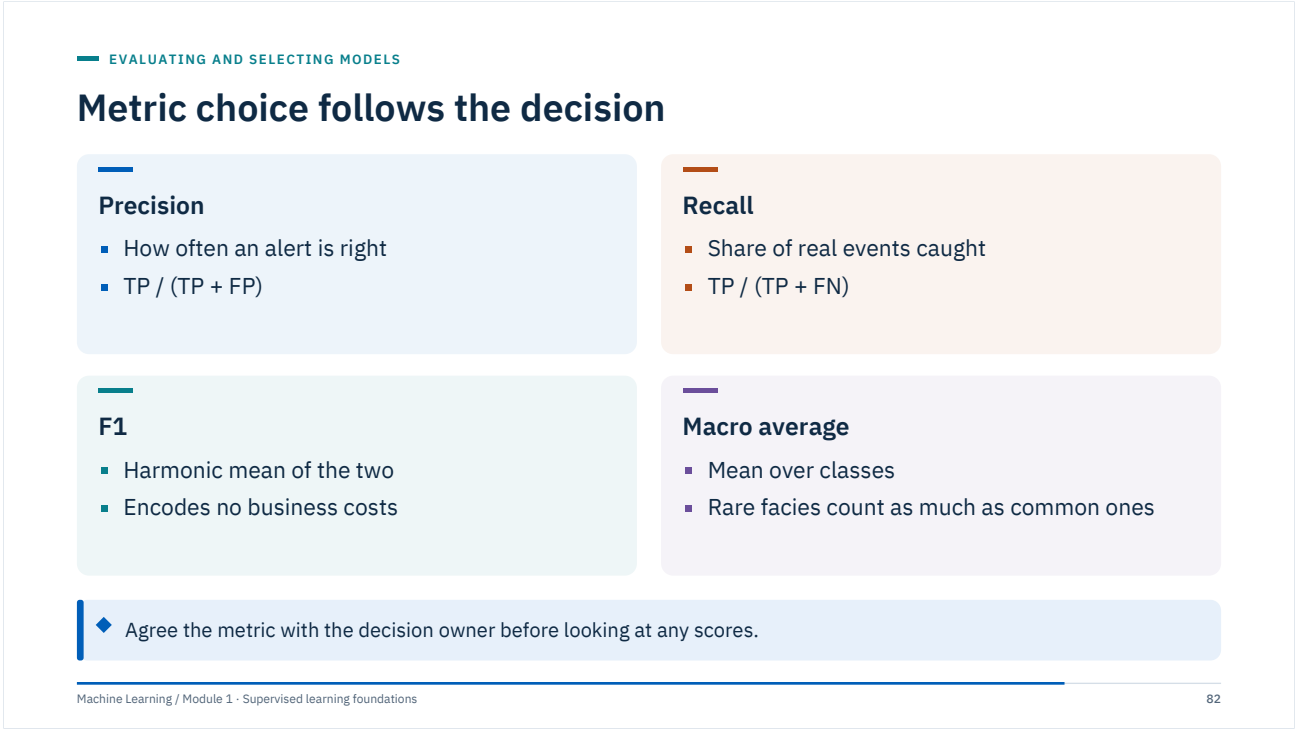


Put the four numbers side by side. The model's 92% accuracy sounds excellent until you see that a model that never raises an alert scores 90%. With 10% events, accuracy is dominated by the easy majority class.

Precision and recall describe the event class directly. Precision answers "when it alerts, how often is it right?" Recall answers "of the real events, how many did it catch?" Both depend on the threshold, and improving one usually costs the other.

Tie back to the baselines of part 1: the majority-class baseline belongs in every classification score table precisely because of this effect. Module 5 develops rare-event evaluation, including event-level metrics where one alert per event counts once. [2 min]

Metric choice follows the decision



Summarize the four tools. Precision matters when each alert triggers an expensive action, such as a crew visit. Recall matters when missing an event is costly, such as a pump trip or a leak. F1, the harmonic mean of precision and recall, is a convenient single number, but it weights the two equally and ignores the actual costs of each error type.

For multi-class problems such as facies, compute each metric per class and then average. The macro average gives every class equal weight; the weighted average weights by class frequency and is dominated by common classes. The next slides show the difference on synthetic facies.

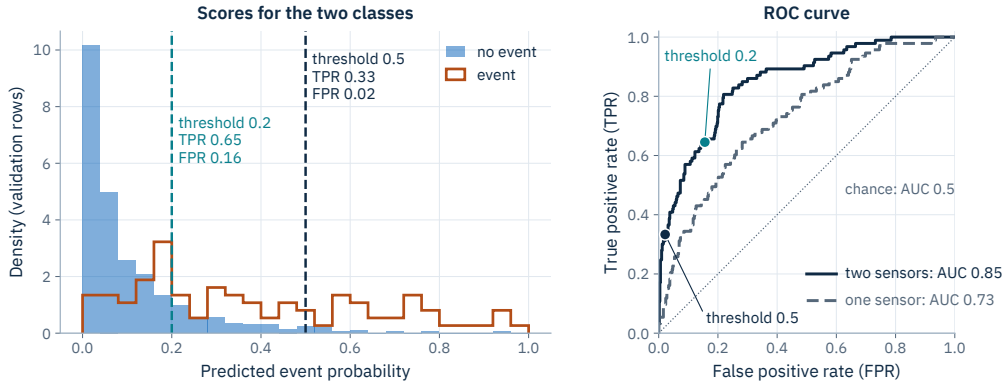
The threshold changes precision and recall without changing the model's ranking of rows; ROC curves, next, summarize ranking over all thresholds. [2 min]

ROC curves summarize every threshold at once

EVALUATING AND SELECTING MODELS

ROC curves summarize every threshold at once

Moving the threshold trades false alarms for missed events; **AUC** summarizes the ranking.



Synthetic ESP-trip data, 15% events (seed 13); logistic regression trained on 1,000 rows, evaluated on 600 validation rows. Precision–recall curves for rare events: Module 5.

Machine Learning / Module 1 · Supervised learning foundations 83

Left: the predicted event probability for the 600 validation rows, by true class. Most non-events score low, events spread widely. At the default threshold of 0.5 the model catches only a third of the events (TPR 0.33) with 2% false alarms (FPR 0.02). Lowering the threshold to 0.2 catches 65% of events at 16% false alarms.

Right: the ROC curve plots the true positive rate (recall) against the false positive rate for every threshold. The two operating points are marked. The area under the curve (AUC) is 0.85 with two sensors and 0.73 with one; 0.5 is chance, the dotted diagonal. AUC equals the probability that a random event scores higher than a random non-event.

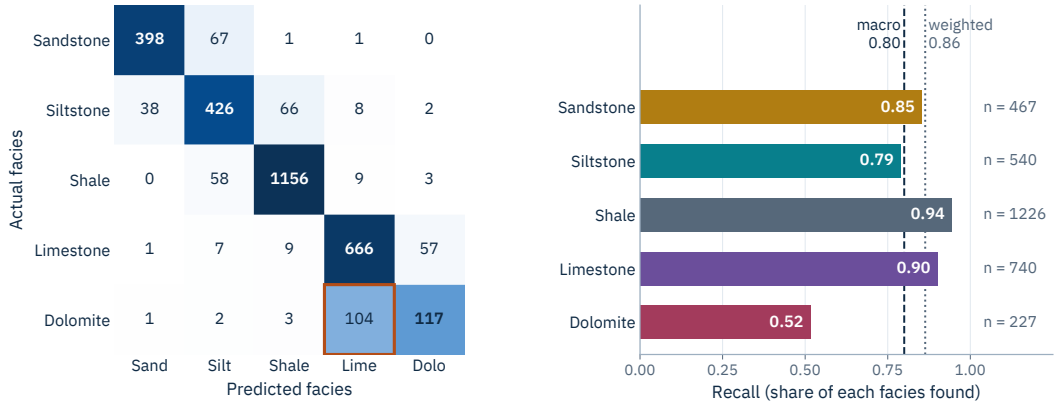
Cautions: AUC measures ranking, not calibration or cost; it says nothing about which threshold to use. With rare events, the false positive rate can look small while false alarms still outnumber true alerts, which is why Module 5 prefers precision–recall curves for rare events. [3 min]

Multi-class errors hide off the diagonal

EVALUATING AND SELECTING MODELS

Multi-class errors hide off the diagonal

Dolomite is found only about half the time; most of its misses are called limestone.



Synthetic 8-well facies data (seed 7); standardized RBF SVM, leave-one-well-out predictions. Colour = share of each actual facies.

Machine Learning / Module 1 · Supervised learning foundations 84

The matrix now has five classes: rows are actual facies, columns predicted, and the colour shows the share of each row. Most mass sits on the diagonal. The largest off-diagonal cell is outlined: 104 dolomite samples predicted as limestone, against 117 dolomite samples found.

The right panel turns each row into a recall: shale 0.94, limestone 0.90, sandstone 0.85, siltstone 0.79 and dolomite 0.52. Dolomite is the rarest facies (227 of 3,200 samples) and overlaps limestone in these logs.

These are geologically sensible confusions: neighbouring facies with similar log responses. In the lab data the notebook mentions similar confusions between siltstone classes. The confusion matrix tells you where to look: here, a better dolomite indicator (for example PE, if run) or a decision to merge classes if the application allows. [3 min]

Macro and weighted averages answer different questions

EVALUATING AND SELECTING MODELS

Macro and weighted averages answer different questions

Recall on the synthetic facies example, from the same leave-one-well-out predictions:

0.52–0.94	0.80	0.86
per facies: one value per class	macro: every facies counts equally	weighted by frequency: equals accuracy

◆ A rare facies found poorly drags the macro average down; the weighted average hides it.

Machine Learning / Module 1 · Supervised learning foundations 85

Computed from the same leave-one-well-out predictions. The macro average of the five recalls is (0.85 + 0.79 + 0.94 + 0.90 + 0.52)/5 = 0.80. The weighted average multiplies each recall by its class share; for recall this equals overall accuracy, 0.86, because the weights cancel the class sizes.

Which to report? Both, plus the per-class values. If every facies matters to the decision, as in reservoir modelling where thin carbonates may be key, the macro average is the honest single number. If the decision is about total footage correctly classified, the weighted average fits.

scikit-learn's classification_report prints per-class precision, recall and F1 with both averages. In GridSearchCV, scoring="f1_macro" or "balanced_accuracy" (macro recall) keeps rare classes visible during tuning. Ask: which average would the always-shale model do better on? Weighted, since shale is 38% of samples. [2 min]

A pipeline keeps training steps together

EVALUATING AND SELECTING MODELS

A pipeline keeps training steps together

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

model = make_pipeline(
    SimpleImputer(strategy="median", add_indicator=True),
    StandardScaler(),
    SVC(kernel="rbf"),
)
```

Every cross-validation fit learns fresh imputation and scaling statistics.

Machine Learning / Module 1 · Supervised learning foundations 86

This is the fold diagram from part 3 in code. make_pipeline chains the steps; calling fit on the pipeline fits each step on the training rows in order, and calling predict applies the fitted steps to new rows. When the pipeline is passed to cross_val_score or GridSearchCV, every fold refits the imputer, the scaler and the model on that fold's training rows only.

X must contain numeric predictor columns for this example. Categorical columns such as formation need a ColumnTransformer that applies OneHotEncoder to them and the numeric steps to the rest; Module 2 builds that. Feature selection, if used, also belongs inside the pipeline.

Pipelines also make deployment safer: the same object carries the medians, the scaling statistics and the model, so new wells are processed exactly like the training wells. This is a focused fragment; data loading is omitted. [2 min]

Reference: https://scikit-learn.org/stable/common_pitfalls.html

Grid search scores every setting with grouped CV

EVALUATING AND SELECTING MODELS

Grid search scores every setting with grouped CV

16 settings × 3 grouped folds = 48 fits and one refit; the test wells are opened once.



Synthetic 8-well facies data (seed 7): GridSearchCV with GroupKFold(3) on wells W1–W6; W7–W8 as the test set.

Machine Learning / Module 1 · Supervised learning foundations 87

Walk the procedure on the left. The grid lists every combination of four C values and four γ values. For each of the 16 settings, grouped 3-fold cross-validation on the six training wells gives a mean macro F1. The best mean wins: C = 10, γ = 0.01, with a CV macro F1 of 0.75. GridSearchCV then refits that setting on all six training wells, and only this locked pipeline is scored on the test wells: macro F1 0.76.

The heatmap shows the landscape. Large γ (10) fails everywhere, as the gamma slide predicted. Two other settings are within 0.01 of the best (C = 1 with γ = 0.1, and C = 100 with γ = 0.01); with only three folds of two wells each, such differences are noise, and choosing the more regularized setting in that plateau is defensible.

That the test score (0.76) happens to be close to the CV score is reassuring but not guaranteed; with two test wells it is one noisy number. Module 5 covers randomized and successive-halving searches and nested cross-validation for an unbiased estimate of the whole tuning procedure. [3 min]

Grouped tuning respects well boundaries

EVALUATING AND SELECTING MODELS

Grouped tuning respects well boundaries

```python
from sklearn.model_selection import GroupKFold, GridSearchCV

search = GridSearchCV(
    model,
    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]},
    cv=GroupKFold(n_splits=3), scoring="f1_macro",
)
search.fit(X_train, y_train, groups=well_train)
print(search.best_params_, round(search.best_score_, 2))
```

OUTPUT

```
{'svc__C': 10, 'svc__gamma': 0.01} 0.75
```

Run on training wells W1–W6 of the synthetic data; W7 and W8 stay closed.

This is the code behind the heatmap. model is the pipeline from the previous slide; parameter names use the step name and a double underscore, so svc__C sets C inside the SVC step. GroupKFold keeps each well in one fold, and passing groups=well_train to fit is essential: without it the splitter has nothing to group by, and scikit-learn raises an error for GroupKFold rather than silently mixing wells.

The printed result comes from running exactly this search on wells W1–W6 of the synthetic facies data. With three folds, each fold holds two wells; require at least as many training wells as folds.

GroupKFold does not balance classes; with few wells, a rare facies can be absent from a validation fold. StratifiedGroupKFold helps where feasible, but inspect the actual folds and their class counts either way. After the search, evaluate search.best_estimator_ once on the test wells and report that score with the fold scores. [2 min]

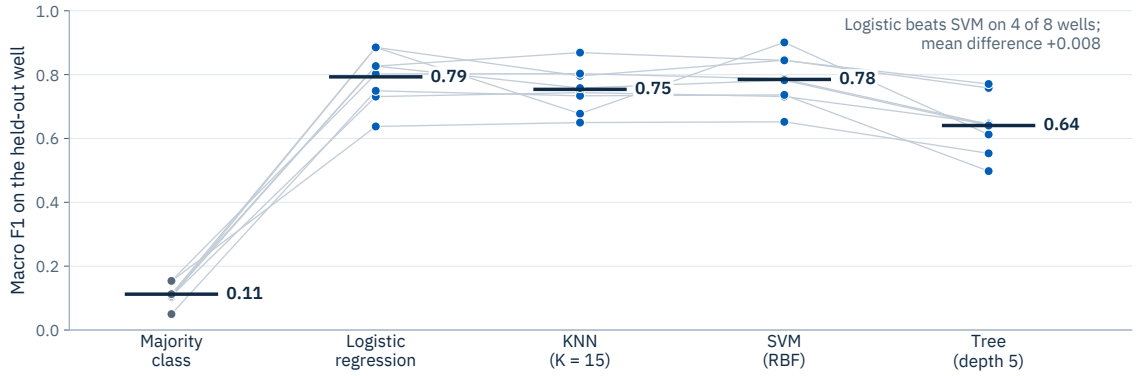
Reference: https://scikit-learn.org/stable/modules/cross_validation.html

Compare models by their fold-score distributions

EVALUATING AND SELECTING MODELS

Compare models by their fold-score distributions

Same eight held-out wells for every pipeline: read the spread and the paired lines.



Synthetic 8-well facies data (seed 7); leave-one-well-out macro F1 for five pipelines on identical folds. Grey lines join scores of one well.

Machine Learning / Module 1 · Supervised learning foundations 89

Each dot is one held-out well; the black bar is the mean over eight wells. The majority-class baseline scores 0.11 in macro F1, the floor. Logistic regression averages 0.79, the RBF SVM 0.78, KNN with K = 15 0.75, and a depth-5 tree 0.64.

Now read the grey lines, which connect the same well across models. Well-to-well differences (roughly 0.64 to 0.90 for the same model) are larger than the differences between the better models. Logistic regression beats the SVM on 4 of 8 wells; the mean difference is +0.008. That is no evidence that either is better. The tree, by contrast, is below logistic regression on every well.

Because all models share the folds, paired comparisons are fair; comparing models evaluated on different folds is not. Module 5 adds statistical tests for such paired fold scores; today's habit is to show all fold scores, never just a table of means. [3 min]

Model comparison is a controlled experiment

EVALUATING AND SELECTING MODELS

Model comparison is a controlled experiment

Candidate	Strong starting point when	Main check
Linear or logistic	Effects are roughly additive	Residual or probability structure
KNN	Local similarity is meaningful	Scaling and prediction cost
SVM	A margin or kernel helps	C, γ and calibration
Tree	Threshold interactions are plausible	Leaf size and stability

Hold the split, folds and metric fixed; the baseline row comes first.

◆ Pair every score with an error review and the runtime at the point of use.

Machine Learning / Module 1 · Supervised learning foundations 90

There is no unconditional winner. Each family has a situation where it is a strong first choice and a characteristic check. Linear and logistic models are fast, interpretable and hard to beat when effects are roughly additive, as the comparison just showed. KNN works when local similarity in a well-scaled space is meaningful, at a prediction cost. SVMs help when a margin or kernel captures the shape of the boundary, but need scaling, tuning of C and γ, and separate calibration for probabilities. Trees capture thresholds and interactions without scaling, at the price of instability.

Run comparisons as controlled experiments: same split, same folds, same metric, same preprocessing where applicable, baseline first. Include a domain review of the errors and the inference constraints, for example latency and missing-data behaviour at the wellsite.

Ensembles in Module 2 combine several of these ideas. [2 min]

Lab 3: a tuned pipeline and a locked test

LABORATORY

25 min

Lab 3: a tuned pipeline and a locked test

- ☐ §1: explain why PCA fitted on all rows leaks information
- ☐ §2–3: fit the 2-D teaching models and record the retained variance
- ☐ §4: run the CV grid search; record the kernel, C and γ it selects
- ☐ §5: add PCA inside the pipeline; compare 2, 10 and 30 components
- ☐ §6: set RUN_FINAL = True once, after choosing, and report that score

NOTEBOOK

`exercises/03_svm/SVM_Exercise_Student.ipynb`

EVIDENCE TO SUBMIT

- What each partition was used for
- A matched comparison table
- One final score with its caveat

Machine Learning / Module 1 · Supervised learning foundations 91

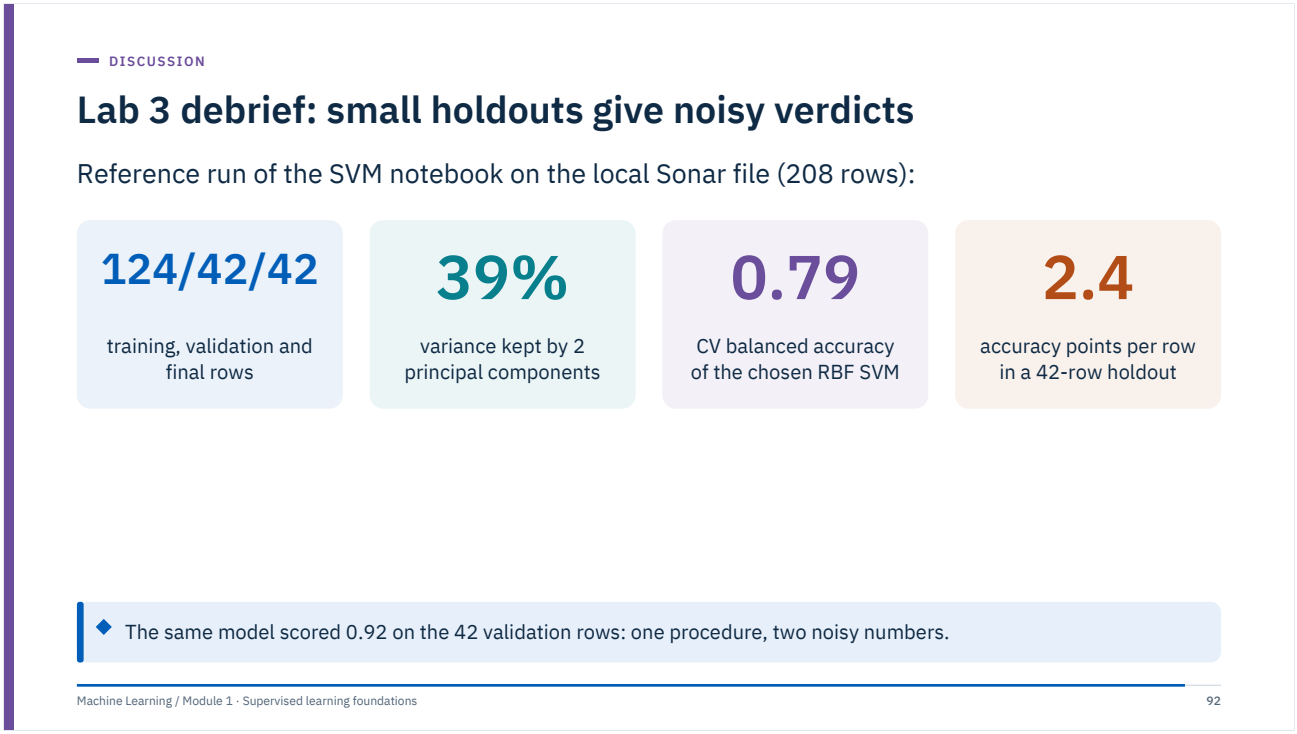
Timing: 25 minutes, then a 5-minute debrief. The notebook uses the local Sonar data (208 acoustic returns, 60 features, rocks versus mines); no download needed. It creates three partitions before any fitting: training, validation and a reserved final holdout.

Section 1 answer: PCA does not use labels, but fitted on all rows it learns directions from the validation and final rows, so they are no longer unseen. In section 5, PCA must go inside the pipeline so that each fold refits it; keep the same cv object for every variant so the comparison is matched.

The notebook states that row holdouts are a classroom fallback because acquisition grouping is not supplied. Ask learners to say what grouping they would want if the data came from several survey campaigns.

Extension for fast finishers: the KNN-from-scratch notebook (exercises/02_knn_from_scratch), which rebuilds distances, neighbour search and voting in NumPy and checks the result against scikit-learn. Common error: rerunning the final cell after changing the model.

Lab 3 debrief: small holdouts give noisy verdicts

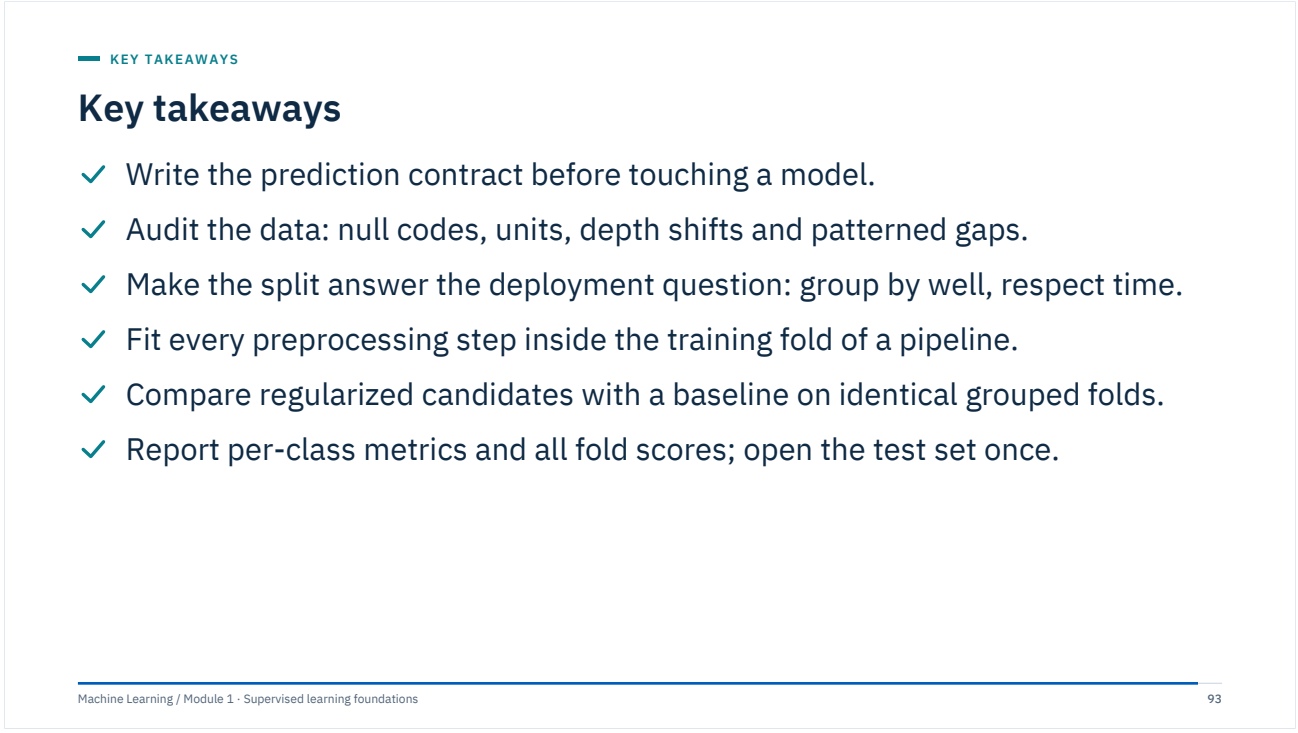


Reference values from the solution notebook (scikit-learn 1.9; 1.7 gives the same); learners with these versions should see the same values. The split gives 124 training, 42 validation and 42 final rows. Two principal components keep only 39% of the variance, so the 2-D plots are a teaching view, not the model: the 2-D validation accuracies were 0.69 (linear), 0.64 (polynomial) and 0.76 (RBF).

The grid search selects an RBF kernel with C = 10 and gamma "scale", with a cross-validated balanced accuracy of 0.79 on the training rows. On the 42 validation rows the same model reaches a balanced accuracy of 0.92. Neither number is wrong: each row of a 42-row set is worth 2.4 accuracy points, so small sets swing widely.

In a reference implementation of section 5 (pca__n_components added to the section 4 grid, best setting per component count), PCA inside the pipeline gave CV balanced accuracy 0.59 with 2 components, 0.75 with 10 and 0.80 with 30: dimensionality reduction removed useful signal here. Learners' implementations may differ slightly; the lesson is the matched comparison and the single, final use of the reserved rows.

Key takeaways



Read the six statements and connect each to a moment of the day: the contract timeline, the log summary exercise, the motivating case, the fold diagram, the model-comparison plot and the confusion matrices.

Ask each learner to pick the one habit they will apply first in their own project and to write it on the exit card. Collect a few aloud.

Bridge to Module 2: ensembles (random forests, gradient boosting), model interpretation and full ColumnTransformer pipelines build directly on today's split and pipeline habits. [3 min]

Exit check: four questions before you leave

PRACTICE

5 min · Individual

Exit check: four questions before you leave

1. Why must scaling run inside a cross-validation pipeline?
2. When is a random row split misleading for well logs?
3. A shale sample was predicted at p = 0.1. What is its log loss, and why is it large?
4. What evidence would justify a more complex model?

Machine Learning / Module 1 · Supervised learning foundations 94

Collect written answers on cards [5 min]; review weak points at the start of Module 2.

Expected answers. (1) Otherwise the scaling statistics include the validation rows, which then are not unseen; the pipeline refits the scaler on each training fold. (2) Whenever neighbouring samples or the same wells appear in both training and validation, because depth samples are correlated and wells differ; use grouped folds for new-well questions. (3) $-\ln 0.1 \approx 2.30$, about 22 times the loss of a confident correct prediction at 0.9, because log loss punishes confident mistakes. (4) A consistent improvement over the baseline and simpler candidates on the deployment-relevant folds, visible across most folds rather than only in the mean, with acceptable error patterns, runtime and maintainability.

Common weak answer to watch for in (4): "a higher accuracy". Probe for which folds, which metric and which baseline.

References and further study

FURTHER STUDY

References and further study

- scikit-learn User Guide: linear models, neighbours, SVM, trees, cross-validation and common pitfalls.
- James, Witten, Hastie, Tibshirani: *An Introduction to Statistical Learning*; Hastie, Tibshirani, Friedman: *The Elements of Statistical Learning*.
- Kaufman, Rosset, Perlich, Stitelman (2012). Leakage in data mining: formulation, detection, and avoidance. *ACM TKDD* 6(4).
- Roberts et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography* 40.
- Hall (2016). Facies classification using machine learning. *The Leading Edge* 35(10).

Use the official scikit-learn documentation for parameter definitions and current defaults; the user guide sections on cross-validation and on common pitfalls match today's evaluation habits closely.

An Introduction to Statistical Learning covers regression, classification, resampling and regularization at the level of this module, with free online editions; The Elements of Statistical Learning goes deeper, including the bias–variance decomposition and the correct and incorrect ways to do cross-validation.

Kaufman and colleagues give a formal treatment of leakage with many practical examples. Roberts and colleagues review blocked and grouped cross-validation for structured data; although written for ecology, it maps directly onto wells and time series. Hall's tutorial introduced a facies classification dataset from the Hugoton field, related to the lab data.

All plots in this module are original computed teaching examples on synthetic data or clearly labelled schematics.

Reference: https://scikit-learn.org/stable/user_guide.html ; https://scikit-learn.org/stable/common_pitfalls.html ; https://www.statlearning.com/ ; https://hastie.su.domains/ElemStatLearn/